

FEDERAL STATE AUTONOMOUS
EDUCATIONAL INSTITUTION
OF HIGHER EDUCATION
NATIONAL RESEARCH UNIVERSITY
“HIGHER SCHOOL OF ECONOMICS”

FACULTY OF COMPUTER SCIENCE

Deltsov Daniil Andreevich

Self-Supervised Learning for Autonomous Driving
MASTER’S DISSERTATION

Field of study: 01.04.02 «Applied Mathematics and Informatics»

Educational program «Master of Data Science»

Student Deltsov D.A.
Supervisor Sanochkin Y.I.
Advisor Pravdivets N.A.

CONTENTS

Аннотация	5
Abstract.....	6
Ключевые слова	7
Introduction	8
1 Theoretical analysis	10
1.1 Literature review	10
1.2 Theoretical analysis of SSL.....	23
1.2.1 Core challenges in self-supervised learning for autonomous driving	23
1.2.2 Current approaches in self-supervised learning for autonomous perception	24
1.2.3 Limitations of existing self-supervised approaches for autonomous driving.....	26
1.2.4 Developing solutions and future directions in self-supervised autonomous driving	28
1.2.5 Concluded analysis of self-supervised learning in autonomous driving	30
1.3 Problem statement	31
1.3.1 Problem definition in autonomous driving vehicles field	31
1.3.2 The importance of a reliable SSL solution for autonomous driving vehicles	32
1.3.3 Evaluation metrics for SSL performance under real-time constraints in autonomous driving	34
1.3.4 Key stakeholders impacted by developing SSL techniques for autonomous driving vehicles	36
1.3.5 Proposed methodological framework for self-supervised learning in autonomous driving	39
2 Dataset, methods and experimental setup	43
2.1 Chapter overview	43
2.2 Dataset: Cityscapes.....	44
2.3 Exploratory data analysis	47
2.3.1 Image-level statistics	47
2.3.2 Class distribution	48
2.3.3 Geographic distribution	51
2.3.4 Qualitative inspection.....	52
2.4 Pre-training methods	55
2.4.1 Random initialization	56
2.4.2 Supervised ImageNet pre-training.....	56

2.4.3	MoCo v3.....	58
2.4.4	DINO	60
2.5	Downstream architecture: segmentation head.....	63
2.5.1	Downstream design	63
2.6	Training protocol.....	65
2.6.1	The two evaluation modes.....	65
2.6.2	Hyperparameters	66
2.6.3	Hardware and runtime	67
2.6.4	Image preprocessing pipeline.....	67
2.7	Evaluation metrics	68
2.7.1	Mean Intersection over Union	68
2.7.2	Per-class IoU on rare classes	69
2.7.3	Probe versus fine-tune delta	69
2.7.4	Inference frames per second.....	70
2.8	Reproducibility	70
2.9	Fine-tuning the best backbone (DINO ViT-S/16).....	71
2.9.1	Reasoning and overview of the approaches	71
2.9.2	Multi-block DPT-Lite segmentation head.....	72
2.9.3	Auxiliary supervision head.....	74
2.9.4	Compound loss	74
2.9.5	Optimizer and layer-wise learning rate decay	76
2.9.6	Mixed precision and exponential moving average.....	78
2.9.7	Two-resolution evaluation and test-time augmentation	79
2.9.8	Training duration and runtime.....	80
2.9.9	Hyperparameter contrast with the controlled comparison	80
2.10	Experiments methodology summary	82
3	Experiments, results and conclusions.....	83
3.1	Chapter overview	83
3.2	Quantitative results.....	84
3.2.1	Headline mIoU on the controlled comparison	84
3.2.2	Training dynamics.....	85
3.2.3	Per-class IoU on the controlled comparison.....	86
3.2.4	Probe versus fine tune delta.....	88
3.2.5	Inference speed and real-time feasibility.....	89
3.2.6	DINO fine-tuning	90

3.3	Qualitative results.....	92
3.3.1	Side by side prediction comparisons.....	92
3.3.2	Failure case analysis.....	94
3.3.3	DINO self-attention visualization	95
3.4	Interpretation of results	95
3.4.1	SSL performance versus supervised pre-training.....	96
3.4.2	Long-tail hypothesis analysis	97
3.4.3	Probe versus fine-tune results.....	98
3.4.4	Architecture difference between controlled experiment and fine-tuning.....	99
3.4.5	Limitations of the study.....	100
3.5	Business application and stakeholder impact.....	102
3.5.1	The labelling-cost problem.....	102
3.5.2	Where SSL pre-training cuts the cost.....	103
3.5.3	Stakeholder breakdown	104
3.5.4	Deployment scenario	107
3.6	Conclusions	108
3.6.1	Summary of contributions	108
3.6.2	Future work	110
	References	114
	Appendix A. Git repository and structure	123
	Appendix B. Abbreviations	124

АННОТАЦИЯ

Самообучение (self-supervised learning, SSL) рассматривается в работе как способ снизить зависимость задач сегментации городских сцен от больших размеченных наборов данных. В рамках работы проводится сравнительная оценка четырёх стратегий предобучения свёрточных и трансформерных энкодеров для задачи семантической сегментации сцен автономного вождения на наборе данных Cityscapes: случайной инициализации, контролируемого предобучения на ImageNet, MoCo v3 на ImageNet и DINO на ImageNet. Каждая стратегия проверяется в двух режимах: линейный пробинг с замороженным энкодером и полная дообучаемая модель. В качестве основной метрики используется средний IoU по классам (mIoU); дополнительно отслеживается IoU по семи редким классам (мотоцикл, грузовик, поезд и т. п.) и метрика разрыва probe-vs-fine-tune, отражающая насколько признаки энкодера готовы к задаче плотного предсказания без дополнительной адаптации. Эксперименты выполнены на полном обучающем разделе Cityscapes (2975 изображений) и на полном валидационном разделе (500 изображений). Работа также включает раздел о бизнес-применении результатов: оценку влияния SSL-предобучения на стоимость разметки данных, разбор стейкхолдеров отрасли (роботакси, OEM-производители, доставочные роботы, симуляторы, страховщики) и обсуждение ограничений практического применения. Полученные результаты позволяют количественно ответить на вопрос: дают ли стратегии SSL-предобучения признаки, которые лучше переносятся на семантическую сегментацию городских сцен, чем контролируемые базовые модели, и какой из методов SSL показывает лучший результат в данной постановке.

ABSTRACT

Self-supervised learning (SSL) is considered in this thesis to reduce dependence on large, labeled datasets for urban scene segmentation. The thesis presents a comparative evaluation of four pre-training strategies for the encoder backbones used in semantic segmentation of autonomous driving scenes on Cityscapes: random initialisation, supervised pre-training on ImageNet, MoCo v3 pre-trained on ImageNet, and DINO pre-trained on ImageNet. Each strategy is examined in two evaluation modes: linear probing with the encoder frozen, and full fine-tuning. The primary evaluation metric is mean intersection over union (mIoU) computed across the 19 Cityscapes classes; additionally per-class IoU is reported on seven rare classes (motorcycle, truck, train and others) which represent less than one percent of pixel coverage, and a derived probe versus fine-tune delta is used to estimate how well the pre-trained features are aligned with the dense prediction task without additional adaptation. Experiments are conducted on the full Cityscapes training split (2975 images) and the full validation split (500 images), making the results directly comparable to existing literature. The thesis also includes a section on business application of the results: an estimation of the impact of SSL pre-training on labeling cost, a breakdown of relevant stakeholders in the autonomous driving industry (robotaxi operators, OEMs, delivery robot startups, simulation vendors, insurers), and a discussion of limitations of practical use. The results provide a quantitative answer to the question whether self-supervised pre-training strategies produce feature representations that transfer more effectively to semantic segmentation in urban driving scenes than supervised baselines, and which SSL method performs best in this setting.

КЛЮЧЕВЫЕ СЛОВА

самообучение, автономное вождение, семантическая сегментация, Cityscapes, MoCo v3, DINO, перенос обучения, длинный хвост, mIoU, Vision Transformer

INTRODUCTION

With the rapid development of technology, especially the constant research conducted in Machine learning and Deep learning models (neural networks) which are the current focus of the research field, new approaches regularly introduced to solve complex, previously impossible problems. DL models specified in computer vision can be trained both on labeled and non-labeled, called supervised and unsupervised learning respectfully. Since labeling data, especially image or video data can be time consuming and high labor-intensive if done manually [1]. Self-supervised learning can be a solution to the complexity of data labeling and annotation. Nowadays, SSL methods are used in a high demand field of autonomous driving systems like self-driving delivery robots, vehicles with autopilot and self-navigating systems.

Self-supervised learning means assigning labels to data automatically before training the model. The idea in the algorithm implies that while the input is provided, some parts of it are hidden, and non-hidden, visible parts are used to predict the hidden ones. The difference between supervised learning and SSL is the fact that it both require labeled data, but SSL specifically does not require human-labeled data, all labels are assigned automatically [2]. An SSL model consists of two parts, where the first part is a pretext task, and the second one is contrastive learning [3]. In the first part of a model generic pseudo labels are generated and assigned to data entries based on dataset structure and its features, which are later used in the second part of a model for detection, segmentation or classification of objects on an image [4].

Autonomous driving vehicles are a trending technology which can increase efficiency of transportation, delivery and driving in general; with a potential to reduce number of traffic incidents and reduce costs in general. Nevertheless, it requires a huge amount of labeled data which is a high labor-intensive process [1]; though SSL methods are aimed to help solve this problem. **The subject of the study** is Self-supervised learning techniques applied to object detection and segmentation in autonomous driving systems. Object detection and segmentation is a key principle of self-driving systems which are aimed to help vehicles navigate the terrain while locating traffic lines, pedestrians, other vehicles and road signs with traffic lights. **The object of the study** is the process of object detection in autonomous driving systems.

Supervised learning is a traditional approach used in image and video segmentation which requires extensive human labor and time to do data labeling and annotation, being expensive and time consuming for large scale applications and systems. The number of labels and information to annotate within one image of a street obtained from a driving vehicle can be too extensive to do manually, especially when it comes to training a model. Labeling enough data will come at a great cost of human resources and time. **The relevance of this study** lies in the growing need for scalable and efficient machine learning solutions in autonomous driving. This comes with SSL models being an alternative to

expensive labor while automatically assigning classes and annotations to objects on an image, reducing the dependency on manually labeled datasets.

The goal of the study is to evaluate and analyze existing state of the art SSL methods and technologies for object detection and segmentation for autonomous driving. This implies identifying the most effective state of the art approaches, evaluating and analyzing their benchmarks, analyzing the datasets, identifying strong and weak points in each approach and comparing them. To achieve the goal, **the tasks of the paper** listed should be accomplished:

1. Identifying and reviewing existing state of the art approaches and methods in SSL for object detection and segmentation.
2. Analyzing key strong points and weaknesses of SSL algorithms.
3. Evaluating models' performance using provided benchmarks on autonomous driving datasets.
4. Identifying key challenges and possible improvement directions for future research and development.

Addressing the following tasks of the study, the paper aims to contribute to SSL research and development in autonomous driving vehicles.

1 THEORETICAL ANALYSIS

1.1 Literature review

Self-supervised learning can be considered a state between supervised learning and unsupervised learning because an SSL model is trained using some portion of labeled input data to predict other portion of hidden, unlabeled data, which is accomplished by autogenerating label feature from unlabeled data with some pseudo-labels. This labeling and training process converts unsupervised model to a supervised one. The process of training is shown on figure 1.

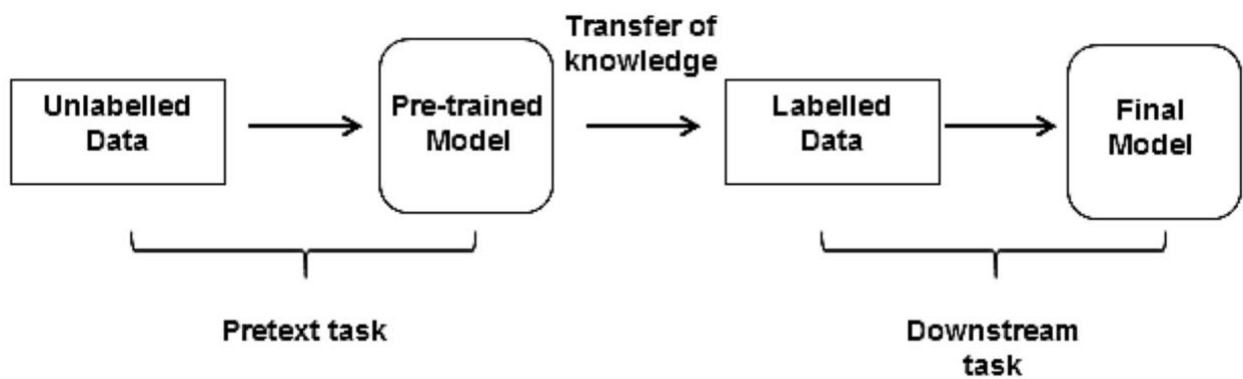


Figure 1. Process of SSL model training [5]

The key feature of SSL being different from other learning techniques is the fact that it does not need any manually labeled or annotated data. SSL learning consists of two main parts: pretext tasks and contrastive learning [3]. Figure 2 illustrates the most common problems SSL is used to work with where left side represents pretext task problems and right side – contrastive problems.

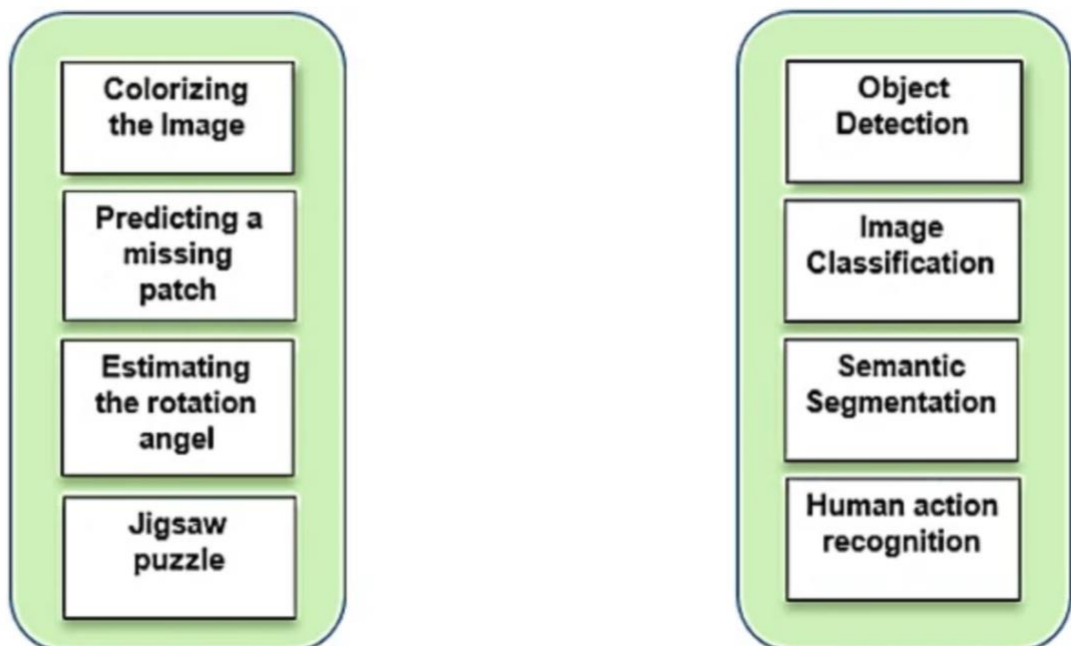


Figure 2. Most common pretext and contrastive learning problems [5]

Convolution neural networks became a ground base foundation for SSL approaches with its neural network architecture influencing the quality and precision of images and visual representations via pretext training task. SSL learning is mostly divided into two buckets: contrastive learning and non-contrastive learning, both occur after pretext task completion.

Contrastive learning is designed to learn and features of an object and then detect features of a similar object which is used in a wide range of applications like NLP and CV. In NLP to change the semantics of a sentence or a phrase, one single word can be changed in position, hence the goal of contrastive learning is to bring semantically related objects/samples closer to each other and keeping not similar objects/samples separate [6]. Contrastive learning takes its origin from the way people learn – people can distinguish different objects without the need to remember every single detail about them. So, in contrastive learning one sample from training data is taken as a starting point which augmented form is represented as a positive sample, while the rest of the data is labeled as a negative data sample. The process is depicted on figure 3, which consists of a set of unlabeled data, augmentation part, an encoder like ConvNet or CNN, and a pretext task. Image augmentation is used to bring stochastic transformations to each object, such as color palette transformations, shape and size modifications, image rotation and flipping. All these steps are needed to change image's visual representation without changing its semantics. Augmented objects are passed into an encoder which detects features and extracts them, the more layers the network has, the better the quality of extracted features will be. Encoders used in contrastive learning: momentum encoders, image encoders, dictionaries [7]. From the extracted features pseudo labels are generated during pretext task to further predict unlabeled data.

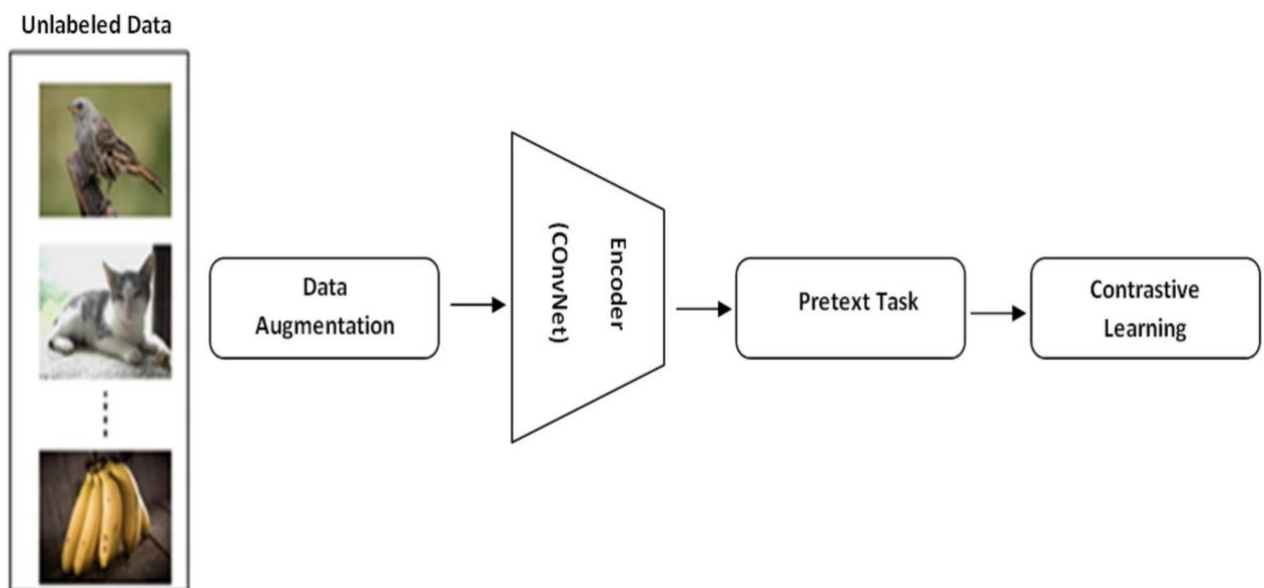


Figure 3. Process of contrastive learning [5]

Non-contrastive learning is different from contrastive learning by only relying on positive samples in training data – this means that all images in any training batch will only contain desired target object. For instance, if the training is done to detect cats, then training batch will consist of only cats’ images and their augmented forms, no other non-target objects will be presented. Non-contrastive SSL models use stop-gradient and extra-predictor to achieve better learning results, since it does not include non-target objects in training samples, hence these models do not suffer from dimensional collapse. In table 1 comparison of contrastive and non-contrastive learning is presented.

Table 1. Comparison of contrastive and non-contrastive learning techniques [5]

№	Feature name	Contrastive learning	Non-contrastive learning
1	Complexity of NN model	Complex models	Much simpler models
2	Suffering from dimensionality collapse	Prone to dimensional collapse	Lesser dimensional collapse
3	Data samples labels	Positive and negative samples	Positive samples
4	Semantics label distance	Minimize distance between positive samples, maximize between negative	Leverage positive data labels distance

SSL models can perform various tasks, for instance pretext tasks: image colorization, predicting a missing patch, estimating rotation angle, jigsaw puzzles; downstream tasks: image classification, semantic segmentation, human action recognition and many more. Some of these tasks combined are used for autonomous driving.

Image colorization is a task of converting grey-scale image to a colorized image. Each pixel from an image is going through a pretext stage to help the model learn patterns and features. Treneska S. et al. used image colorization model based on Generative Adversarial Network with SSL methods which can produce the most realistic results [8]. Each extracted feature from an image and a pixel was passed to downstream task after applying the GAN model, specifically multi-label image classification and semantic segmentation and semantic labels were assigned to each pixel of an image in semantic segmentation part of the training. A single image was assigned to multiple labels in multi-label classification part of the model training. In the paper 11987 training images were used and tested on 1713 test sample images by the authors. The research was conducted on PascalVOC 2012 dataset. The authors state that based on their research self-supervised learning based on visual feature learning by using GANs on the image colorization task is worthy of further exploration and has potential.

Predicting a missing patch is a pretext task that aims to predict a position of a missing patch on an image; to achieve that, a model is trained accordingly. Doersch C. et al. proposed using SSL to predict a missing patch of an image [9]. In the research the authors picked a random part of an image and

predicted relative position of second and central patches. The patches were numbered from 1 to 8, and after selecting a random image patch, each part was passed into a CNN with AlexNet architecture. In the architecture weights were shared between layers and a gap was added aiming to prevent model over-fitting. To predict relative position of each path softmax was used as a final network layer.

Estimating the rotation angle of an image or an object on an image is a task which requires instance discrimination [10]. Predicting rotation of a picture is an effective method though simple for identifying rotation discriminative features. NN is trained on pairs of rotated images of an unlabeled dataset and an image then is rotated and passed into CNN model. Rotation is usually 0, 90, 180 and 270 degrees. The model aims to understand the location, position or pose, and type of an object depicted to understand the rotation angle of it. Devgon S. et al. developed an SSL model to identify and estimate object rotation angle on an image between the desired and current object's location [11]. The authors of the paper used the model to identify the rotation angle between two depth images.

Jigsaw puzzle is a task that requires not only having information and features extracted from a single patch but also knowledge on the relations between different patches and image in general. Understanding relationship between patches and their biased features help in solving the jigsaw puzzle. Since the model cannot use any formula or direct method to instantly find the right position for the patch to be inserted, it must try all possible outcomes, hence multiple permutation functions are generated to find the right position for each patch [12]. Li R. et al. researched on solving jigsaw puzzles via using GAN models [13]. To achieve this, a multitasking pipeline was created for solving unpaired jigsaw puzzles, where all possible permutations of patch locations were classified into a separate branch and then GAN model was used to extract separate features and reconstruct an image in proper patch order. This separate branch included pseudo labels generated via shuffling image patches, having the GAN model iterate over them and extract semantic information of each piece.

Downstream task is a task that occurs after pretext task and defines model's purpose by itself [14]. While doing the pretext task, the model learns and extracts various useful features to help it accomplish downstream task like classification or object detection with insufficient labels, action recognition or semantic segmentation. The process of a downstream task can be done in two ways: either model fine-tuning or using a linear classifier.

Image classification is a process detecting and classifying each objects class on an image which has a variety of popular NN developed for this exact purpose, for instance: AlexNet, ConvNet, ResNet, DenseNet, GoogLeNet, VGG and others [15]. While one image may include multiple objects and classes, only one class is used as a target feature to be classified on it. SSL model is used to extract any features of an object from an image, based on an SSL model a classifier then trained to perform final classifying task – downstream task. Image classification may also include image object detection to extract not only object's features but also its relative position which can be important in applied computer

vision applications like autonomous driving, robotics [16]. The most common datasets for training and evaluation model's performance for the following of object detection task are MOSOCO and OpenImage, the most popular and high performing models are ConvNet models like CNN, R-CNN, Fast-RCNN [17], and YOLO models [18].

Semantic segmentation is a process of assigning labels not for each object but for each pixel on an image which makes each pixel belong to some group of other pixels, eventually masking the whole object and dividing it from others. The following task is also used in autonomous driving and robotics just like image classification and object detection with some popular models as: VGG, ResNet, AlexNet, CNN and Fully Connected Network.

Human Action Recognition is a process consisting of detecting people on an image or video and then recognizing – classifying the type of activity they are doing. This task makes the use of spatial and temporal features necessary, making possible to build reliable recognition systems with few labeled training samples in the dataset by relying on SSL techniques [19]. You W. et al. suggested using view-enhanced jigsaw puzzle to recognize 3D human actions by capturing multi-angle view data and making the encoder independent high-level features from various angles in 3D space, tracking human skeleton.

SSL in computer vision is commonly used today for object detection and classification, human action recognition, graph classification, real time object tracking and so on. For classification problems mostly two approaches are used: one-stage and two-stage detectors. One-stage detector implies that classification and bounding boxes of an object are done on the same stage while two-stage classification uses two steps where on the first object is detected and on the second it is classified to a proper label. Since the idea behind SSL is to build a model that can perform a task and give some output without a need to manually label huge datasets, it is becoming more popular in computer vision. Gidaris S. et al. studied on ConvNet based architectures and methods to recognize 2D rotations on an image [20]. Huang G. et al. suggested that SSL should be used for few shots object detection and instance segmentation which is based on few labels helping the model to understand never-seen classes previously [21]. This is a proper fit for SSL since it resides around the idea that it does not need time consuming manual labelling. The authors compared multiple object detection benchmark datasets and evaluated the models with DETReg being the top performing one. Liu S. et al. Proposed using Self-EMD method for learning spatial-visual representations [22]. The authors took multiple set of images from various perspectives, cropped them to 224x224 and augmented them with random Gaussian blur, color distortion, random horizontal flipping. The actions were performed on COCO dataset. Amrani E. et al. proposed using SSL methods for detecting and classifying an object from an unlabeled video sample [23]. The model evaluated the video frame by frame and captured similar frames by a common theme composition. Contrastive learning was used for background elimination and new clusters of data were formed by a high level of label noise.

Florent C. et al. studied the integration of SSL techniques in autonomous driving perception using both analytical and learning methods – hand-crafting features and deep learning models to try to eliminate the need of large manually labeled dataset and enhance model’s ability to adapt to unexpected events [24]. The authors researched that an approach of using SSL methods can benefit from using automatically labeled data from additional sensors such as LIDAR and stereo-cameras which helps to understand features on the road better – dynamic object detection, depth estimation, road segmentation. The authors also highlighted the importance of online learning and self-improvement in SSL systems which allows a model to adapt to dynamic environment and improve performance over time. SSL models for road segmentation demonstrated effective performance on both off-road and urban environments based on authors’ research. It was stated that SSL can reduce the need of manually labeled large datasets while maintaining high accuracy [24].

The challenge of video class agnostic segmentation in autonomous driving was addressed by Mennatullah S. et al. with the goal to successfully segment both known and unknown objects – the ones that were not presented in the training dataset [25]. The authors suggested using contrastive learning approach with semantic and temporal guidance to help the model better detect and segment unknown objects in video sequence samples. The proposed method included pixel-wise semantic and temporal consistency to improve the difference between known and unknown objects which is important in autonomous driving – model needs to react to unknown objects and obstacles for safety reasons. Semantic guidance means using prototype-level contrastive learning where class signatures and features are extracted using masked average pooling; and to improve the accuracy in segmentation between known and unknown objects contrastive loss is applied to the model. Temporal guidance implies that temporal consistency is maintained between frames in a video sequence sample, it uses optical flow to transfer representations of one object between frames and applies contrastive loss to aligned regions. The authors also release a large-scale synthetic dataset generated in the CARLA simulation environment, which includes diverse and rare unknown objects to evaluate the proposed method. The research was conducted on the Cityscapes-VPS and synthetic CARLA datasets. The researched demonstrated that the proposed methods enhance segmentation performance of class-agnostic IoU of unknown objects and proved being effective in small datasets with not much labeled objects [25].

One of key problems in SSL algorithms in autonomous driving is annotating and labeling 3D LiDAR data, which is extremely time consuming, so Corentin S. et al. introduced Superpixel-driven LiDAR Representations which is based on SSL method for pre-training 3D LiDAR NN by accumulating and distilling knowledge from SSL pre-trained models on 2D image NNs [26]. The main goal of the paper the authors aimed to achieve is to improve 3D semantic segmentation and object detection with the use of LiDAR technologies but without having to annotate the data manually. The proposed method uses synchronized videos of camera capture and LiDAR data to align 3D point features with 2D image

features using a superpixel-driven contrastive loss, which groups visually similar regions in both modalities. The process can be seen on figure 4.

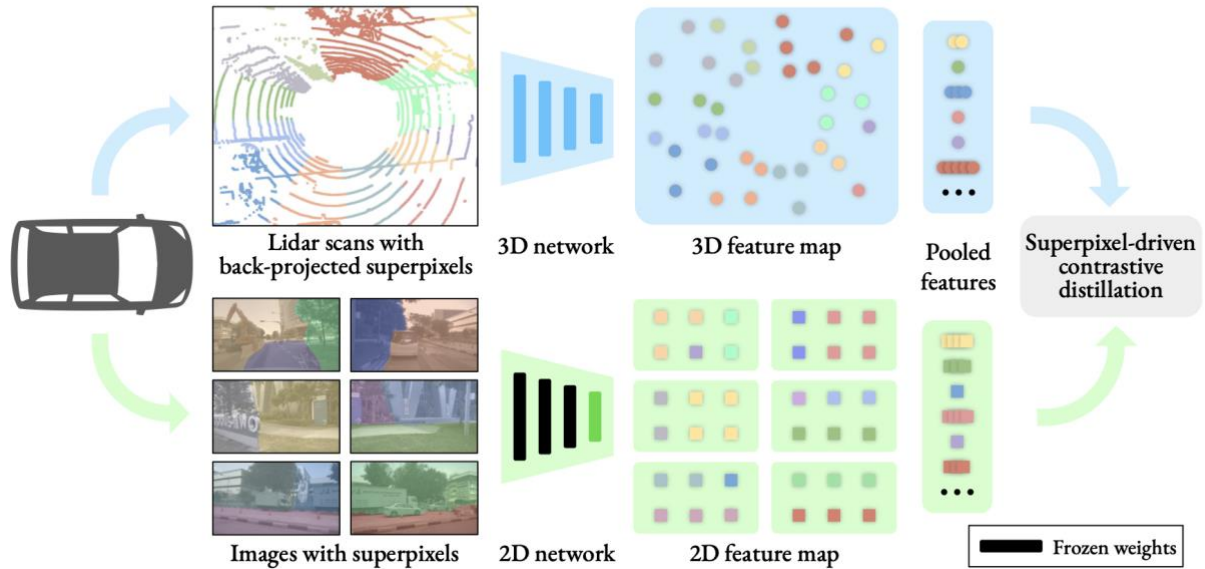


Figure 4. Superpixel-driven LiDAR Representations training algorithm [26]

The authors developed and proposed a contrastive distillation loss that operates at superpixel level and not on pixel, scene level [26]. Superpixels are used to group pixels and points into visually similar regions, which are likely to belong to the same object. The contrastive loss aligns the pooled features of those regions across 2D images and 3D LiDAR points sample. The proposed approach eliminated occlusions, motion artifacts, and density variations in LiDAR point cloud issues. The authors also introduced image projection head to make transfer of features from 2D space to 3D space meaningful. The datasets used by authors of the research were nuScenes for semantic segmentation and SemanticKITTI for object detection. The results proved that SLidR outperforms existing self-supervised pre-trained models, including PointContrast and DepthContrast, especially in few-shot learning scenarios where only a small fraction of annotated data is available. SLidR achieves significant improvements in mean Intersection over Union (mIoU) for semantic segmentation and mean Average Precision (mAP) for object detection, demonstrating its ability to transfer knowledge effectively from 2D to 3D representations.

Motion forecasting is an important challenge in autonomous driving. Prarthana B. et al. proposed a solution to this challenge without requiring additional labeled data by using pseudo-labels generated from the data itself to reduce the need in heavily labeled datasets and improve forecasting accuracy with no manual annotations [27]. Motion forecasting is predicting the trajectories of traffic objects like vehicles, pedestrians based on their past trajectories and environment. The authors proposed using self-

supervised tasks that improved map and object-level information to increase performance and accuracy of motion forecasting models without the need to add additional annotations. The key principle resides around SSL – adding annotations (pseudo-labels) generated by initial data itself to help the model learn in more generalized way. The model was trained to learn local map structure information by encouraging it to recover masked lane features. Next self-supervised task was to predict the shortest path distance from lane nodes to intersection to help the model maintain global topology information. One additional task was to make model classify the performing maneuver of an object based on its trajectory – whether it is acceleration, turning, deceleration, etc. the last task was to make model classify whether each objects maneuver was successful or not – did the object successfully reach its aimed point and the end of its trajectory. All tasks were integrated into a standard motion forecasting model which consisted of agent encoder, map encoder, interaction model, trajectory decoder. The SSL tasks were used as auxiliary losses to help the model build more generalized understanding of the surrounding terrain. The dataset used was the Argoverse dataset. The results showed that SSL-Lanes outperforms state-of-the-art methods in terms of minADE (minimum Average Displacement Error), minFDE (minimum Final Displacement Error), and miss rate (MR). Specifically, the Success/Failure classification task achieves the best performance, reducing minFDE by 9.8% and miss rate by 22.4% compared to the baseline. The authors also demonstrate that SSL-Lanes achieves a better trade-off between accuracy, efficiency, and model complexity, making it suitable for real-time autonomous driving applications [27].

Scene semantics and structure guided clustering pre-training aims to address such challenges of autonomous driving as imbalanced data and complex scene geometry. The following problem was researched by Maciej K. et al. by using SSL techniques [28]. The authors aimed to improve representation learning for autonomous driving tasks by solving imbalanced object classes distribution, diverse object sizes, and complex scene geometry. The datasets contained small objects like bicycles and motorcycles which can be seen on figure 5.

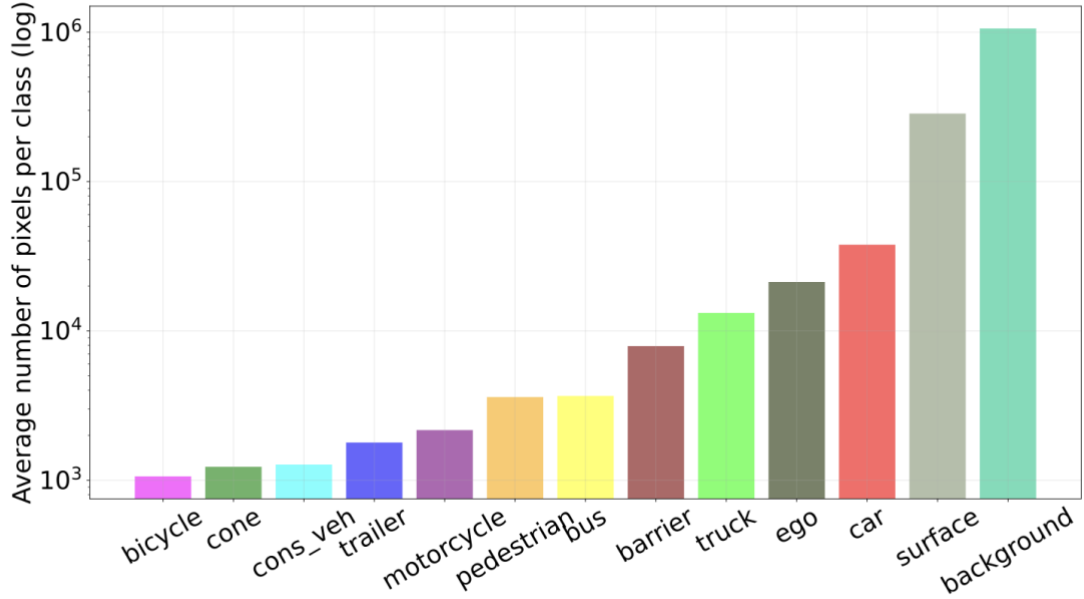


Figure 5. Pixel distribution per object [28]

Semantic distribution consistent clustering was used to encourage model to better understand representations of rare classes by using a von Mises-Fisher normalized formulation which handled long-tailed distributions and allowed flexible cluster to spread. Object diversity consistent spatial clustering handled imbalanced object sizes by relaxing uniformity assumption in spatial clustering, enabling the identification of both large background areas and small objects. Depth-guided spatial clustering was used to incorporate depth information from LiDAR to regularize learning based on the 3D structure of the scene, improving the separation of objects at different depths. The research proved that the suggested approach significantly improves performance on downstream tasks such as 2D semantic segmentation and 3D object detection on datasets like nuScenes, nulmages, and Cityscapes [28]. It also demonstrates strong domain generalization capabilities, outperforming baseline methods. The method demonstrated strength in segmenting rare and small objects.

The challenge of reconstructing dynamic street scenes without costly annotations was addressed by Nan H. et al. with the use of SSL methods to generalize across different driving scenarios and improve accuracy of simulations used in training and validating autonomous driving models [29]. The introduced approach S³ Gaussian which relies on decomposing static and dynamic elements from 4D space consistently without requiring any additional annotations and labels. The suggested method extended 3D Gaussian Splatting to 4D space by incorporating a spatial-temporal field network to model dynamic scenes. Each scene is represented using 3D Gaussians, which are enhanced with a spatial-temporal field to capture dynamic deformations. To implement the following, NN was constructed of a multi-resolution hex-plane structure encoder and multi-head Gaussian decoder. The hex-plane encoder decomposed the

4D grid into multi-resolution feature planes, separating static and dynamic elements. The Gaussian decoder predicted deformations and color changes for the 3D Gaussians based on the encoded features. During training LiDAR point cloud was used to ensure better geometric structure; and a combination of losses such as RGB, depth, semantic feature, and total-variational loss. The research demonstrated that the suggested approach outperforms existing methods in both static and dynamic scene reconstruction, particularly in handling dynamic objects and distant scenes with both consistency across multiple camera views and rendering quality [29].

A new approach to enhance LiDAR models' performance with language models was introduced by Guibiao L. et al. with the use of vision and language foundation models to enhance 3D scene understanding in autonomous driving [30]. The proposed method addressed the challenges of sparse LiDAR point clouds and noisy annotations by focusing on region-level contrastive learning, combining image, text, and LiDAR data to improve 3D representation learning. The approach in the research used was based on generating fine-grained, region-specific text descriptions using the BLIP-2 model and then creating region-level text embeddings which helped to enrich available semantic information for contrastive learning. The two main components were used: semantic-filtered region learning and region semantic assignment. The first part filtered out negative samples in contrastive loss by using semantic distance between regions, making sure that the regions with same semantics are close to each other. The second part assigned semantic labels to regions based on the most frequent category in each region, which helped to improve accuracy and reduce noise, since LiDAR data is noisy. The final step of the method was region-level contrastive learning which was based on grouping pixels and points into regions using masks generated by Segment Anything Model. This approach captured more contextual information than the usual approaches. The results of the research showed that the suggested method significantly outperformed existing self-supervised methods on autonomous driving datasets like nuScenes and SemanticKITTI. The method achieved state-of-the-art performance in 3D semantic segmentation, particularly in scenarios with limited annotated data. Also, it improves the mean Intersection-over-Union (mIoU) by 6.84% over the second-best method in few-shot fine-tuning tasks on nuScenes [30].

One more research on challenge of the need of large-scale datasets in autonomous driving with high-level labeled and annotated objects for training was conducted by Jakob M. et al. to address this [31]. The paper proposed a self-supervised method for generating training data for drivable area segmentation in autonomous driving with the approach of using deterministic stereo-based ground plane detection algorithm to automatically label large datasets, eliminating the need for manual annotation. This method is usually used for training deep NN for semantic segmentation. The process of working with data is depicted on figure 6.

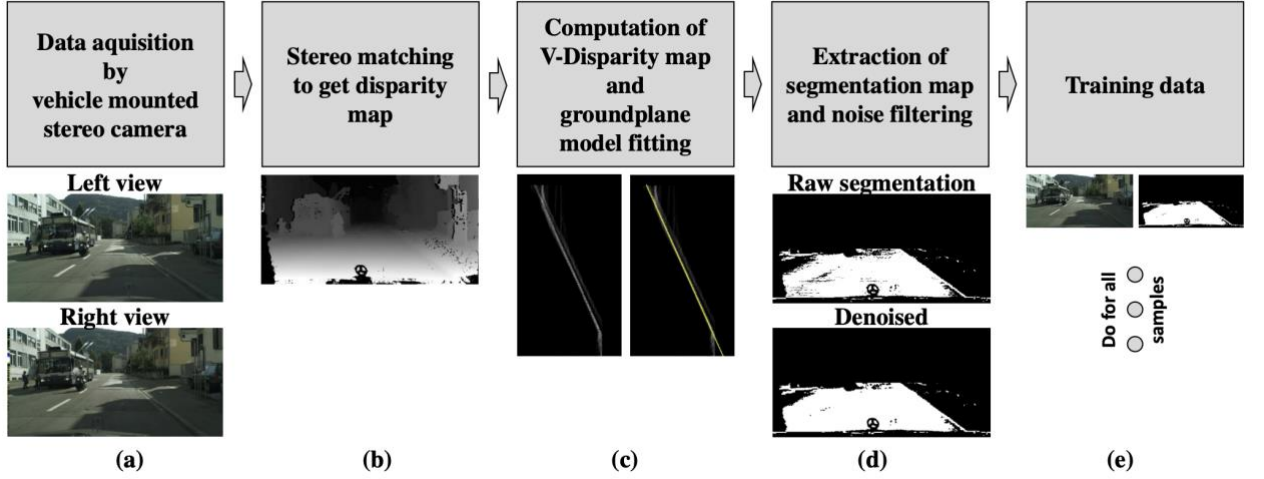


Figure 6. Model data gathering and processing pipeline [31]

The key principles that were used are: training data generator, fully convolutional network, self-supervised learning algorithm [31]. Training data generator used stereo vision and the v-disparity map technique to automatically label the drivable area in images and the v-disparity map was used to detect the ground plane by fitting a linear model to the disparity values, which are then used to segment the drivable area. The process includes post-processing steps like morphological filtering to reduce noise. For semantic segmentation U-Net and SegNet were used as main models with Adam and stochastic gradient descent being used optimizers respectively. The models in the research did not need any manually annotated data and used only labeled objects from training data generator. The method was evaluated on the Cityscapes and KITTI datasets. The results demonstrated that the deep NNs trained on the automatically labeled data significantly outperform the training data generator in terms of recall, precision, accuracy, and Intersection-over-Union (IoU). For example, on the Cityscapes dataset, U-Net achieved an IoU of 80.01%, compared to 66.46% for the training data generator. On the KITTI dataset, U-Net achieved an IoU of 72.70%, compared to 51.45% for the training data generator. The method also demonstrated robustness to challenging scenarios, such as the presence of shadows and unusual road textures.

Research using SSL techniques with contrastive triplet loss use and unsupervised feature extraction to avoid human bias in feature selection was conducted by Jinxin Z. et al. proposing a comprehensive framework for clustering large-scale autonomous driving scenario data using self-supervised feature extraction. The goal of the paper was to improve the completeness and fidelity of simulation tests for autonomous driving validation by grouping similar driving scenarios without relying on handcrafted features, which can introduce human bias [32]. The authors introduced a self-supervised deep learning approach to extract spatial and temporal features from driving data, which includes both vehicle trajectories and map information. Uniform data representation was used to render raw data into

a four-layer image using the Frenet coordinate system, ensuring uniformity across different data sequences. Spatial data compression was applied by a CNN based Variational Autoencoder to compress the image representations into compact frame feature vectors. Then a contrastive triplet loss was applied over it to make sure that temporally close frames have similar feature representations. Temporal data compression was implemented by a RNN based autoencoder to further compress the sequence of frame features into a single sequence feature vector; and the model was trained with both reconstruction and prediction tasks to confirm the extracted features are expressive. The K-means algorithm was applied to the sequence feature vectors to group similar driving scenarios. The proposed method achieved a True Positive Rate (TPR) of 99.88% and a False Positive Rate (FPR) of 0.10% on a private dataset, significantly outperforming the benchmark method (TPR: 77.27%, FPR: 3.0%). On the public nuScenes dataset, the method achieved a TPR of 98.70% and an FPR of 2.25%, demonstrating its scalability and effectiveness [32].

The concept of self-supervised navigation for autonomous driving uses sensor data to automatically label and annotate objects without human interference or supervision was researched by Gregory K. et al. where they developed and introduced Berkeley Autonomous Driving Ground Robot (BADGR) which was a full scales end-to-end self-navigating autonomous system for self-driving robots [33]. Unlike traditional geometric-based navigation systems, BADGR learns physical navigational affordances like traversal route in tall grass, preference for smooth terrain etc. from raw visual perception and autonomously collected data without human supervision. The system used retrospective self-supervision to label data based on past experiences, which allowed it to predict future events such as collisions, terrain properties and plan actions accordingly. To train the model LiDAR and sensor data was used, which allowed to use real world environments and label events based on gathered data. The reference images of training data and labeling can be seen on figure 7.



Figure 7. Training reference images for BADGR [33]

The model took RGB images and a sequence of future actions as input and predicted future events like collisions, position, terrain properties [33]. The model is trained using a combination of reconstruction and prediction tasks. BADGR used the trained model to plan actions that maximize a user-defined reward function, which encoded tasks like reaching a goal while avoiding collisions and bumpy terrain. The system used a model predictive control (MPC) framework to iteratively plan and execute actions. The research demonstrated that BADGR could successfully navigate through off-road environments, avoiding collisions and traversing tall grass, which a LIDAR-based policy incorrectly labels as untraversable. BADGR reached the goal 1.2 times faster on average compared to the LIDAR policy. In urban environments, BADGR avoided collisions and preferred smooth concrete paths over bumpy grass, outperforming the LIDAR policy, which fails to distinguish between smooth and bumpy terrain. BADGR improved its performance as it gathered more data. In experiments, a model fine-tuned with additional data from a new environment outperformed the initial model, demonstrating the system’s ability to adapt to new conditions.

Self-supervised learning approaches dominate supervised learning approaches based on the conducted research in all fields and require much less human labeled and annotated data. Self-supervised learning approaches were effective in difficult downstream tasks such as image classification, object detection, image segmentation, and other tasks with little labeled input. SSL combined with other learning methods surpasses every other supervised learning method in all researched benchmarks as various pretext tasks generate different supervision signals, which can help the network learn more typical characteristics of observed data. This research paper explored recent advancements in self-supervised learning techniques in autonomous driving. The reviewed literature covered multiple aspects of SSL, including scene understanding, motion forecasting, navigation systems, and feature extraction for clustering autonomous driving scenarios. A significant part of the conducted research focused on self-supervised methods to enhance perception in autonomous driving. For example, the S3PT model introduced scene semantics and structure-guided clustering to improve self-supervised pre-training, particularly addressing challenges in class imbalance and diverse object sizes in driving scenes . Similarly, the VLM2Scene model leveraged vision-language foundation models to enhance LiDAR-based 3D scene understanding through image-text-LiDAR contrastive learning, demonstrating significant improvements over prior methods. In motion forecasting SSL-Lanes research paper was introduced as a novel approach integrating self-supervised learning into motion prediction. It demonstrated that SSL could outperform traditional supervised approaches in predicting future trajectories of vehicles and pedestrians, offering a more generalizable and robust forecasting framework. Meanwhile, research of scenario clustering highlighted the benefits of self-supervised feature extraction in grouping autonomous driving data, improving the efficiency, quality and resolution of simulation systems. Self-supervised learning was also applied to navigation systems, with studies such as BADGR

demonstrating how reinforcement learning-based approaches could enable robots to autonomously improve their navigation capabilities using off-policy data. BADGR showcased the ability to generalize across novel environments without explicit supervision. Finally, research on self-supervised distillation between image and LiDAR data revealed promising methodologies for transferring pre-trained image representations into 3D space. The SLidR method, for instance, utilized superpixels for feature pooling to enhance object detection and semantic segmentation in sparse LiDAR point clouds. The reviewed papers highlight the growing potential of self-supervised learning in autonomous driving. These advancements address critical limitations such as data annotation costs, generalization across environments, and the ability to learn from unstructured data.

Self-supervised learning has developed into a transformative approach in machine learning, particularly in fields like autonomous driving, computer vision, and natural language processing. By handling large amounts of unlabeled data, SSL reduces dependency on costly manual annotations while achieving state-of-the-art performance in tasks such as perception, scene understanding, motion forecasting, and navigation. The reviewed research demonstrates that SSL techniques, including contrastive learning, clustering-based pre-training, and cross-modal distillation, significantly enhance model robustness, generalization, and adaptability to real-world conditions. Future research should explore optimizing loss functions, integrating multiple self-supervised tasks, and enhancing SSL models’ scalability across diverse autonomous driving datasets.

1.2 Theoretical analysis of SSL

Self-supervised learning has developed as a promising and ambitious approach to address the fundamental bottlenecks in autonomous driving systems: the massive annotation requirements and data collection issues that slow down traditional supervised learning methods. However, despite its potential to operate with vast amounts of unlabeled driving data, SSL for autonomous driving faces numerous significant challenges and limitations that impact its real-world deployment and effectiveness.

1.2.1 Core challenges in self-supervised learning for autonomous driving

Autonomous driving systems rely on robust perception models to interpret complex environments in real time. These models traditionally depend on large scale manually annotated datasets for supervised training, which presents significant limitations. First, the annotation process is labor-intensive and costly, particularly for 3D LiDAR data, where each point cloud must be manually labeled. Second, supervised learning struggles with generalization, as models trained on limited datasets often fail in unseen or edge-case scenarios, such as adverse weather conditions or rare traffic events.

One of the most critical challenges in SSL for autonomous driving is addressing the “long tail” of unusual events that occur on roads [38]. While it is relatively straightforward to collect large amounts of unlabeled sensor data from vehicle fleets, the main bottleneck is in validating that the full coverage of rare events was captured. These edge cases, such as unusual pedestrian behaviors, unexpected

obstacles and complex weather conditions may represent only a small part of driving situations but have major safety implications.

Self-supervised learning works with unlabeled sensor data to automatically generate training signals. However, applying SSL to autonomous driving introduces unique challenges. One major issue is the inefficient feature learning from multi-modal data. While SSL excels in 2D image tasks, autonomous systems require the integration of additional inputs like LiDAR point clouds, RGB cameras, and radars. Current SSL methods often treat these inputs separately which leads to suboptimal feature alignment and reduced accuracy.

Another critical challenge is domain shift [39]. Autonomous vehicles operate in dynamic environments where lighting, weather, and road conditions vary continuously. Most SSL frameworks are trained on static datasets and lack mechanisms to adapt to real-world diversity, resulting in performance degradation when deployed in new settings. Additionally, real-time processing constraints strict efficiency requirements on SSL models, which must balance high accuracy with low computational latency to ensure safe vehicle operation on the go. SSL approaches in autonomous driving suffer significantly from dataset bias and distribution mismatch issues [40]. Training datasets often exhibit selection bias, where certain demographic groups, weather conditions, or geographical regions are underrepresented. For instance, autonomous driving datasets may lack precise representation of mobility-impaired people, certain racial groups, or specific weather conditions.

Finally, evaluation benchmarks for SSL in autonomous driving remain inconsistent. While supervised learning relies on standardized metrics like mAP and mIoU, SSL methods are often tested using simple artificial tasks or limited datasets, making fair comparisons difficult. A unified evaluation framework is needed to objectively measure progress in self-supervised perception for autonomous systems. Addressing these challenges is essential to unlock the full potential of SSL in real-world driving applications.

A fundamental challenge in SSL for autonomous driving lies in designing effective pretext tasks that align with downstream driving tasks [41]. Current SSL approaches often employ generic pretext tasks such as frame prediction, rotation correction, or contrastive learning, but ensuring that the learned features are relevant for critical driving tasks like obstacle avoidance remains challenging. The disconnect between pretext tasks and actual driving requirements can result in learned representations that perform well on SSL objectives but fail to capture safety-critical features necessary for autonomous driving. This misalignment becomes particularly problematic when dealing with dynamic environments where temporal consistency and motion understanding are crucial.

1.2.2 Current approaches in self-supervised learning for autonomous perception

Recent advancements in self-supervised learning for autonomous driving have focused on overcoming the limitations of supervised methods through innovative pretext tasks and representation

learning techniques. These approaches can be categorized into three buckets: contrastive learning, generative modeling, and predictive coding, each offering distinct advantages for perception tasks.

Contrastive learning methods, which are used in SimCLR and MoCo frameworks, have shown success in learning visual representations from unlabeled image data [34]. These approaches work by maximizing agreement between differently augmented views of the same image while minimizing similarity with other images in the dataset. In the context of autonomous driving, contrastive learning has been used to process multi-view camera inputs where different camera perspectives of the same scene serve as natural augmentations. However, these methods often struggle with 3D point cloud data, where traditional image-based augmentation strategies prove less effective due to the irregular structure of LiDAR measurements. Contrastive learning loss which is used in these models is shown in formula 1 [34].

$$L = -\log \frac{\exp((\text{sim}(q, k^+))/(\tau))}{\sum_{i=1}^N \exp((\text{sim}(q, k_i))/(\tau))} \quad (1)$$

where:

q = query embedding

k^+ = positive key embedding

k_i = negative samples

τ = temperature parameter

$\text{sim}(q, k)$ = cosine similarity

Generative approaches which are based on masked autoencoding have developed into a tool for learning from both visual and LiDAR data. These methods randomly mask portions of the input and train the model to reconstruct the missing information, forcing it to learn meaningful representations of the underlying scene structure. The success of models like Point-BERT for point clouds demonstrates the potential of this approach. However, current implementations often require significant computational resources and may learn redundant features that do not directly translate to improved downstream task performance. Example of triplet loss function which can be used in these models is shown in formula 2 [35].

$$L = \max(\|f(x^a) - f(x^p)\|^2 - \|f(x^a) - f(x^n)\|^2 + \alpha, 0) \quad (2)$$

where:

x^a = anchor sample

x^p = positive sample
 x^n = negative sample
 a = margin

Predictive coding methods represent a third approach where models are trained to predict future sensor inputs from current observations. This is particularly relevant for autonomous driving since it aligns with the temporal nature of vehicular movement perception. Techniques like contrastive predictive coding have been adapted to process sequential LiDAR scans and camera frames resulting in learning representations that capture both spatial and temporal dependencies. These methods typically require careful design of the prediction horizon and can be sensitive to noise in the sensor data.

Recent hybrid approaches attempt to combine the strengths of these paradigms. For instance, some frameworks use contrastive learning for camera data and generative modeling for LiDAR, with shared latent spaces to enable cross-modal representation learning. Other approaches use geometric consistency constraints between sensor modalities to improve robustness. Despite these innovations, significant challenges remain in achieving precise, scalable, and generalizable self-supervised perception systems for autonomous vehicles. Current methods often have performance gaps compared to supervised approaches, particularly in safety-critical scenarios requiring precise object detection and semantic understanding. The SSL continues to evolve with ongoing research addressing limitations in computational efficiency.

1.2.3 Limitations of existing self-supervised approaches for autonomous driving

While current self-supervised learning methods demonstrate promising results for autonomous driving applications, some fundamental limitations persist. These limitations exist across multiple dimensions of model performance and practical implementation.

A primary issue is in the data efficiency of existing approaches. Most self-supervised methods require substantially larger quantities of unlabeled data compared to their supervised counterparts to achieve comparable performance levels. This requirement comes from the high difficulty of learning meaningful representations without direct supervision signals and labels. In autonomous driving scenarios, where edge cases and rare events are critical for safety but naturally scarce in collected datasets, this data inefficiency becomes particularly problematic. The models often fail to capture tiny but crucial features necessary for reliable operation in complex urban environments [40]. A critical limitation of current SSL approaches is the lack of effective uncertainty quantification and interpretability. Autonomous driving systems must be able to assess their confidence levels and identify when they encounter scenarios outside their training distribution. Existing SSL methods often lack mechanisms for communicating uncertainty to downstream planning and control modules making it difficult to implement safety measures when model confidence is low [43]. This limitation becomes

particularly problematic when dealing with out-of-distribution scenarios that require conservative driving behavior.

The modality gap is another significant challenge. Current self-supervised techniques typically treat different sensor inputs like cameras, LiDAR, radar in isolation, developing separate representation learning pipelines for each data type. This fragmented approach fails to use the combined output of multi-sensor systems, resulting in suboptimal fusion of information at later processing stages. Moreover, the temporal dimension of autonomous driving data is frequently not used to the full scale, with many methods processing individual frames independently instead of learning full spatiotemporal representations across sequences.

Computational requirements are a hard barrier to deployment [44]. Many state-of-the-art self-supervised algorithms demand huge computing resources for both training and inference, making them unsuitable for real-time operation on vehicle-embedded systems. The memory footprint and processing latency of these models often exceed the requirements for autonomous driving applications where decisions must be made within milliseconds to ensure safe navigation. Autonomous driving systems operate under strict real-time constraints, requiring SSL models to process sensor data and make decisions within milliseconds. The computational overhead associated with complex SSL architectures can conflict with the latency requirements of safe autonomous driving. Balancing model complexity with computational efficiency presents ongoing challenges when deploying SSL models on resource-constrained hardware. The challenge is compounded by the need to process high-resolution, multi-modal sensor data in real-time.

Evaluation methodologies for self-supervised approaches remain inconsistent across the field [45]. While numerous benchmarks exist for supervised perception tasks, the lack of standardized metrics for validating self-supervised representations makes objective comparison between methods challenging. Many studies report performance on downstream tasks after fine-tuning, which conflates the quality of learned representations with the effectiveness of the fine-tuning procedure itself. This makes it difficult to isolate and evaluate the true contribution of the self-supervised component. The absence of standardized evaluation metrics and validation frameworks specifically designed for SSL in autonomous driving presents significant challenges since traditional computer vision metrics may not precisely capture safety-critical aspects of driving performance [46]. Developing comprehensive evaluation metrics that validate both SSL model performance and safety measures remains an ongoing challenge. The rarity of safety-critical events makes it difficult to validate model behavior across the full scale of possible driving scenarios. Additionally, SSL models must adapt to evolving road conditions, traffic patterns, and infrastructure changes without forgetting previously learned behaviors. This requires developing trustworthy approaches to learning that maintain performance across diverse and evolving driving contexts and metrics [47].

Generalization capabilities represent one of the most critical limitations [48]. Models trained using self-supervision often have poor transfer learning performance across different geographic regions, weather conditions, or operational domains. The learned representations may capture dataset-specific artifacts and not fundamental features of driving scenes, leading to significant performance degradation when deployed in environments that differ from the training data distribution. This limitation is particularly concerning for autonomous driving systems which must operate reliably across diverse and unpredictable real-world conditions. The challenge of achieving robust cross-domain generalization while maintaining safety standards requires SSL approaches that can adapt to new environments without compromising learned safety behaviors. SSL models trained in specific geographical regions or driving contexts often fail to generalize to new domains with different road infrastructure, traffic patterns, or cultural driving behaviors. This limitation restricts the deployment of SSL-based autonomous driving systems.

The interpretability and safety validation of self-supervised systems present additional challenges. Unlike supervised approaches where the learning objective is explicitly defined, the representations learned through self-supervision can be difficult to analyze and verify. This uncertainty raises concerns about the ability to validate these systems for safety-critical applications where understanding model behavior and failure modes is essential. The black-box nature of many self-supervised learning algorithms complicates efforts to validate their decisions and ensure robust performance in all possible scenarios.

1.2.4 Developing solutions and future directions in self-supervised autonomous driving

Recent advances in self-supervised learning are beginning to address the limitations mentioned previously, suggesting promising ways towards more robust and efficient autonomous perception systems. These developing solutions use modern architectural designs and training algorithms specifically tailored to the unique requirements of autonomous driving applications.

A significant development involves the creation of unified frameworks for multi-modal representation learning [49]. Modern approaches now incorporate cross-modal consistency objectives that directly force alignment between different sensor modalities during the self-supervised pre-training phase. For instance, some methods employ contrastive losses between LiDAR point clouds and camera images, encouraging the model to learn shared representations that capture equivalent semantic information across modalities. This approach not only improves downstream task performance but also enhances the system’s stability to sensor failures or cope with conditions affecting a single modality.

Temporal modeling has emerged as another critical area of innovation. Advanced architectures now integrate recurrent neural networks or transformer-based sequence models to process temporal sequences of sensor data. These models learn to predict future frames or estimate ego-motion from historical observations, capturing the dynamic nature of driving environments more effectively than

single-frame approaches. Some implementations combine these temporal objectives with spatial reasoning tasks, creating comprehensive spatiotemporal representations that better reflect the continuous nature of real-world driving scenarios. For instance, masked autoencoder reconstruction loss is depicted in formula 3 [36] and temporal loss in formula 4 [37].

$$L = \frac{1}{|M|} \sum_{i \in M} \left\| x_i - \text{Dec}(\text{Enc}(x_{\text{masked}}))_i \right\|^2 \quad (3)$$

where:

M = masked positions

x_{masked} = input with masked regions

$\text{Dec}(\cdot), \text{Enc}(\cdot)$ = decoder/encoder functions

$$L = \frac{1}{T-1} \sum_{t=1}^{T-1} \|f_t(x_t) - f_{t+1}(x_{t+1})\|^2 \quad (4)$$

where:

T = sequence length

x_t = input at time step t

f_t = feature extractor at time t

SSL for autonomous driving must capture complex temporal relationships and dynamic scene evolution [42]. Traditional SSL approaches often focus on static image analysis, but driving scenarios require understanding motion patterns, predicting future states, and maintaining object identity across time. Achieving temporal consistency in SSL models while handling weather, lighting changes, and dynamic object interactions presents significant computational challenges. The models must learn to distinguish between meaningful temporal patterns and false correlations that may not generalize to new scenarios.

Growing interest is happening in self-supervised 3D perception, particularly for LiDAR data processing. Modern point cloud representation learning methods now incorporate geometric consistency constraints and hierarchical feature extraction to better capture the 3D structure of environments. Techniques such as point cloud completion and part-aware segmentation have shown promise in learning meaningful representations from unlabeled LiDAR data. These approaches often outperform their 2D counterparts in tasks requiring precise spatial understanding, such as obstacle detection and path planning.

Efficiency improvements represent another important direction of progress [49]. Recent work has developed specialized network architectures and training procedures that maintain representation quality while significantly reducing computational requirements. Knowledge distillation techniques where smaller student models learn from larger teacher models, have proven particularly effective in creating compact yet powerful representations suitable for real-time operation. Additionally, some approaches now incorporate adaptive sampling strategies that focus computational resources on the most informative regions of the input data, further optimizing processing efficiency.

Future research directions are likely to focus on several key areas. First, the development of more precise evaluation protocols will be crucial for properly validating and comparing different self-supervised approaches. Second, techniques for continuous learning and domain adaptation will become increasingly important as systems need to operate across diverse environments. Finally, the integration of self-supervised learning with other algorithms such as reinforcement learning and world models may open new possibilities for end-to-end autonomous driving systems that can learn from raw sensor data with minimal human supervision. These developments point towards a future where self-supervised learning forms the foundation for scalable, adaptable, and efficient autonomous driving systems.

1.2.5 Concluded analysis of self-supervised learning in autonomous driving

The throughout examination of self-supervised learning approaches for autonomous driving presented in the paper displays both the remarkable potential and significant challenges of this rapidly evolving field. With the systematic analysis of current methodologies critical components of SSL were identified that shape the modern approaches in autonomous vehicle perception systems.

The review first established the fundamental limitations of supervised learning approaches, particularly their dependence on costly manual annotation and their inability to generalize beyond labeled scenarios. This analysis demonstrated why SSL has emerged as a promising alternative capable of leveraging vast amounts of unlabeled sensor data to learn robust representations. Exploration of contrastive learning, generative modeling, and predictive coding approaches revealed how each algorithm offers distinct advantages for different aspects of autonomous perception – from visual feature extraction to spatiotemporal understanding.

A key finding from the analysis is the existing gap between SSL performance and supervised approaches in safety-critical tasks. While current methods show promising results on benchmark datasets, they often struggle with real-world complexities such as adverse weather conditions, sensor noise, and rare edge cases. This performance discrepancy highlights the need for more sophisticated pretext tasks that better capture the nuances of driving scenarios, as well as improved evaluation protocols that more accurately reflect real-world operational conditions.

The review also uncovered important architectural considerations for SSL systems in autonomous driving. The most successful approaches combine several key characteristics: multi-modal

processing capabilities, efficient temporal modeling, and compact representation learning. These features address the fundamental requirements of autonomous systems for self-driving vehicles while remaining computationally maintainable for real-time operation. The exploration of existing developing hybrid architectures suggests that future progress will likely come from systems that can effectively integrate these various components into solid frameworks.

The analysis identified several critical research directions that are considered for further investigation. These include the development of more robust evaluation metrics, improved techniques for continual learning and domain adaptation, and better methods for ensuring the safety and interpretability of self-supervised systems. The integration of SSL with other learning paradigms like reinforcement learning and world models, presents particularly promising opportunities for creating more capable and generalizable autonomous driving systems.

The limitations section of the review served to handle expectations while providing clear guidance for future work. While SSL has demonstrated impressive capabilities in controlled settings, substantial challenges remain in achieving the level of reliability required for real-world deployment. Issues like computational efficiency, generalization across environments, and safety verification must be addressed before these methods can fully replace supervised approaches in production systems.

The foundational work reviewed in the paper has established the reasoning of the approach and stated that significant innovation will be required to overcome the remaining technical hurdles. The most promising path forward appears to lie in the development of more learning algorithms that can combine the strengths of self-supervision with targeted human guidance, creating systems that learn efficiently from experience while respecting crucial safety constraints. The insights gathered suggest that while SSL may not yet be ready to fully replace supervised learning in autonomous driving applications, it is rapidly approaching the point where it can significantly reduce dependence on manual annotation while improving system robustness and scalability. As the field continues to develop, self-supervised approaches will play an increasingly central role in the development of next-generation autonomous vehicles, potentially unlocking new capabilities and enabling more widespread deployment of this transformative technology.

1.3 Problem statement

1.3.1 Problem definition in autonomous driving vehicles field

The development of robust autonomous vehicle perception systems faces significant challenges based on the limitations of traditional supervised learning approaches [50]. Contemporary autonomous driving systems require huge, labeled datasets [51] to achieve acceptable performance in object detection, semantic segmentation, and motion forecasting tasks. However, the manual annotation of driving data presents significant costs and scalability constraints that significantly slow the advancement of technology [52].

The labeling bottleneck occurs in multiple dimensions: temporal annotation costs can exceed thousands of dollars per hour of driving footage, particularly for dense 3D annotations required for LiDAR and multi-camera systems [52]. The annotation process for autonomous driving datasets is exceptionally labor-intensive and costly. For instance, datasets like nuScenes and KITTI require extensive human effort to annotate objects, road boundaries, and other critical elements, making the process unsustainable for scaling to real-world applications. Furthermore, supervised learning models trained on completed labeled datasets struggle with generalization to novel driving scenarios, weather conditions, and geographical regions not represented in the training data. This limitation becomes particularly acute in safety-critical applications, where encountering rare but dangerous scenarios like construction zones, emergency vehicles, or unusual pedestrian behaviors can lead to catastrophic failures [53].

The scalability challenge is driven by the exponential growth in sensor data volume, with modern autonomous vehicles generating approximately 1 GB of data per second from their sensor suites [54]. The scalability of supervised learning is further constrained by the dynamic nature of autonomous driving environments. As vehicles encounter new geographic regions, traffic patterns, or infrastructure designs, the models must adapt, necessitating additional labeled data for retraining. This creates a bottleneck in the development cycle, delaying the deployment of autonomous systems and increasing costs. Moreover, the quality of annotations can vary, introducing biases or inaccuracies that may compromise the safety and reliability of the autonomous perception system. For example, studies have shown that dataset biases, such as underrepresentation of certain demographic groups or rare driving scenarios, can lead to performance degradation in real-world deployments, posing significant safety risks. Traditional supervised approaches cannot efficiently leverage this vast amount of unlabeled data, creating a significant opportunity cost in terms of potential model improvements.

1.3.2 The importance of a reliable SSL solution for autonomous driving vehicles

Self-supervised learning has developed as a transformative algorithmic approach that addresses the fundamental limitations by learning meaningful representations from unlabeled data through carefully designed pretext tasks [55]. Recent advances in SSL for autonomous driving demonstrate remarkable potential across multiple perception tasks, including monocular depth estimation, 3D occupancy prediction, and motion forecasting.

The importance of addressing the limitations of supervised learning through self-supervised methods in autonomous driving cannot be omitted since it directly impacts the safety, scalability, and adaptability of these systems [55]. Autonomous vehicles operate in highly dynamic and unpredictable environments where perception errors can have life-threatening consequences. Current supervised learning approaches, while effective in controlled settings, struggle to generalize to the vast array of real-world scenarios, particularly those involving rare or unforeseen events such as sudden pedestrian

crossings, adverse weather conditions, or complex urban interactions. These edge cases are critical for ensuring the reliability of autonomous systems, but they are often underrepresented in manually labeled datasets due to the high costs and labor involved in their annotation. SSL offers a viable solution by enabling models to learn from unlabeled data, thereby expanding the diversity and coverage of training scenarios without the need for expensive human labelling [56].

The need for labeled datasets also introduces biases that can compromise the fairness and robustness of perception systems. Studies have demonstrated that datasets like Cityscapes and BDD100K include geographic and demographic biases, leading to performance reduction across different regions or population groups. For instance, models trained mostly on data from urban environments in developed countries may underperform in rural or low-resource settings, raising ethical and safety concerns. SSL mitigates these issues by leveraging raw sensor data collected from diverse environments, allowing models to learn more representative features without being constrained by the limitations of human annotation. This is particularly important for global deployment, where autonomous vehicles must adapt to varying traffic rules, road infrastructures, and cultural driving behaviors. Moreover, the scalability of autonomous driving technology demonstrates the ability to process and learn from the enormous volumes of data generated by vehicles. Supervised learning with its dependency on labeled examples can't keep pace with the exponential growth of sensor data, creating a bottleneck in model improvement and deployment. SSL can continuously adjust its representations using real-world driving data, enabling incremental learning and adaptation without the need for constant human oversight. This capability is essential for long-term autonomy, as it allows vehicles to improve their perception systems over time, learning from new experiences and evolving road conditions. SSL techniques such as contrastive learning, masked autoencoding, and temporal consistency modeling can use unlabeled driving data to learn robust feature representations without human annotation. For instance, recent work has shown that SSL pretraining can improve object detection performance by up to 5.8% mAP compared to supervised baselines when fine-tuned on downstream tasks. Similarly, SSL approaches for motion forecasting have demonstrated superior generalization capabilities while requiring only a fraction of the labeled data compared to traditional methods [57, 58, 59].

Despite these advantages, current SSL methods are not ready enough to fully replace supervised approaches in safety-critical applications. Challenges such as weak multi-modal fusion, inadequate temporal modeling, and computational inefficiencies delay their real-world applicability. Addressing these limitations is not an academic exercise but a practical task for the advancement of autonomous driving. The stakes are high – stakeholders ranging from automotive manufacturers like Tesla and Waymo to regulatory bodies and insurance providers all depend on the reliability of perception systems to ensure passenger safety, legal compliance, and public trust. Furthermore, the broader adoption of autonomous vehicles promises benefits such as reduced traffic accidents, improved mobility for

underserved people and optimized transportation logistics. All of that rely on robust, scalable, and adaptable perception technologies. Advancing SSL for autonomous driving is not just about improving algorithmic performance but about enabling a transformative technology that can operate safely and stable across diverse real-world conditions. The development of more generalized, efficient, and reliable SSL frameworks will be a solution to eliminating the gap between laboratory research and widespread deployment determining the possibility of autonomous vehicles as a mainstream mode of transportation.

1.3.3 Evaluation metrics for SSL performance under real-time constraints in autonomous driving

The evaluation of self-supervised learning methods for autonomous driving requires complex metrics to validate their performance, robustness, and real-world applicability. Unlike traditional supervised approaches where evaluation primarily focuses on accuracy metrics derived from labeled test sets, SSL models must be validated across several dimensions to ensure they meet the strict demands of autonomous vehicle perception. A comprehensive assessment framework must consider not only standard computer vision benchmarks but also domain-specific criteria that reflect the dynamic and safety-critical nature of driving environments.

At the core of performance evaluation are standard computer vision metrics such as mean Average Precision (mAP) for object detection and mean Intersection over Union (mIoU) for semantic segmentation [61]. mAP is a standard metric for object detection that measures the precision across different IoU (Intersection over Union) thresholds. For autonomous driving, mAP at IoU thresholds of 0.5 and 0.7 are commonly used to evaluate 3D object detection performance. mIoU is a key metric for semantic segmentation tasks that measures the overlap between predicted and ground truth segmentation masks. Higher mIoU values indicate better segmentation quality, which is crucial for understanding the driving environment. These metrics quantify the model’s ability to correctly identify and localize objects, which is fundamental for tasks like pedestrian detection, vehicle tracking, and drivable area segmentation.

However, in the context of SSL, these metrics must be interpreted with additional care since as the quality of pseudo-labels generated during pre-training can introduce noise that affects downstream task performance. For instance, models pre-trained with contrastive learning may achieve high mAP on common objects but struggle with rare classes that were underrepresented in the pretext task. Similarly, temporal tasks such as motion forecasting are evaluated using metrics like minimum Average Displacement Error (minADE) and minimum Final Displacement Error (minFDE) [27]. minADE measures the average L2 distance between the predicted trajectory and the closest point on the ground truth trajectory. Lower minADE values indicate better motion prediction accuracy. minFDE evaluates the L2 distance between the final predicted position and the ground truth final position. This metric is particularly important for long-term trajectory planning. Miss Rate (MR) quantifies the percentage of

predictions that fail to come within a certain threshold of the ground truth trajectory. Lower miss rates indicate more reliable motion forecasting. These metrics are particularly important for assessing how well SSL models capture the dynamics of moving objects, a critical capability for collision avoidance and path planning.

Beyond standard benchmarks, scenario-based robustness testing is needed to evaluate how SSL models generalize across diverse and challenging driving conditions. Autonomous vehicles must operate reliably in varying lighting (day/night), weather (rain, snow, fog), and terrain (urban, highway, off-road), but many SSL methods are validated only on clear-weather datasets, masking potential vulnerabilities. For example, a model trained with rotation prediction as a pretext task may perform well on sunny urban scenes but fail in foggy conditions where visual features are obscured. Similarly, the ability to handle rare but critical edge cases such as sudden pedestrians, construction zones, or erratic drivers must be systematically tested, as these scenarios often determine real-world safety. Datasets like nuScenes-LIDARSeg and Argoverse 2 include challenging conditions specifically designed for robustness evaluation, but SSL methods must be further tested in simulation environments like CARLA or NVIDIA DRIVE Sim to expose weaknesses before real-world deployment.

Some other relevant metrics work and evaluate models' performance in real-time. Inference time measured in milliseconds quantifies the time required for a model to process input data and generate predictions. For autonomous driving, inference times below 100ms for perception tasks and 50ms for motion forecasting are typically required to meet real-time constraints [62]. Frames per second metric indicates how many frames a model can process per second, directly reflecting real-time capability. Higher FPS values suggest better real-time performance, with autonomous driving systems typically requiring at least 10-30 FPS depending on the specific task. End-to-end latency measures the total time from sensor input to actionable output, including all processing steps [63]. This comprehensive metric better reflects real-world performance than isolated component measurements. Computational efficiency metrics like inference latency, memory footprint, and training scalability are vital for determining whether SSL models can meet the real-time processing requirements of autonomous systems. Many state-of-the-art SSL methods relying on large transformer architectures or complex contrastive objectives, demand substantial computational resources, making them impractical for embedded vehicle hardware. For instance, methods like masked autoencoders achieve impressive accuracy but may have significant latency for real-time LiDAR processing. Evaluating metrics like frames per second on target hardware like NVIDIA Drive Xavier ensures that SSL advancements translate to deployable solutions without compromising safety-critical timing constraints [64].

Another critical dimension is domain transfer and few-shot adaptability, which measure how well SSL models generalize to unseen environments with minimal additional labeled data. Autonomous driving systems are frequently deployed in new geographic regions with distinct road layouts, traffic

signs, and driving behaviors, making adaptability a key requirement. SSL’s promise is in its ability to learn generalizable representations from unlabeled data, but current methods often exhibit significant performance drops when tested on out-of-distribution datasets. For example, a model pre-trained on European road scenes may struggle when applied to Asian urban environments due to differences in infrastructure and traffic patterns. Few-shot evaluation, where models are fine-tuned with very limited labeled data from the target domain, is thus a crucial test of SSL’s practicality. Metrics like few-shot mIoU or relative improvement over supervised baselines quantify this capability, with higher values indicating better transferability.

In summary, evaluating SSL for autonomous driving requires a solid approach that combines standard accuracy metrics, robustness testing across diverse scenarios, domain adaptation benchmarks, and computational efficiency analyses. This complex evaluation framework not only highlights the strengths and weaknesses of current SSL methods but also guides future research toward closing the gap between theoretical performance and real-world usability.

1.3.4 Key stakeholders impacted by developing SSL techniques for autonomous driving vehicles

The advancement and deployment of self-supervised learning techniques for autonomous driving perception systems involve a complex ecosystem of stakeholders, each with distinct operating contexts and requirements that shape the development and implementation of these technologies. The autonomous vehicle deployer category splits into three operationally distinct groups since robotaxi operators, automotive OEMs and delivery-robot startups face different cost structures, different fleet scales and different regulatory exposures.

Robotaxi operators such as Waymo and Cruise are companies that run commercial autonomous fleets in a defined set of geographic markets, and their perception stack is fine-tuned on a labelled set that is collected from the same operating cities to which the fleet is deployed. Their primary concern lies in balancing the labelled data budget that is needed when the fleet expands to a new city against the SSL pre-training compute budget that allows the team to use the unlabeled fleet collection from the same city as the source of pre-training signal. The potential of SSL to reduce the requirement for expensive labelled datasets while maintaining perception performance aligns directly with their need to accelerate per market launch cycles and reduce the marginal cost of every new market entry. The unlabeled fleet collection that these operators are already storing for purposes such as scenario mining and edge-case analysis becomes an additional asset under an SSL strategy since the same collection can serve as the pre-training source at no incremental data-collection cost. These stakeholders also face strict regulatory pressures since they must demonstrate that SSL derived models meet or exceed the precision of traditional supervised approaches before deployment in production vehicles [65].

Automotive OEMs such as Tesla, Mercedes-Benz and BMW operate at much larger fleet scale than robotaxi operators since their perception stacks are deployed across multiple vehicle programs and across multiple model years, and any in house pre-training capability that the OEM develops is amortized across the entire vehicle portfolio of the company. The trade-off they face is the same as the one of robotaxi operators on the labelling versus pre-training axis, though the larger and longer-lived fleet means that the marginal payoff of an in-house SSL pre-training capability is small at any given year and meaningful when summed across the full vehicle program lifetime. The unlabeled fleet collection of an OEM also typically covers driving conditions that public pre-training datasets like ImageNet do not capture, including night driving, snow, rain and regional traffic infrastructure, which is part of the value proposition of the in-house SSL approach. These stakeholders are subject to the same regulatory acceptance bar as robotaxi operators when it comes to deploying SSL-derived perception models, and they have an additional requirement of stack consistency across vehicle programs which makes any change to the pre-training source a multi-year decision.

Delivery-robot and last mile autonomy startups such as Nuro and Starship operate at a much smaller fleet scale than the two groups above, and the labelled data budget that these companies typically work with is in the same range as a research scale dataset of approximately 2,975 to 5,000 fine-annotated images, which is the same scale as the Cityscapes training split used in this thesis. The relevance of the experimental setup of this thesis to these stakeholders is therefore direct since the labelled data budget that is studied here matches the labelled data budget that they realistically have available, and the SSL versus supervised comparison answers their backbone selection question without further extrapolation. The same regulatory pressures apply though at a less strict threshold since the operating speed and the deployment domain of delivery robots is more constrained than a passenger autonomous vehicle, which makes the safety certification bar more attainable in practice.

Simulation vendors create another crucial group of stakeholders, as they provide the virtual environments and synthetic data generation tools that are increasingly important for SSL research and validation. Companies like NVIDIA with DRIVE Sim and Cognata offer platforms that enable large-scale generation of diverse driving scenarios, including rare edge cases that are difficult to capture in real-world data collection. These stakeholders have interest in advancing SSL techniques since the ability to learn from synthetic or unlabeled data enhances the value proposition of their simulation products. The integration of SSL methods with simulation platforms creates a synchronized mechanism where simulations provide large quantities of diverse data needed for SSL pre-training, while SSL enables more efficient use of simulated data by reducing the need for manual annotation. This dynamic is particularly important for addressing the long-tail problem in autonomous driving, where SSL models trained on carefully labeled artificial edge cases may demonstrate improved performance in real world operations [65].

Academic groups, industrial research labs and the open-source community play an important role as knowledge creators and validators in the SSL for autonomous driving ecosystem. University groups together with corporate research labs contribute important algorithmic advances and provide independent evaluation of different approaches, while the open-source community releases reference implementations and benchmark code that allow other groups to reproduce and extend the published results. These stakeholders often focus on pushing the boundaries of what is theoretically possible with SSL, exploring modern pretext tasks, more efficient contrastive learning frameworks, and improved multimodal fusion techniques. Their work helps establish benchmarks and best practices that guide industry adoption, while also identifying limitations and potential failure modes that must be addressed before real world deployment. The research and open-source community also serves as an important bridge between foundational machine learning advances and their practical application to autonomous driving through collaborative projects with industry partners.

Government regulators and transportation safety agencies represent one of the most cautious and influential stakeholders in this domain. Organizations like the National Highway Traffic Safety Administration (NHTSA) in the United States, the European Union's European Road Safety Charter and other regional regulatory organizations are responsible for establishing the safety standards that SSL based perception systems must meet [65]. These stakeholders are primarily concerned with verification and validation methodologies - how to certify that models trained with SSL techniques are at least as safe as traditional approaches, given the black box nature of many self-supervised methods [66]. Their conservative position comes from the consequences of perception failures in autonomous vehicles, which could lead to loss of life and significant damage. As SSL matures regulators will need to develop new frameworks for evaluating these systems that go beyond traditional supervised learning benchmarks, potentially incorporating modern complex metrics for assessing representation quality, failure mode diversity, and out of distribution detection capabilities [67].

Insurance providers and risk assessors represent an emerging group of stakeholders who will play an increasingly important role as SSL-based autonomous systems approach deployment. These organizations are concerned with understanding and quantifying the risk profiles of SSL systems compared to traditional approaches. Their assessments will need to consider questions such as: do SSL models fail in systematically different ways than supervised models? How does their performance degrade in edge cases? Can their behavior be explained and certified for liability purposes? The answers to these questions will directly influence insurance tariffs and risk assessments for autonomous vehicles, creating financial ground for adopting SSL techniques [66].

The complex connection between these stakeholder groups creates both opportunities and challenges for the advancement of SSL in autonomous driving. While the technology promises significant efficiency gains, its path to wide, real-life adoption requires careful navigation of technical,

regulatory and commercial considerations. Successful integration will depend on continued collaboration across these stakeholders to align research directions with practical needs, develop appropriate validation frameworks, and establish trust in these advanced machine learning systems. The goal shared by all parties is safer and more efficient transportation systems, and this unifying objective is what helps balance the diverse priorities and perspectives within this ecosystem.

1.3.5 Proposed methodological framework for self-supervised learning in autonomous driving

The methodological framework proposed in this thesis is a controlled comparison of self-supervised pre-training strategies for the perception backbone of an autonomous driving model, evaluated on the urban semantic segmentation task. The framework is motivated by the labelling cost problem and the long-tail problem identified in the previous subsections of this chapter since both problems are tied to the choice of pre-training method when the labelled set is small relative to the unlabeled fleet collection that an autonomous driving operator can use. The framework consists of two parts: a controlled comparison that varies only the pre-training method while keeping every other component of the downstream segmentation pipeline constant, and a fine-tuning experiment that takes the best backbone of the controlled comparison and characterizes the practical mIoU that the same backbone can reach when the controlled limitation restrictions are lifted. Both parts together aim to provide an answer to the research question that motivates this thesis with the precision that a controlled comparison allows, and an upper bound on the practical mIoU that a deployment team can target with the winning backbone.

The matched downstream protocol is the central design choice of the controlled comparison since it isolates the pre-training method as the single variable under study. Every backbone is paired with the same lightweight three-convolution segmentation head that produces a per pixel class prediction at the half Cityscapes input resolution of 512 x 1024 pixels, and the head is trained end to end together with the backbone in the fine-tune mode and trained alone with a frozen backbone in the linear-probe mode. The optimizer is the same Adam configuration with the same default beta parameters, the same learning rate, the same cosine annealing schedule and the same number of training epochs across all experimental models, and the loss function is the standard cross-entropy with the standard ignore-index of 255 used to skip the void pixels of the Cityscapes annotation. The data split is the full Cityscapes fine-annotated set of 2,975 training images and 500 validation images at the resolution stated above, and no augmentations are applied to the input images during training so that the input distribution is identical across all backbones and the only varying factor is the pre-training method. This deliberate restriction of every component of the downstream pipeline to a fixed identical configuration is what allows the resulting mIoU values to be compared directly across the seven experimental models of the controlled experiment.

The four pre-training strategies that are compared in the controlled experiment are random initialization of the backbone weights, supervised pre-training on ImageNet labels, contrastive self-supervised pre-training using the MoCo v3 method, and self-distillation self-supervised pre-training using the DINO method. Random initialization serves as the no pre-training baseline that gives a reference point for the value of any pre-training, supervised ImageNet serves as the established label-based baseline that the SSL approaches must outperform to be worth the additional pre-training compute, and the two SSL methods cover the contrastive family represented by MoCo v3 and the self-distillation family represented by DINO which together cover most of the modern SSL design space described in section 1.1. Each backbone is evaluated under two protocols since the linear probe method freezes the backbone and trains only the segmentation head while the full fine-tune method trains the entire stack end to end, and the two methods together separate the quality of the pre-trained features themselves from the additional capacity that the fine-tune mode provides. Random initialization is run only in the fine-tune mode since a linear probe on random features is not informative, which gives a total of seven experimental models for the controlled comparison.

The fine-tuning experiment is the second part of the framework and is run only on the best backbone of the controlled comparison, with the goal of characterizing the practical mIoU that the same backbone can reach when the deliberate restrictions of the controlled experiment are lifted. The lifted restrictions include a heavier multi-block segmentation head, an auxiliary supervision branch on an intermediate transformer block, a compound loss that adds the Lovász-softmax surrogate of the IoU and an auxiliary cross-entropy term to the standard cross-entropy, an AdamW optimizer with layer-wise learning rate decay across the transformer blocks, a cosine schedule with a short warmup, the standard scale and crop plus flip plus color-jitter augmentation pipeline, mixed-precision training with an exponential moving average of the model weights, and two-resolution test time augmentation at the full Cityscapes resolution of 1024 x 2048 pixels. The output of this experiment is a single mIoU number that quantifies the head and method contribution on top of the same backbone of the controlled comparison, together with the per-class IoU breakdown across all 19 evaluation classes. The role of the experiment within the framework is to provide a practical upper bound for the same backbone that the deployment teams of section 1.3.4 can use as a planning reference instead of the controlled mIoU which is artificially conservative because of the restrictions of the matched protocol.

The evaluation metrics used by the framework are the standard mean Intersection over Union as the primary metric, the per-class IoU on the seven rare classes that occupy less than one percent of the Cityscapes pixel mass as the long-tail proxy, the difference between the fine-tune and the linear-probe mIoU per backbone as a derived metric that quantifies the gap between feature quality and end to end capacity, and the inference frames per second on the same hardware as the training as a real-time feasibility proxy. The mean IoU is the metric that the literature reviewed in section 1.1 uses to compare

SSL pre-training methods on Cityscapes and is the metric on which the controlled comparison is decided, while the per-class IoU on the rare classes is what surfaces the long-tail behavior that the literature reviewed in this chapter identifies as the limiting constraint on safety-critical deployment. The probe versus fine-tune delta separates the representation quality contribution from the fine-tune capacity contribution per backbone and explains why two backbones with similar fine-tune mIoU can have very different probe mIoU. The inference frames per second is reported with the explicit caveat that the absolute number is measured on a desktop-class data-center GPU and not on automotive embedded hardware, so the relative ordering between the backbones is the part of the result that is meant to carry to a deployment context, while the absolute numbers are not.

The framework deliberately scopes out several concerns that the broader autonomous driving deployment context discussed in the previous subsections does require but that this thesis does not claim to address. The framework does not test multi-modal sensor fusion since the input is a single RGB camera image and not a synchronized camera radar LiDAR sensor stream, and the framework does not test depth estimation, motion forecasting or any temporal task since the input is a single frame and not a sequence. The framework does not implement uncertainty estimation, out of distribution detection or fail-safe behavior and does not run an on-vehicle deployment loop with edge inference, fallback policies or over the air updates. These components are listed as future-work directions in section 3.6.2 since each of them is a research direction and a separate research project would be needed to address each of them since the controlled comparison of this thesis applies to the pre-training method choice.

The reproducibility commitments of the framework cover both the experimental setup and the implementation. The hyperparameter values for both the controlled comparison and the fine-tuning experiment are reported in full in chapter 2, the supervised ImageNet, MoCo v3 and DINO backbones are loaded from publicly available official checkpoints with no in-house pre-training step that would be difficult to reproduce, and the Cityscapes dataset is used at the standard 2,975-training and 500-validation split that is shared with the rest of the literature on this benchmark. Every experimental model is run with a single seed since the available compute budget did not allow multi-seed runs, and this restriction is acknowledged as a limitation in section 3.4.5 and listed as a future-work direction in section 3.6.2. The implementation code is released in the public repository [102], which contains the dataset loader, the backbone constructors, the segmentation head, the training and evaluation loops and the inference timing script, so that a follow-up study can directly extend the experimental models of this thesis with additional backbones or with the same backbones evaluated on a different segmentation dataset.

The chapter that follows describes the dataset, the methods and the experimental setup of the framework introduced in this section, including the Cityscapes dataset and the exploratory data analysis on it, the four pre-training methods and their official checkpoints, the segmentation head and the matched

training protocol, the evaluation metrics, the reproducibility commitments and the additional configuration of the fine-tuning ceiling experiment. The third chapter then reports the experimental results, discusses the implications for the research question and the long-tail problem, translates the technical numbers into a business-application analysis for the stakeholder categories of section 1.3.4, and lists the conclusions and the future-work directions of the thesis.

2 DATASET, METHODS AND EXPERIMENTAL SETUP

2.1 Chapter overview

The following chapter describes the data, the pre-trained models, the downstream segmentation architecture, the training method and the evaluation metrics that were used in this thesis to compare self-supervised pre-training strategies for semantic segmentation in driving scenes. The literature review and problem statement in chapter 1 demonstrated that SSL methods have potential to reduce the dependency on manually labeled data while still maintaining strong performance. The existing comparisons in the literature are spread across multiple sensors, downstream tasks and datasets, which makes it difficult to measure the effect of the pre-training method by itself. To make the comparison precise and reproducible the experiments in this thesis are restricted to a single, popular image-only segmentation benchmark and a small set of pre-training strategies that can be loaded from publicly available checkpoints, where everything except the backbone is fixed across the experimental models. The following sections cover the dataset and exploratory data analysis, the four pre-trained backbones being compared, the shared segmentation head used as the downstream model, the training protocol with its hyperparameters, the evaluation metrics and the reproducibility details needed to reproduce the results.

The research question of the thesis is the following: do self-supervised pre-training strategies produce feature representations that transfer more effectively to semantic segmentation in driving scenes than supervised baselines, and which SSL method performs best in this setting. To answer it, four backbones are evaluated: a ResNet-50 with random initialization which serves as a lower-bound reference, a ResNet-50 pre-trained with supervised classification on ImageNet which serves as the standard transfer baseline, a ResNet-50 pre-trained with MoCo v3 which represents a contrastive SSL approach, and a ViT-S/16 pre-trained with DINO which represents a non-contrastive self-distillation SSL approach. Each backbone is evaluated under two protocols, linear probe where the backbone is frozen and only the segmentation head is trained, and full fine-tune where the entire network is trained end-to-end. The end-to-end pipeline is displayed on figure 8.

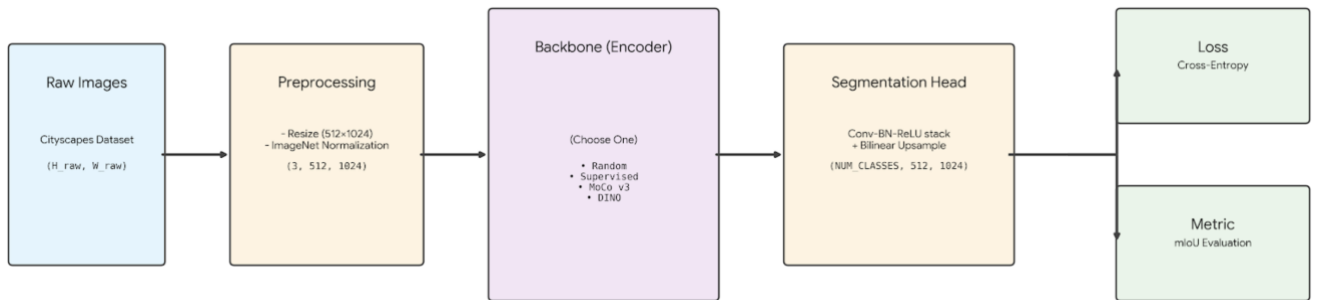


Figure 8. End-to-end experimental pipeline

2.2 Dataset: Cityscapes

The dataset used in this thesis is Cityscapes [68]. The dataset is a large-scale benchmark for semantic scene understanding designed by Cordts M. et al. and released in 2016. Cityscapes dataset was chosen for multiple reasons. First, it has developed into a standard evaluation benchmark for SSL approaches in autonomous driving and was used in multiple SSL works reviewed in chapter 1 such as SLiDR [26] and S3PT [28], which makes the results of this thesis directly comparable to existing literature. Second, the dataset provides dense pixel-level fine annotations across 19 evaluation classes, allowing direct measurement of mean Intersection over Union without any need for label conversion or proxy metrics. Third, it consists of real driving footage that was captured from a stereo camera mounted on the ego-vehicle, which directly satisfies the requirement to work with real captured data and not with simulated or synthetically generated scenes. Several draw backs were accepted with this choice: Cityscapes contains only camera data and does not include LiDAR point clouds, so 3D perception methods were out of scope; the data was collected in Germany and a few neighboring countries, so the geographic coverage is limited and the dataset includes only daylight and dry-weather conditions which means that adverse-weather and night-time scenarios are not represented [68]. These draw backs do not alter the research question of this thesis since the comparison between pre-training strategies is performed under exactly the same data, the same protocol and the same evaluation metric for every backbone, so any difference in mIoU between the experimental models can be attributed to the pre-training method itself and not to the data or to the downstream model. The absolute mIoU numbers reported in chapter 3 are specific to the daylight European urban setting captured by Cityscapes and would need to be re-validated on data collected in different geographies and weather conditions before being used for any deployment decision, and this point is returned to in the limitations section of chapter 3. The decision to keep the comparison single dataset is also a practical one given the compute budget available for this thesis, since each of the experiments described in section 2.1 must be retrained for every additional dataset, and a multi-dataset evaluation is left for future work.

The data collection process was described by the authors of the dataset in detail. Cityscapes dataset was recorded across 50 different cities, mostly in Germany with a smaller number of recordings in neighboring countries, captured during several months of spring, summer and fall in good to medium weather conditions [68]. The recordings were obtained from a stereo camera mounted behind the windshield of the vehicle, and the images were carefully selected from longer driving sequences to maximize the diversity of objects, scene layouts and backgrounds. The full dataset contains 5,000 images with high quality fine pixel-level annotations and additional 20,000 images with coarse polygon-level annotations which are intended for weak-supervision experiments. In this thesis only the fine-annotated set is used since the comparison protocol relies on dense pixel-level supervision and on the standard validation split for which fine annotations are provided; the 20,000 coarse-annotated images are not used

[68]. The native resolution of all images is 1024 by 2048 pixels, which is uniform across the dataset and makes batching straightforward without aspect-ratio handling.

The annotations themselves were produced by trained human annotators following a strict quality protocol described by the authors. Each fine-annotated image was labeled at the pixel level using polygon-based annotation tools, and the authors report that producing one fine annotation took approximately 1.5 hours of manual work on average [68]. The annotation process also included quality control passes by additional annotators which is one of the reasons why the fine-annotated set of 5,000 images is considerably smaller than the coarse-annotated set of 20,000 images, since fine pixel-level annotation does not scale efficiently with the volume of unlabeled data that can be collected from a vehicle fleet. This annotation cost is one of the principal motivations for studying SSL pre-training strategies in autonomous driving since SSL allows the backbone to be pre-trained on unlabeled data and only the downstream segmentation head plus the optional fine-tuning step to require any labeled data.

The fine-annotated set is split into three parts by the authors: 2,975 training images, 500 validation images and 1,525 test images [68]. The test set has its labels held server-side on the official Cityscapes evaluation server so that submissions are scored externally and the labels are never released publicly, which prevents tuning on the test set. The official server also limits the number of submissions per researcher per period to prevent indirect overfitting through repeated probing of the test labels [68]. In this thesis only the training and validation splits are used, with all four backbones being trained on the full 2,975 training images and evaluated on the full 500 validation images. Reporting results on the held-out test set was not pursued since the goal of this thesis is a controlled comparison between pre-training strategies under the same protocol and not a competitive submission to the Cityscapes leaderboard, and the validation split is enough to answer the research question. The training cities and validation cities are divided, with the validation set drawn from Frankfurt, Lindau and Münster [68], which gives a mild form of geographic generalization test built into the standard split and is discussed further in section 2.3.3 as part of the exploratory data analysis.

The class taxonomy used for evaluation contains 19 semantic classes that are grouped by the authors into seven higher-level categories: flat surfaces (road, sidewalk), construction (building, wall, fence), object (pole, traffic light, traffic sign), nature (vegetation, terrain), sky, human (person, rider) and vehicle (car, truck, bus, train, motorcycle, bicycle) [68]. The original raw annotations cover approximately 30 classes in total, but for evaluation only 19 of them are used since some of the remaining classes are too rare in the dataset to provide good enough supervision (for instance caravan, trailer and rail track), some of them are visually ambiguous or overlap with other already-evaluated classes (such as parking and ground which are difficult to separate reliably from road, sidewalk and terrain), and some of them are intentionally treated as non-informative regions of the image (such as the vehicle hood that appears at the bottom of every image and the rectification borders along the image edges) [68]. The

unused raw classes together with any pixels that the annotators marked as void are mapped to a special ignore index of 255 which is excluded from both the cross-entropy loss during training and the mIoU computation during evaluation, so that the model is not penalized on regions where the ground truth is not informative or not available. The mapping between the raw label IDs in the dataset files and the train IDs in the 19-class taxonomy is the standard one used in the official Cityscapes scripts and is implemented in the LABELID_TO_TRAINID dictionary in the data preparation code of this thesis. Cityscapes also provides instance-level annotations for the eight ‘thing’ classes (person, rider, car, truck, bus, train, motorcycle, bicycle) which support instance segmentation. The experiments in this thesis use only the semantic segmentation labels since the research question is about semantic transfer of pre-trained features and adding instance segmentation would introduce a second downstream architecture and break the shared-head comparison protocol that is the core of the experimental design. The full taxonomy with the official RGB color codes used for visualization is presented in table 2.

Table 2. Cityscapes 19 evaluation classes

№	Category	Class name	Train ID	RGB colour
1	Flat	Road	0	(128, 64, 128)
2	Flat	Sidewalk	1	(244, 35, 232)
3	Construction	Building	2	(70, 70, 70)
4	Construction	Wall	3	(102, 102, 156)
5	Construction	Fence	4	(190, 153, 153)
6	Object	Pole	5	(153, 153, 153)
7	Object	Traffic light	6	(250, 170, 30)
8	Object	Traffic sign	7	(220, 220, 0)
9	Nature	Vegetation	8	(107, 142, 35)
10	Nature	Terrain	9	(152, 251, 152)
11	Sky	Sky	10	(70, 130, 180)
12	Human	Person	11	(220, 20, 60)
13	Human	Rider	12	(255, 0, 0)
14	Vehicle	Car	13	(0, 0, 142)
15	Vehicle	Truck	14	(0, 0, 70)
16	Vehicle	Bus	15	(0, 60, 100)
17	Vehicle	Train	16	(0, 80, 100)
18	Vehicle	Motorcycle	17	(0, 0, 230)
19	Vehicle	Bicycle	18	(119, 11, 32)

A representative example of a Cityscapes image and its corresponding fine pixel-level annotation is shown on figure 9.



Figure 9. Representative Cityscapes image with its pixel-level annotation overlay

The image illustrates the typical composition of the dataset: an urban driving scene captured from the vehicle perspective with a wide horizontal field of view, dense annotations covering the entire image with no large unlabeled regions, and small classes such as persons, poles and traffic signs that are frequently partially occluded by larger structures like cars and buildings. Several visual challenges that appear across the dataset are also visible in this single example: the horizon line is variable depending on the road and the camera mounting; large connected regions of road, building and vegetation dominate the pixel count of every image and tend to bias models towards over-predicting these common classes; and the small classes that matter the most for driving safety such as person, rider and traffic sign appear only on a thin slice of the image and are often partially clipped by the image boundary or covered by other objects in the scene.

2.3 Exploratory data analysis

This section presents an exploratory data analysis of the Cityscapes training split that was conducted in this thesis to understand the visual statistics, the class composition and the geographic composition of the data before any model training is described. The analysis was performed with the script `cityscapes_edu.py` which is included in the code repository, and it produces the figures and the statistics that are reported in this section. The image-level statistics were measured on a random sample of 50 training images and the pixel-level class statistics were measured on a separate random sample of 200 training images, since a sample of this size is large enough to give a stable estimate of the statistics while keeping the EDA itself cheap to recompute. The training of the four backbones described later in this chapter uses the full set of 2,975 training images, the EDA samples reported here are not used as a training subset themselves and they are computed only for descriptive purposes.

2.3.1 Image-level statistics

All images in the Cityscapes fine-annotated set share the same native resolution of 1024 by 2048 pixels which was confirmed across the 50-image sample, with no variation in either dimension. The uniformity of the resolution simplifies the preprocessing pipeline since no aspect-ratio handling, padding or letterboxing is required, and the only resize step in the training pipeline is the half-resolution downscaling to 512 by 1024 pixels that is described in section 2.6.4.

The per-channel mean and standard deviation of the image pixel values, computed across the same 50-image sample after scaling the pixel values to the $[0, 1]$ range, are presented in table 3. The mean RGB vector measured on Cityscapes is $[0.296, 0.334, 0.291]$ and the standard deviation vector is $[0.182, 0.187, 0.183]$, which are visibly darker and less variable than the corresponding ImageNet statistics of mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$ reported for the original ImageNet ILSVRC training set [69]. The lower mean values reflect the visual character of the dataset since urban driving scenes are dominated by neutral surfaces such as asphalt road, concrete pavements, building facades and shadowed regions, all of which tend to have lower brightness than the diverse mix of natural and indoor scenes that ImageNet contains. The narrower standard deviation reflects the more constrained scene type which is restricted to urban street scenes from a roughly fixed camera viewpoint, while ImageNet contains a much wider variety of scene types, viewpoints, lighting conditions and object categories which produces a higher pixel variance overall.

Table 3. Per-channel mean and standard deviation of the image pixel values

№	Dataset	Mean R	Mean G	Mean B	Std R	Std G	Std B
1	Cityscapes (50-image sample)	0.296	0.334	0.291	0.182	0.187	0.183
2	ImageNet (ILSVRC)	0.485	0.456	0.406	0.229	0.224	0.225

Despite the visible difference between the Cityscapes statistics and the ImageNet statistics shown in table 3, the training pipeline used in this thesis keeps ImageNet normalization and not re-computing the normalization constants on Cityscapes. This choice is justified by the fact that all three pre-trained backbones being compared, the supervised ImageNet ResNet-50, the MoCo v3 ResNet-50 [70] and the DINO ViT-S/16 [71], were themselves pre-trained with ImageNet normalization, so the convolutional and attention features that were learned during pre-training are calibrated to the ImageNet input distribution. Re-normalizing the inputs to Cityscapes statistics during the downstream training step would shift the input distribution away from what the backbones expect and would partially invalidate the pre-training. The random-initialization backbone has no pre-training assumption to preserve, however applying the same normalization to it as well keeps the input distribution identical across all four backbones, which is consistent with the requirement of section 2.1 that everything except the backbone is held fixed across the experimental models.

2.3.2 Class distribution

The pixel-level class distribution of the training data was computed on a random sample of 200 training images, which is enough to produce a stable estimate of the share of each class in the data given that the Cityscapes images are large and densely annotated, so even 200 images provide on the order of $200 \times 1024 \times 2048 \approx 4.2 \times 10^8$ labeled pixels. The sample was processed with the “cityscapes_eda.py”

script which loads each fine-annotation file, applies the LABELID_TO_TRAINID mapping described in section 2.2 to convert the raw label IDs to the 19 train IDs, and accumulates the pixel count per class. The resulting per-class pixel share values are presented in table 4.

Table 4. Pixel share of each of the 19 Cityscapes evaluation classes

№	Class name	Pixel share (%)
1	Road	32.34
2	Building	20.72
3	Vegetation	13.59
4	Car	6.09
5	Sidewalk	5.65
6	Sky	3.73
7	Person	1.09
8	Pole	1.06
9	Terrain	0.944
10	Fence	0.841
11	Wall	0.716
12	Traffic sign	0.517
13	Bicycle	0.399
14	Train	0.21
15	Traffic light	0.181
16	Bus	0.154
17	Rider	0.147
18	Truck	0.098
19	Motorcycle	0.097

A visual representation of the table can be observed on the figure 10 below.

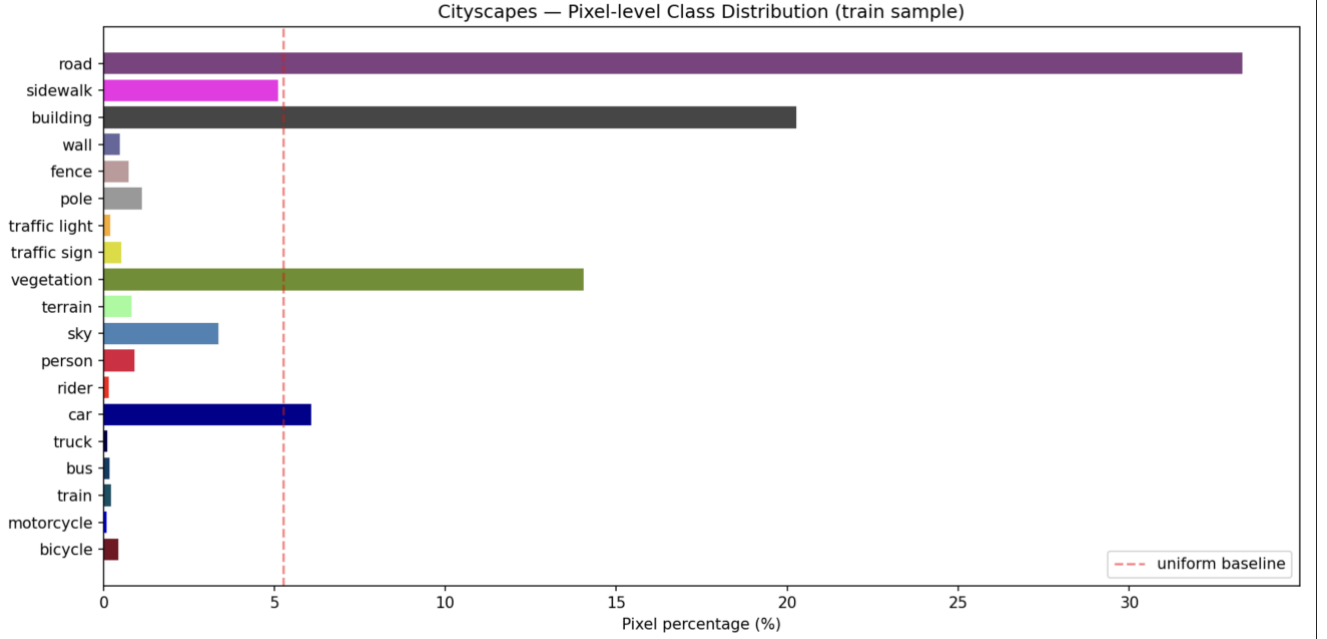


Figure 10. Pixel-level class distribution of the Cityscapes training data

The distribution shown on figure 10 is highly skewed. The largest single class is road which covers 32.34% of all labeled pixels, followed by building at 20.72% and vegetation at 13.59%, and these three classes together cover almost two thirds of the labeled pixel volume. The remaining common classes with at least 1%-pixel share are car (6.09%), sidewalk (5.65%), sky (3.73%), person (1.09%) and pole (1.06%). All eleven other classes fall under 1%-pixel share and form the long tail of the distribution: terrain (0.944%), fence (0.841%), wall (0.716%), traffic sign (0.517%), bicycle (0.399%), train (0.21%), traffic light (0.181%), bus (0.154%), rider (0.147%), truck (0.098%) and motorcycle (0.097%). The imbalance ratio between the most common class which is road and the least common class which is motorcycle is approximately 333 to 1, which means that for every pixel of motorcycle in the training data the model sees on average 333 pixels of road.

This long-tail pattern is a well-documented property of urban driving datasets and was discussed in the literature reviewed in chapter 1 [28, 38]. The S3PT work [28], for instance, motivates its scene- semantics-and-structure-guided clustering pre-training strategy specifically with the observation that pixel distributions in autonomous driving datasets are dominated by a small number of common classes with bicycles and motorcycles being among the smallest, which is the same pattern that emerges from the Cityscapes EDA conducted here. The practical meaning is that any model trained with a per-pixel cross-entropy loss on this distribution will receive a strong gradient signal for the common classes and a very weak gradient signal for the rare classes, which biases the learned representation towards the visual features of road, building and vegetation and away from the visual features of motorcycle, rider and truck.

This thesis tests the hypothesis that SSL pre-training keeps features for rare classes better than supervised ImageNet pre-training because the SSL pretext signal is not driven by per-class label gradient and is not biased towards the most common classes in the same way that supervised cross-entropy is. The DINO paper [71] reports that the self-attention maps produced by a self-distilled ViT often align with object boundaries even before any fine-tuning and that the attention focuses on the most visible visual structures in the image regardless of their frequency in the training set, which is a direct mechanism by which SSL pre-training may produce more useful features for the rare classes in Cityscapes. The MoCo v3 ResNet-50 [70] does not have the emergent-attention property of DINO but is also pre-trained without per-class labels, so the contrastive loss does not weight features by class frequency in the way supervised cross-entropy does.

2.3.3 Geographic distribution

The training set is composed of 2,975 fine-annotated images drawn from 18 different cities across Germany and a few neighboring countries, with the per-city image count taken from the city prefix in each image filename. The number of images per city is not uniform across the 18 cities, and the per-city image count is shown on figure 11.

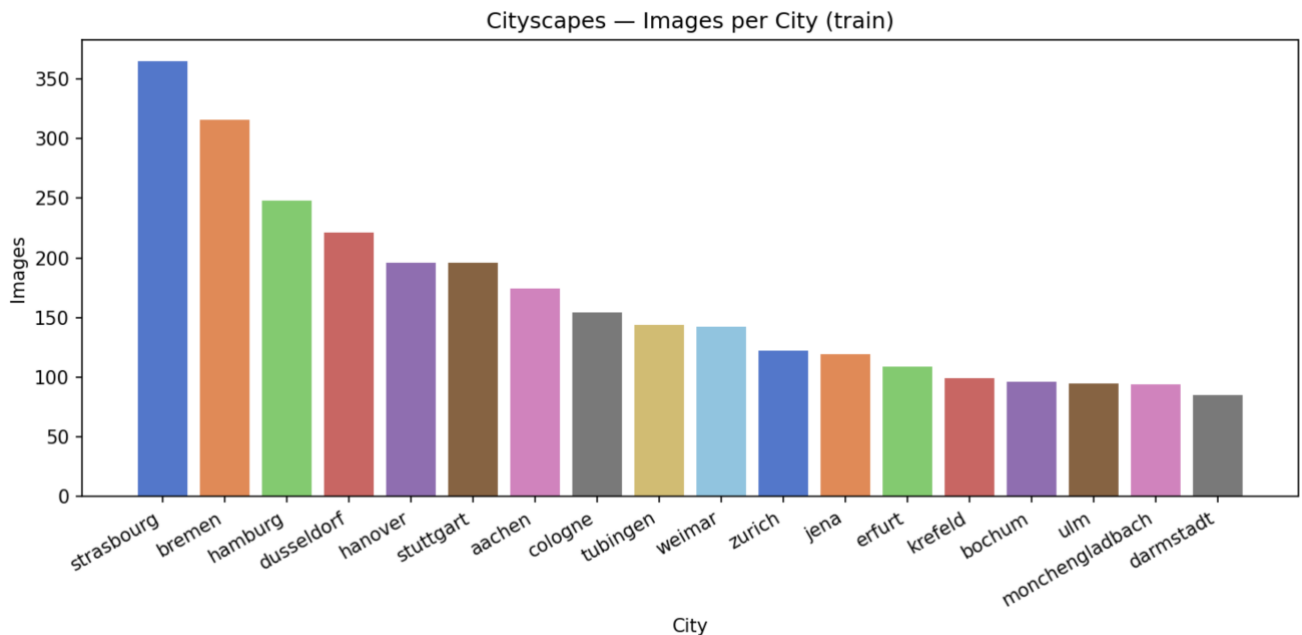


Figure 11. Number of training images per city in the Cityscapes training split

The most heavily represented city in the training split is Strasbourg with 365 images, while the least represented city is Darmstadt with 85 images. The top five cities by image count together cover roughly 50% of the entire training set, which means that half of the training signal comes from a small minority of the available cities. It is a property of the original Cityscapes split as released by the authors and is consistent across any future use of the public training split. The validation cities are Frankfurt,

Lindau and Münster as already mentioned in section 2.2, so the validation mIoU reported in chapter 3 is measured on cities that the model has never seen during training.

The skewed city distribution introduces a domain-shift risk between the training data and the validation data. A model that fits the over-represented cities particularly tightly will learn the street layouts, the building facade styles, the road textures of those cities, and these learned features may transfer to the three validation cities which have their own architectural and infrastructure characteristics. This risk is consistent with the broader concern about geographic and dataset biases in autonomous driving datasets that was raised in the chapter 1 problem statement [48], where it was noted that models trained in specific geographical regions or driving contexts often fail to generalize to new domains with different road infrastructure and traffic patterns.

2.3.4 Qualitative inspection

Beyond the quantitative statistics, a visual inspection of a small sample of images and their annotations gives a qualitative view of the dataset that the per-class pixel-share numbers cannot fully capture. A grid of six randomly sampled training images together with their fine pixel-level annotations is shown on figure 12.

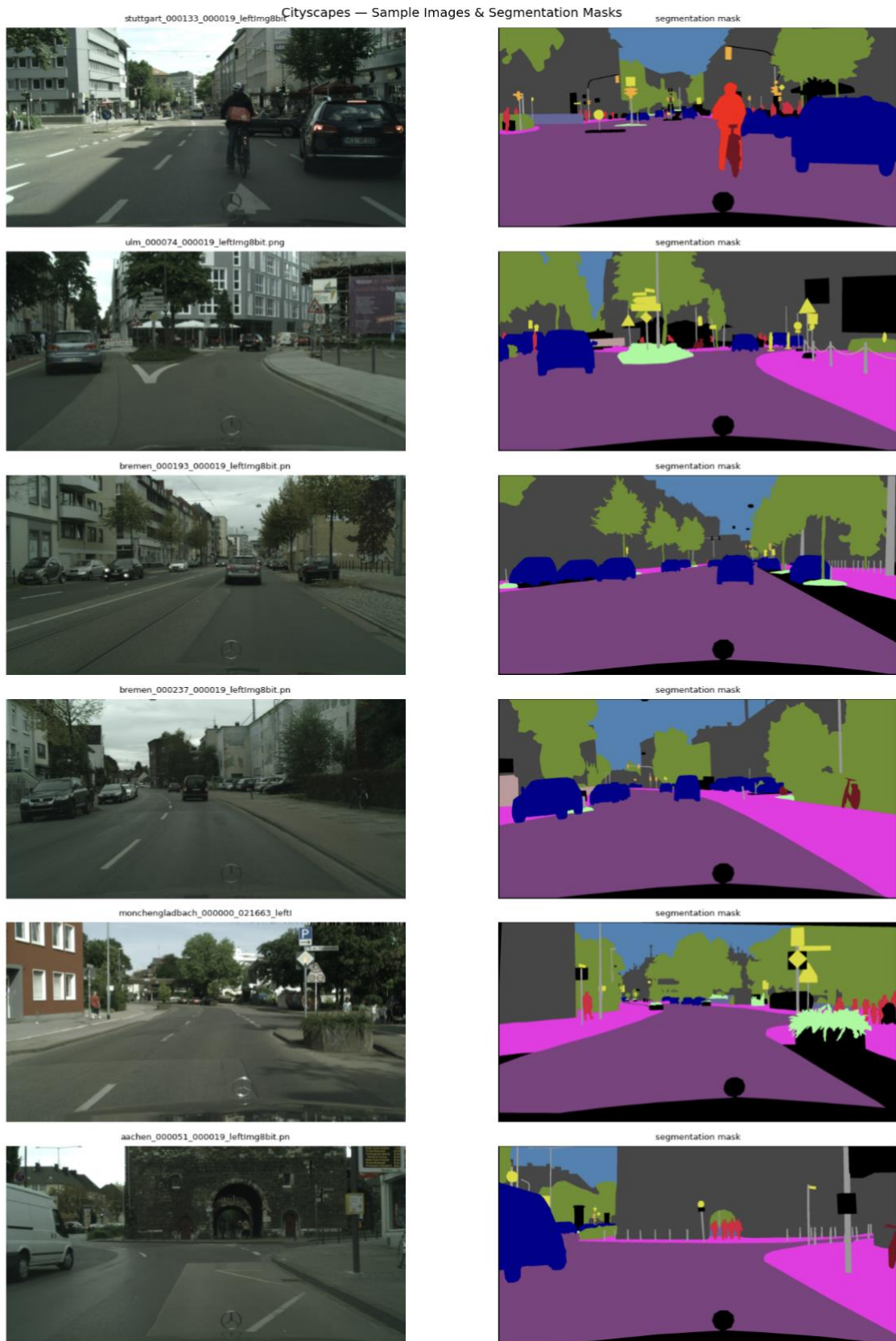


Figure 12. Six randomly sampled training images from the Cityscapes

The color legend that maps each of the 19 evaluation classes to its visualization color is shown on figure 13.

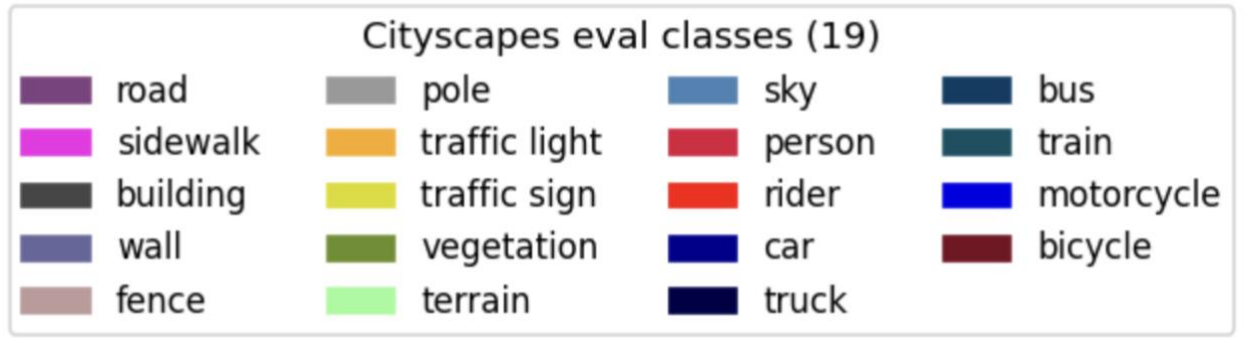


Figure 13. Color legend for the 19 Cityscapes evaluation classes

Several qualitative observations are visible from figure 12. First, every image is densely annotated with no large unlabeled regions, which is consistent with the high quality of the fine annotations described in section 2.2 and confirms the human annotation effort reported by the authors of the dataset. Second, the scenes are visually dense with each image contains many distinct object instances at multiple scales, with large background structures such as road, building, vegetation and sky typically occupying most of the image area and small objects such as poles, traffic signs, persons, riders and parked cars crowded into the remaining area. Third, the small classes are frequently partially covered by larger objects, for instance pedestrians are often partly hidden behind parked cars and traffic signs are often partly hidden behind branches of vegetation, which makes their pixel-level boundaries non easy to predict and which shows that the rare-class IoU is typically lower than the common-class IoU in segmentation models trained on this data. Fourth, the horizon line is changing across the sample images depending on the road grade, the camera mounting position and the local topography, which means that a model cannot rely on a fixed image region for the sky class or for the distant background and instead must learn a viewpoint-invariant representation. Fifth, the vehicle hood is visible at the bottom edge of some of the frames as a dark horizontal strip that is mapped to the ignore index in the annotations.

The color legend in figure 13 shows that the 19 evaluation classes were assigned visually distinct colors by the authors so that segmentation overlays remain readable in print and on screen, with related classes such as truck, bus, car and train all using shades of dark blue while clearly different categories such as person (red), road (purple-pink) and vegetation (olive green) sit far apart in color space. This color palette is the standard one used in the official Cityscapes scripts and is reused in all the visualizations in this thesis.

The exploratory data analysis confirms three properties of the training data that directly support the modelling decisions made in the rest of this chapter. The uniform image resolution at 1024 by 2048 pixels removes the need for any aspect-ratio handling in the preprocessing pipeline and allows a single

downscale-and-normalize step which is described in section 2.6.4. The mean and standard deviation of the Cityscapes image pixel values differ from the ImageNet normalization constants, but the training pipeline keeps ImageNet normalization to preserve the pre-training of the three pre-trained backbones. The long-tail class distribution with a 333 to 1 imbalance ratio between the most common and the least common class supports the per-class IoU reporting on the rare classes that is introduced in section 2.7.2, and motivates the hypothesis tested in section 3.4.2 that SSL pre-training may preserve rare-class features better than supervised ImageNet pre-training. The geographic skew of the training cities, which is roughly half of the training data in five cities out of eighteen, is acknowledged as a limitation of the single-dataset and single-region evaluation protocol and is discussed in section 3.4.5 of chapter 3.

2.4 Pre-training methods

This section describes in detail the four pre-training strategies that are compared in this thesis: a random-initialization baseline, supervised ImageNet pre-training, the MoCo v3 contrastive SSL approach and the DINO self-distillation SSL approach. Each subsection describes one method. The four backbones share the same downstream architecture, which is the segmentation head described in section 2.5, and they are trained with the same hyperparameters which are described in section 2.6, so the only variable that differs across the experiment is the pre-training method analyzed here.

The selection of random initialization, supervised ImageNet ResNet-50, MoCo v3 ResNet-50 and DINO ViT-S/16 covers the most informative four points in the design space given the compute budget available for this thesis. SimCLR [34] is conceptually subsumed by MoCo v3 since MoCo v3 incorporates the projection head and the augmentation set that SimCLR introduced and adds the queue replacement and the prediction head on top, and the official open-source SimCLR ResNet-50 checkpoints are also reported to be weaker than the corresponding MoCo v3 checkpoints in transfer evaluations [70], so SimCLR is not included as a separate experimental model. BYOL [76] is conceptually closer to DINO since both methods avoid negative samples by relying on a target network with EMA-based updates and stop-gradient on the target path, and DINO has the additional emergent-attention property described in section 2.4.4 above which is more directly relevant to the dense-prediction question of this thesis, so BYOL is also not included. SLidR [26] which was reviewed in chapter 1 requires synchronized LiDAR and camera data for its image-to-LiDAR contrastive distillation objective, however Cityscapes does not provide LiDAR point clouds as already noted in section 2.2, so SLidR cannot be evaluated on this dataset and is out of scope for this thesis. S3PT [28] also reviewed in chapter 1 requires multi-GPU pre-training at a scale that exceeds the single-GPU compute budget of this thesis, so it is also not pre-trained from scratch here. Finally, MAE [36] which was discussed in chapter 1 as a generative SSL approach based on masked image modelling would have been a fourth pre-training family worth comparing alongside MoCo v3 and DINO, however adding it would expand the

experimental matrix beyond what fits in the compute budget and it is left for future work as discussed in section 3.6.2 of chapter 3.

2.4.1 Random initialization

The first backbone is a ResNet-50 [72] with no pre-training, where all convolutional weights are initialized with the Kaiming initialization scheme [74] and the batch normalization parameters with the standard PyTorch defaults of $\gamma = 1$ and $\beta = 0$. This corresponds to the default behavior of the torchvision ResNet-50 constructor when called without a weights argument, which is the way the random-init backbone is created in the implementation script of this thesis. The Kaiming initialization was proposed by He et al. for deep networks and is designed to keep the variance of the activations approximately constant across layers, which prevents the gradients from vanishing or exploding in the deeper stages of the network and which is the standard for convolutional networks of this depth [74]. Since the same initialization scheme would also be applied silently to any randomly initialized parts of the other ResNet-50 backbones if they were not loaded from a checkpoint, using it here does not introduce any architectural bias to the pre-trained backbones in this thesis.

The purpose of this baseline is to provide a lower-bound reference for the segmentation experiments. It shows what mIoU is achievable on Cityscapes when the backbone has no transferable features at all and any segmentation performance has to be learned from scratch from the 2,975 training images alone, so any positive performance gap between this random-init backbone and the pre-trained backbones can be assigned entirely to the pre-training step and not to the architecture or to the downstream optimization. The random-init backbone is run only in the fine-tune mode and not in the linear probe mode, since linear probing on top of a randomly initialized feature extractor is known to produce uninformative results because the features are not aligned with any meaningful signal and a linear classifier cannot be expected to recover useful structure from random parts of the input. This method of evaluating linear probe only on top of pre-trained features is the one used in SimCLR [34], in MoCo v3 [70] and in DINO [71], and is followed in this thesis as well to keep the comparison consistent with the literature.

2.4.2 Supervised ImageNet pre-training

The second backbone is a ResNet-50 [72] pre-trained with supervised classification on the ImageNet ILSVRC-2012 dataset [69, 73] across 1,000 object categories. ImageNet ILSVRC-2012 contains approximately 1.28 million training images that were drawn from the broader ImageNet image database [69] and labeled across 1,000 leaf categories of the WordNet hierarchy as part of the ILSVRC challenge organized by Russakovsky O. et al. [73]. The supervised pre-training step minimizes a per-sample softmax cross-entropy loss over the 1,000 categories, which is the standard supervised classification objective used in the original ResNet paper [72], and which has become the default reference point for transfer learning in computer vision over the past decade. The training of the

supervised pre-trained checkpoint is not repeated in this thesis since the goal is to compare publicly available pre-training strategies and not to reproduce them from scratch, and the official torchvision checkpoint identified by the constant `ResNet50_Weights.IMAGENET1K_V1` in the implementation script is loaded directly. This checkpoint is the default supervised ImageNet ResNet-50 weights distributed with torchvision and is the most used reference in the SSL transfer literature [70, 71], which makes the results of this thesis directly comparable to the dense-prediction transfer numbers reported in those papers.

The architecture used as the segmentation backbone is the truncated ResNet-50 with the classification head removed. The implementation in this thesis loads the full torchvision ResNet-50, takes the children of the model up to the global average pool and the linear classifier, and uses the resulting truncated network as the dense feature extractor. For an input image of shape $3 \times H \times W$, the truncated backbone returns a feature tensor of shape $2048 \times H/32 \times W/32$, since the standard ResNet-50 contains five stages of stride-2 downsampling between the input layer and the final convolutional block. For the Cityscapes input resolution of 512×1024 used in this thesis, the spatial output of the backbone is $2048 \times 16 \times 32$, which is the input that is consumed by the segmentation head in section 2.5. The schematic of the truncated backbone and of the layers that are kept is displayed on figure 14.

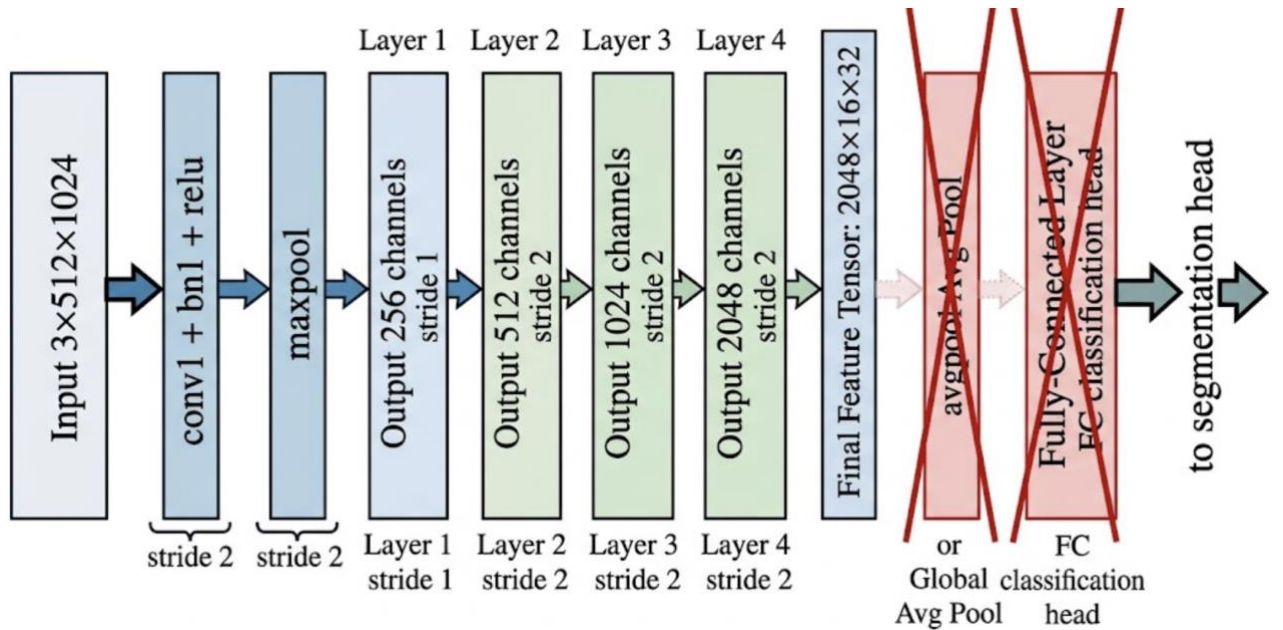


Figure 14. Truncated ResNet-50 backbone [72]

The supervised ImageNet ResNet-50 is the correct supervised baseline for the comparison in this thesis. The backbone capacity is the same as for the MoCo v3 backbone since both use the ResNet-50 architecture, the pre-training data is the same since both are pre-trained on ImageNet at the ILSVRC-2012 scale, and the only factor that differs between the supervised baseline and the MoCo v3 backbone

is the pre-training supervision signal: per-class label cross-entropy in the supervised case versus a self-supervised contrastive objective without labels in the MoCo v3 case. Any difference in segmentation mIoU between the two can therefore be assigned to the supervision signal and not to the architecture or to the data scale, which is the cleanest possible isolation of the SSL contribution given the publicly available pre-training resources. The general form of the contrastive loss that MoCo v3 uses was already introduced as formula (1) in chapter 1 of this thesis from the SimCLR paper [34], and the present section reuses the same form of contrastive objective but with the specific choices of dictionary, momentum encoder and projection head that distinguish MoCo v3 from the more general formulation, all of which is described in section 2.4.3 below.

2.4.3 MoCo v3

Contrastive learning follows the principle that two augmented views of the same image should produce similar feature representations, while views of different images should produce not similar feature representations. The logic is that if the features are constant to a wide range of augmentations such as random crops, color jitter, blur and grayscale conversion but still preserve enough information to distinguish one image from another in a large set, then those features are likely to encode the semantic content of the image instead of its low-level appearance, which makes them useful for downstream tasks like classification, detection and semantic segmentation [34, 7]. SimCLR [34] showed that this idea works at scale on ImageNet using a large training batch as the source of negative samples and an MLP projection head between the encoder backbone and the contrastive loss, while MoCo v1 [7] showed that comparable representations can be learned without the very large batches required by SimCLR by using a queue-based dictionary of recent feature vectors as the source of negatives and a momentum-updated key encoder that produces consistent key features across iterations.

MoCo v3 [70] is the third iteration of the Momentum Contrast family of contrastive SSL methods. MoCo v1 [7] introduced the two key components that distinguish the family from SimCLR: a queue of feature vectors that acts as a large dictionary of negative samples without requiring a large training batch, and a momentum-updated key encoder whose weights are an exponential moving average of the query encoder weights and which produces consistent key features across training iterations. MoCo v2 [75] retained these two components and added two improvements borrowed from SimCLR. An MLP projection head between the encoder backbone and the contrastive loss, and a stronger augmentation set including blur and stronger color jitter. These changes brought MoCo v2 to comparable accuracy with SimCLR on the ImageNet linear probe with a much smaller training batch.

MoCo v3 [70] makes three changes to the MoCo design relative to MoCo v2. First, the queue of negative samples is dropped, and the in-batch negatives are used instead, which is closer to the SimCLR design and is enabled by the larger batches that became practical when this paper was written. Second, an additional prediction head is added on top of the projection head on the query side, which is borrowed

from the BYOL design [76] and which empirically improves the stability of training when ViT backbones are used. Third, the loss is formulated symmetrically: the query encoder produces a feature that is matched against the key produced by the momentum encoder for the same image and against the keys of all other images in the batch, and the same is then done with the query and key roles swapped which doubles the gradient signal per training step. The two encoders are otherwise identical in architecture, and the key encoder is updated as an exponential moving average of the query encoder and not by gradient descent.

The InfoNCE contrastive loss used by MoCo v3 [70] is presented in formula 5.

$$L_q = -\log \frac{\exp(q \cdot k^+ / \tau)}{\sum_{i=1}^N \exp(q \cdot k_i / \tau)} \quad (5)$$

where:

q = query feature produced by the query encoder followed by the projection head and the prediction head

k^+ = positive key feature produced by the momentum-updated key encoder for the same image as q

k_i = each of the N key features in the batch which contain one positive and $N - 1$ negatives

τ = temperature parameter that scales the dot products

$q \cdot k$ = dot product of two L2-normalised feature vectors which is equivalent to cosine similarity

The momentum update rule for the key encoder is presented in formula 6 which produces a slowly evolving teacher signal that stabilizes the contrastive loss [70, 7].

$$\theta_k \leftarrow m \cdot \theta_k + (1 - m) \cdot \theta_q \quad (6)$$

where:

θ_k = parameters of the key encoder

θ_q = parameters of the query encoder updated by gradient descent

m = momentum coefficient typically set to 0.99 or larger

The training schematic of MoCo v3 is displayed on figure 15.

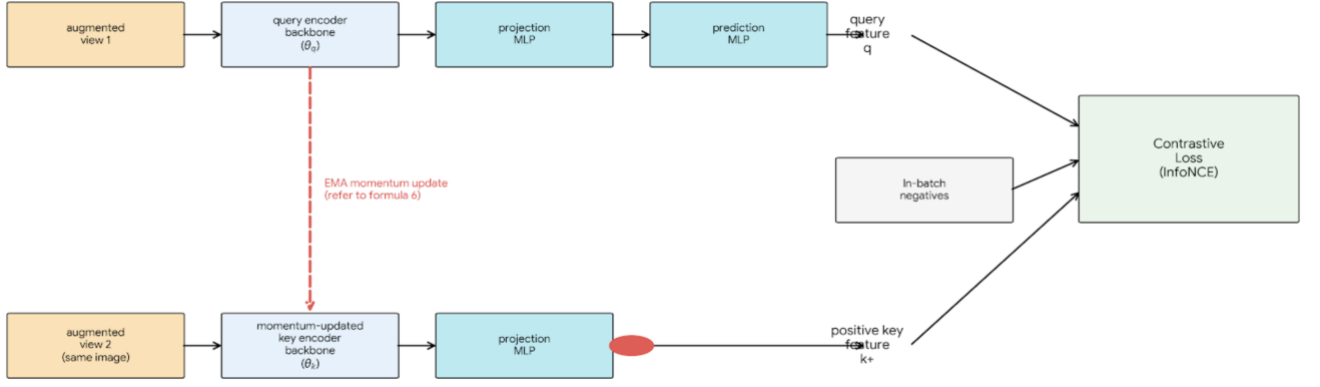


Figure 15. Training schematic of MoCo v3 [70]

The specific MoCo v3 checkpoint loaded in this thesis is the official ResNet-50 1000-epoch checkpoint released by the authors of the MoCo v3 paper. The checkpoint is downloaded from the official URL referenced in the implementation script [101] and is loaded into the truncated ResNet-50 backbone described in section 2.4.2 above, with the projection head, the prediction head and any classification layers stripped during the load. MoCo v3 is supposed to learn that the set of low-level and mid-level visual differences that are forced by the contrastive augmentation set: invariance to small spatial crops, to color jitter, to blur and to grayscale conversion, while remaining sensitive to the global semantic content of the image which allows the model to discriminate between different images in the training set. The original MoCo v3 paper [70] reports competitive dense-prediction transfer numbers on COCO detection and instance segmentation when this contrastive ResNet-50 is used as the backbone, which is the primary published evidence that the learned features are useful for tasks that require pixel-level reasoning and not only image-level classification.

2.4.4 DINO

DINO [71] is a self-distillation SSL method that does not use negative samples, does not use a contrastive loss and does not require an explicit cluster assignment. Instead, two networks of the same architecture (a student and a teacher) are trained together: the student is trained by gradient descent to match the output distribution of the teacher on different augmented views of the same image, while the teacher is updated as an exponential moving average of the student weights. To prevent the collapse where both networks output a constant vector for every input, DINO applies a centering operation to the teacher output (subtraction of a running mean over the batch) and a sharpening operation (division by a low temperature followed by softmax), and the combination of centering and sharpening keeps the teacher output away from both the uniform-distribution collapse and the one-hot collapse [71]. This idea places DINO in the non-contrastive learning bucket that was already discussed in chapter 1 of this thesis, alongside other non-contrastive methods like BYOL [76] which also avoids negatives by relying on a target network whose updates are stop-gradient and EMA based.

The DINO backbone used in this thesis is a Vision Transformer ViT-S/16 [77]. Unlike the ResNet-50 backbones used in the other three experimental conditions, ViT-S/16 is not a convolutional network and does not have the spatial inductive bias that comes with stacked convolutions. The input image is split into non-overlapping square patches of 16 by 16 pixels, each patch is linearly projected to a 384-dimensional patch token, a learned positional embedding is added to each token, and the resulting sequence of tokens is processed by a stack of 12 self-attention transformer blocks each with 6 attention heads [77]. A special learned CLS token is added to the sequence at the input layer and is the token whose final-layer embedding is used by the original ViT for image classification. For the Cityscapes input resolution of 512 by 1024 used in this thesis. The patch grid is 32 by 64 patches which gives 2048 patch tokens plus the CLS token, and the resulting feature tensor at the output of the transformer stack is of shape 2049 x 384.

The DinoBackbone wrapper in the implementation script of this thesis is responsible for converting this sequence-shaped output into a spatial feature map that the segmentation head can consume. The wrapper discards the CLS token from the final-layer output, takes the remaining 2048 patch tokens, and reshapes them into a spatial feature grid of shape 384 x 32 x 64, which corresponds to the original spatial layout of the patches in the input image. This 384 x 32 x 64 tensor is then upsampled by the segmentation head described in section 2.5 with an upsample factor of 16 to recover the full 19 x 512 x 1024 logit tensor at the input resolution. The lower upsample factor of 16 (compared to 32 for the ResNet-50 backbones) reflects the smaller stride of the patch tokenization and is the only structural difference between the segmentation pipelines of the DINO backbone and the three ResNet-50 backbones.

The DINO loss is a cross-entropy between the sharpened, centered teacher output distribution and the student output distribution, computed over a learned set of K prototypes that act as a discrete vocabulary in the projection space [71]. The forward pass produces a K-dimensional logit vector per network per view, which is converted to a probability distribution by softmax. The loss is presented in formula 7.

$$L_{\text{DINO}} = - \sum_x P_t(x) \cdot \log P_s(x) \quad (7)$$

where:

$P_t(x) = \text{softmax}((g_t(x) - C)/\tau_t)$ is the centred and sharpened teacher distribution over K prototypes for view x

$P_s(x) = \text{softmax}(g_s(x)/\tau_s)$ is the student distribution over the same K prototypes for view x

g_t, g_s = projection heads on top of the teacher and the student backbones respectively

C = running centre vector of the teacher output that is subtracted before the softmax

τ_t, τ_s = teacher and student temperatures with τ_t typically much smaller than τ_s

The teacher backbone parameters θ_t are updated as an exponential moving average of the student parameters θ_s , presented in formula 8, and the center vector C is updated as an exponential moving average of the teacher output across the current batch, presented in formula 9. Both moving averages serve to prevent representation collapse and to stabilize the training [71].

$$\theta_t \leftarrow \lambda \cdot \theta_t + (1 - \lambda) \cdot \theta_s \quad (8)$$

$$C \leftarrow \alpha \cdot C + (1 - \alpha) \cdot \frac{1}{B} \sum_{x \in \text{batch}} g_t(x) \quad (9)$$

where:

θ_t, θ_s = parameters of the teacher and the student backbones respectively

λ = teacher EMA coefficient typically scheduled from 0.996 upwards during training

α = centring EMA coefficient

B = current batch size

The training schematic of DINO is depicted on figure 16.

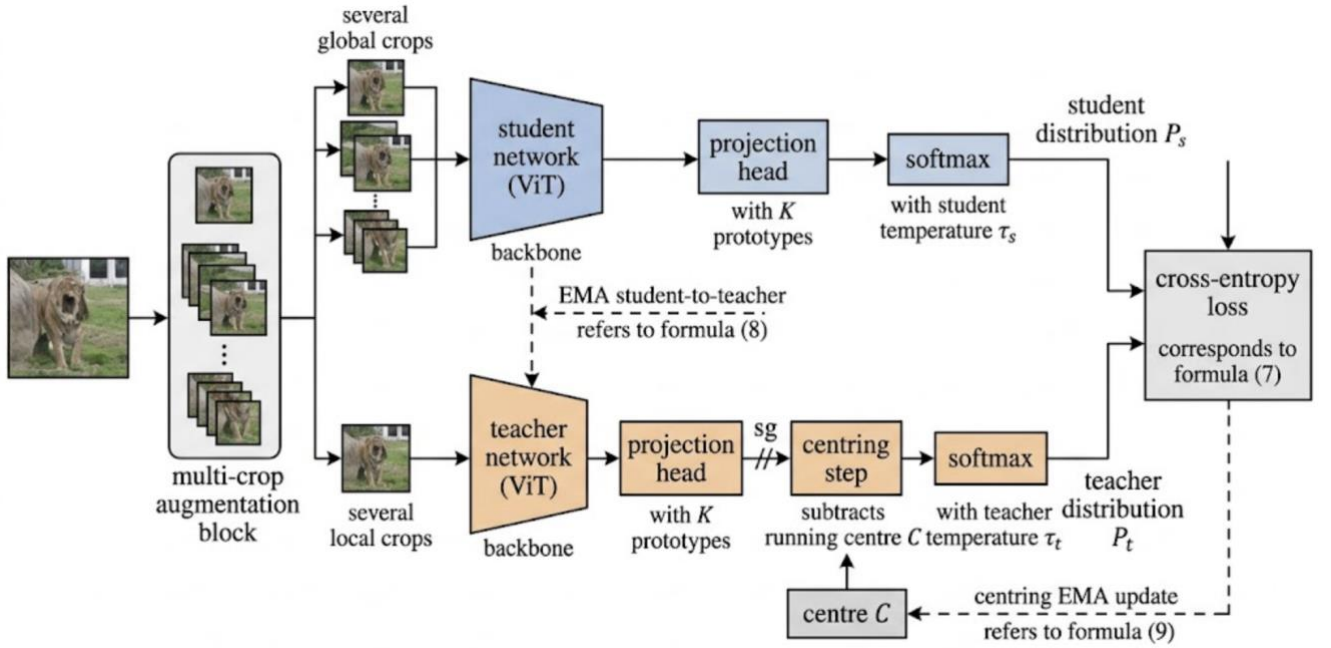


Figure 16. Training schematic of DINO [71]

A relevant property of DINO for the semantic segmentation experiments in this thesis is that the self-attention maps produced by the self-distilled ViT often align with object boundaries even before any fine-tuning is done [71]. The original paper visualizes the attention maps of the CLS token across the heads of the last transformer layer and shows that on natural images the top attention heads cluster on semantically meaningful regions such as the foreground object. This attention has been highlighted by the authors of DINO as a property that does not appear in supervised ViT pre-training and that does not appear in the contrastive SSL methods to the same degree, and it is a direct mechanism by which DINO features may be more useful for dense prediction tasks like semantic segmentation than features learned by contrastive or by supervised methods. This property is one of the main motivations for including DINO as one of the four backbones compared in this thesis, and the qualitative attention maps produced by the loaded DINO ViT-S/16 are revisited in section 3.3.3 of chapter 3 alongside the segmentation predictions.

The DINO checkpoint loaded in this thesis is the official ViT-S/16 DINO checkpoint released by the authors and obtained via the torch.hub mechanism from the facebookresearch/dino:main repository as referenced in the implementation script. ResNet-50 has approximately 25 million parameters whereas ViT-S/16 has approximately 21 million parameters, and the two architectures differ in their inductive biases. This thesis does not attempt a controlled architecture match, which would require pre-training MoCo v3 on a ViT-S/16 and DINO on a ResNet-50 from scratch. The best available open-source SSL checkpoint for each method as released by the original authors is being compared instead, which is also the same comparison made in most published SSL transfer studies [70, 71]. The architecture-pre-training is acknowledged as a known limitation of the experimental design and is discussed again in section 3.4.4 of chapter 3 [96].

2.5 Downstream architecture: segmentation head

This section describes the small segmentation head that is on top of each of the four pre-trained backbones described in section 2.4 to produce per-pixel class predictions for Cityscapes. The head is identical across all four experimental conditions and only the final upsampling factor differs between the ResNet-50 and the ViT-S/16 backbones, so any difference in segmentation mIoU between the experiment reported in chapter 3 is attributable to the backbone and to the pre-training method, and not to any architectural variation in the downstream model.

2.5.1 Downstream design

The segmentation head used in this thesis is a small fully convolutional network in the lineage of the original FCN architecture proposed by Long J., Shelhamer E. and Darrell T. [78]. The head receives a low-resolution feature tensor from the backbone and produces a logit tensor at the input resolution by applying three convolutional layers followed by a single bilinear upsampling step. Concretely the head consists of a 3 x 3 convolution that reduces the channel dimension from the

backbone output (2048 channels for the ResNet-50 backbones and 384 channels for the ViT-S/16 backbone) down to 256 channels, followed by batch normalization and a ReLU activation; then a second 3×3 convolution that further reduces the channel dimension from 256 to 128 channels, again followed by batch normalization and a ReLU activation; then a 1×1 convolution that maps the 128 channels to the 19 output classes; and finally a bilinear interpolation step that upsamples the resulting logit tensor to the full input resolution of 512×1024 pixels. The schematic of the head and how it attaches to each backbone is displayed on figure 17.

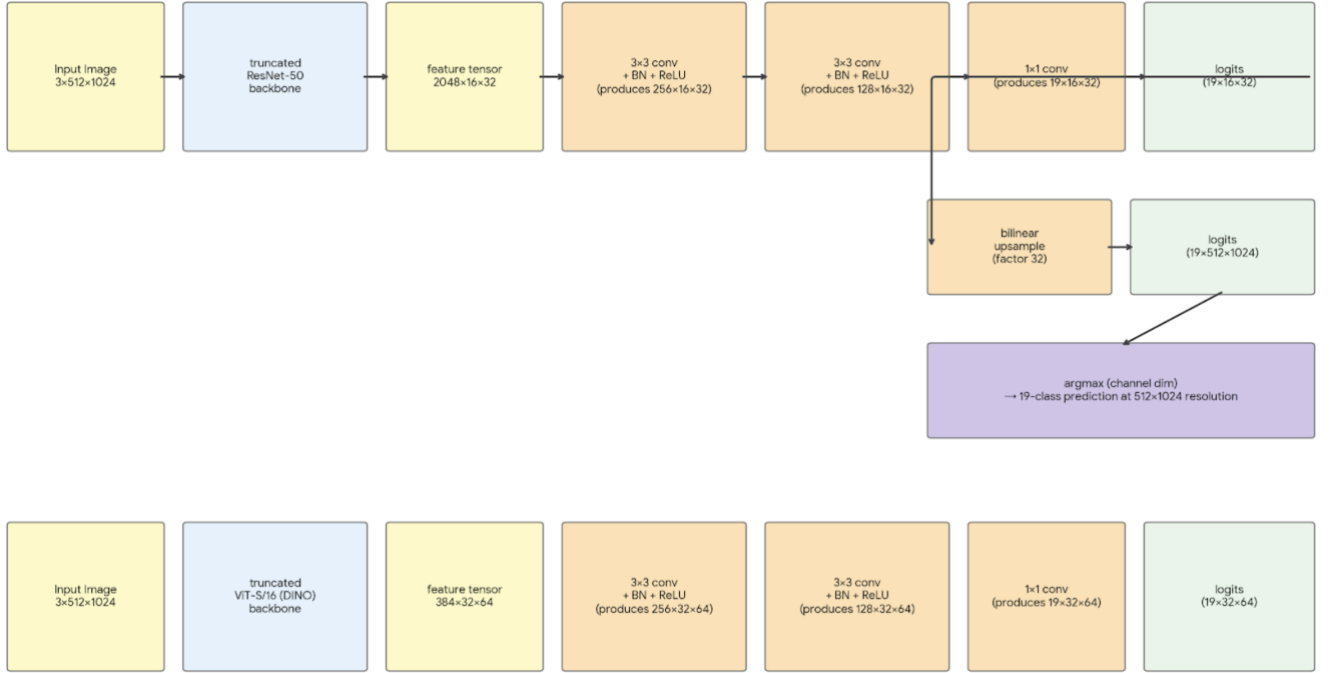


Figure 17. Segmentation head

The implementation in the SegmentationHead module of the script accepts the backbone output tensor, passes it through the three-block decoder defined above, and applies the upsampling at the very end. The number of output classes is fixed at 19 to match the Cityscapes evaluation taxonomy described in section 2.2. The upsampling uses bilinear interpolation with `align_corners` set to `False` which is the standard PyTorch convention for semantic segmentation, and the same module is instantiated identically for each of the four backbones with the input channel count being the only constructor argument that differs between models.

A heavier downstream architecture such as DeepLab v3+, UPerNet or any of the recent transformer-based segmentation heads would almost certainly produce higher absolute mIoU on Cityscapes than the small FCN head described above, however a heavier head would also undermine the comparison that this thesis is set up to make. The variable under study is the quality of the pre-trained backbone features and the way they transfer to dense prediction, and a heavy head with its own large

parameter budget has the capacity to compensate for weak backbone features by relearning the missing semantics within the head itself, which masks the contribution of the pre-training step and conflates “good backbone” with “good head”. A lightweight head with very limited capacity has the opposite property: it cannot recover from weak backbone features and so the segmentation mIoU it produces is a more direct readout of how much useful signal is present in the backbone output. This is the same methodological argument made by the authors of MoCo v3 [70] and DINO [71] in their dense-prediction transfer protocols where they intentionally use lightweight decoders on top of the pre-trained backbone for exactly the reason of isolating the backbone contribution. The MAE paper [36] which was reviewed in chapter 1 makes a related argument when it reports linear probe and lightweight fine-tune protocols separately from heavy-head fine-tune. The choice in this thesis follows the same configuration and the head architecture is kept deliberately small.

The output of the head is a logit tensor of shape $19 \times 512 \times 1024$ which is converted to a per-pixel class prediction by taking the argmax across the channel dimension, producing a prediction tensor of shape 512×1024 with integer class IDs in the range 0 to 18. During training the prediction is compared to the resized label image at the same 512×1024 resolution using the cross-entropy loss with the ignore index of 255 set to skip the void pixels described in section 2.2. At evaluation time the same argmax prediction is compared to the validation label image using the mean IoU metric defined in section 2.7.1, with the ignore index again excluded from both the numerator and the denominator of every per-class IoU. The full input-output flow including the head and the backbone is the one already shown on figure 8 in section 2.1.

2.6 Training protocol

This section describes the training protocol that is shared across the experiment: the two evaluation modes (linear probe and full fine-tune), the hyperparameter values, the hardware setup and the image preprocessing pipeline. As with the head, the protocol is designed to keep everything constant across the experiment except the backbone and the freeze flag.

2.6.1 The two evaluation modes

Each pre-trained backbone is evaluated under two protocols: linear probe and full fine-tune. In the linear probe mode, the backbone is frozen by setting `requires_grad` to False on every parameter of the backbone, and only the segmentation head described in section 2.5 is trained by gradient descent. The probe measures how useful the pre-trained features are as a fixed feature extractor for dense prediction, which is a direct readout of the quality of the pre-training step. In the full fine-tune mode, no parameters are frozen and the entire network (backbone plus head) is updated end-to-end by gradient descent. The fine-tune mode measures the upper bound on segmentation mIoU that the model can reach with the given pre-training as a starting point and the available training data and compute budget.

Both protocols are run for every backbone except the random-init baseline which is run only in the fine-tune mode. The two-protocol design is the same one used in the original SimCLR transfer evaluation [34] and in the DINO transfer evaluation [71], where the linear probe is taken as the primary readout of feature quality and the fine-tune mIoU is reported as the upper-bound complementary measurement. The difference between the two modes per backbone is an interpretive quantity: a small difference indicates that the pre-trained features were already close to ready for dense prediction and that the frozen backbone is competitive with end-to-end training, while a large difference indicates that the frozen features required adaptation in order to produce competitive segmentation predictions.

2.6.2 Hyperparameters

The hyperparameter values used for the experiment in this thesis are presented in table 5. All values are taken directly from the implementation script and are identical across the experiment, with the single exception of the backbone-frozen flag which depends on the evaluation mode.

Table 5. Training hyperparameters

Nº	Hyperparameter	Value
1	Input resolution	512 x 1024 pixels
2	Batch size	32
3	Number of epochs	20
4	Optimizer	Adam with default $\beta_1 = 0.9$ and $\beta_2 = 0.999$
5	Initial learning rate	2×10^{-4}
6	Learning rate schedule	CosineAnnealingLR with $T_{\text{max}} = 20$
7	Loss function	CrossEntropyLoss with <code>ignore_index = 255</code>
8	Augmentations	none (Resize, ToTensor, Normalize only)
9	Number of output classes	19
10	Backbone-frozen flag	True for linear probe, False for full fine-tune

The optimizer is the standard Adam optimizer without any modification to the default β coefficients, which is the most used optimizer for small-scale segmentation fine-tuning experiments and is the one used by the SSL transfer protocols of MoCo v3 [70] and DINO [71]. The learning rate is fixed at 2×10^{-4} for both probe and fine-tune models, with a cosine schedule that brings the learning rate towards zero over the 20 epochs of training. The batch size of 32 is enabled by the 80 GB of GPU memory of the A100 instance described in section 2.6.3 and is shared between probe and fine-tune and across all four backbones, since holding the batch size constant across the experiment keeps the optimization directly comparable and removes batch size as a confound between the experimental conditions. The cross-entropy loss aggregates per-pixel losses over the entire 19-class output and the `ignore_index` of 255 makes sure that the void pixels described in section 2.2 do not contribute to the gradient. The number of epochs of 20 is sufficient for the validation mIoU curve to stale in the dev runs

that were used to debug the pipeline, which suggests that more epochs at the same learning rate would not change the conclusions of chapter 3 in any informative way.

2.6.3 Hardware and runtime

The experiment of the controlled comparison was run on a Google Colab Pro+ A100 instance with 80 GB of GPU memory and 280 GB of system RAM, which is the high-memory tier of the A100 accelerator available through Colab Pro+ and which is the same compute machine across the experiment of this controlled comparison. The 80 GB of GPU memory is what makes the batch size of 32 in section 2.6.2 possible at the 512 x 1024 input resolution with the ResNet-50 backbones, and the 280 GB of system RAM allows the full set of 2,975 training images to be cached in memory across epochs which keeps the per-epoch wall-clock dominated by the GPU forward and backward passes and not by disk reads from the mounted dataset volume. The total GPU time spent to run the experiment is approximately 6 A100 GPU-hours. The follow-up fine-tuning experiment described in section 2.9 was run on a separate machine with a NVIDIA RTX PRO 6000 Blackwell Server Edition GPU with 102 GB of GPU memory, since the larger 768 x 768 random crops, the additional auxiliary head, the deeper DPT-Lite decoder and the mixed-precision activations of that experiment together exceed the memory available on the A100 80 GB instance used for the controlled comparison and the larger memory of the RTX PRO 6000 was needed to keep the same batch size and the same input crop without resorting to gradient checkpointing or to gradient accumulation.

2.6.4 Image preprocessing pipeline

The image preprocessing pipeline is identical across the experiment and consists of three steps applied to every input image: a resize from the native resolution of 1024 x 2048 to the input resolution of 512 x 1024 pixels using bilinear interpolation, a conversion to a PyTorch float32 tensor in the range [0, 1] via ToTensor, and a per-channel normalization using the ImageNet mean and standard deviation values [0.485, 0.456, 0.406] and [0.229, 0.224, 0.225] discussed in section 2.3.1. The corresponding label image is resized in parallel from the native 1024 x 2048 to the matching 512 x 1024 resolution using nearest-neighbor interpolation and not bilinear, since nearest-neighbor preserves the integer class IDs of the annotations exactly and does not recreate intermediate class IDs at object boundaries which would happen if bilinear interpolation were applied to a label image. The label resize is followed by the LABELID_TO_TRAINID lookup described in section 2.2 which converts the raw label IDs to the 19 train IDs plus the ignore index of 255.

The pipeline contains no augmentations of any kind. There is no random crop, no horizontal flip, no color jitter, no scale jitter and no other form of stochastic transformation between the loader and the model. If different models saw different random augmentations during training, the input distribution to each backbone would be slightly different on every training step and any difference in segmentation mIoU between the models would partly reflect the augmentation pipeline instead of the backbone alone.

By holding the input distribution identical across the experiment, the protocol isolates the pre-training method as the only varying factor. This approach is consistent with the convention used in the dense-prediction transfer evaluations of MoCo v3 [70] and DINO [71] which omit augmentations during the linear probe step and use only minimal augmentation during the fine-tune step. Adding more aggressive augmentations would likely improve the absolute mIoU of every model by some amount, however it would not change the relative ordering between the models in any informative way, and it is left out of the experiment used here.

2.7 Evaluation metrics

This section describes the four metrics used to evaluate the experiment in chapter 3: mean Intersection over Union, per-class IoU on the rare classes, the probe-versus-fine-tune delta and inference frames per second on the A100 GPU.

2.7.1 Mean Intersection over Union

The primary evaluation metric used in this thesis is the mean Intersection over Union (mIoU), which is the standard segmentation metric used in the Cityscapes benchmark [68] and in the semantic segmentation literature reviewed in chapter 1 [61]. The per-class Intersection over Union is the ratio of the number of pixels correctly classified as belonging to the class to the total number of pixels predicted as the class or labeled as the class. It is presented in formula 10.

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c} \quad (10)$$

where:

c = class index in the 19-class Cityscapes taxonomy

TP_c = number of pixels correctly predicted as class c (true positives for class c)

FP_c = number of pixels predicted as class c that are some other class (false positives for class c)

FN_c = number of pixels labeled as class c that the model predicted as some other class (false negatives for class c)

The mean IoU is the average of the per-class IoU values over the classes that are present in the evaluation tensor, presented in formula 11.

$$\text{mIoU} = \frac{1}{|C_{\text{present}}|} \sum_{c \in C_{\text{present}}} \text{IoU}_c \quad (11)$$

where:

C_{present} = set of classes for which the union $TP_c + FP_c + FN_c$ is non-zero in the current evaluation tensor

Classes with an empty union are skipped from the mean and not counted as zero, which matches the implementation in the `compute_miou` function of the implementation script. The ignore index of 255 is excluded from both the numerator and the denominator of every per-class IoU, so the void pixels do not influence the metric. The validation mIoU reported in chapter 3 is computed on the full Cityscapes validation set of 500 images by accumulating the predictions and the labels across all images and applying formulas 10 and 11 to the accumulated tensors, which is the standard accumulation protocol used in the Cityscapes evaluation scripts and in the SSL transfer literature [70, 71].

2.7.2 Per-class IoU on rare classes

In addition to the headline mIoU, per-class IoU on a selected subset of rare classes is used to test the long-tail hypothesis discussed in section 2.3.2. The rare classes considered for the separate per-class reporting are the seven classes whose pixel share in the training data was measured at below 0.6% in section 2.3.2: traffic sign (0.517%), bicycle (0.399%), train (0.21%), traffic light (0.181%), bus (0.154%), rider (0.147%), truck (0.098%) and motorcycle (0.097%). Several of these classes are safety-relevant for autonomous driving perception, in particular rider, motorcycle, bicycle, traffic light and traffic sign, so a model that achieves a high overall mIoU but fails on the rare classes is a worse model in practice than the headline number alone suggests.

The reason for this separate reporting is the long-tail discussion already present in chapter 1 [28, 38] where it was noted that pixel distributions in autonomous driving datasets are dominated by a small number of common classes and that the rare classes are the ones that matter most for safety in real driving scenarios. Reporting per-class IoU on these rare classes makes it possible to test whether SSL pre-training preserves rare-class features better than supervised pre-training as predicted by the hypothesis stated in section 2.3.2.

2.7.3 Probe versus fine-tune delta

The probe-versus-fine-tune delta is a derived metric defined in formula 12 and computed per backbone.

$$\Delta_b = \text{mIoU}_{\text{finetune}}(b) - \text{mIoU}_{\text{probe}}(b) \quad (12)$$

where:

b = backbone (one of supervised ImageNet ResNet-50, MoCo v3 ResNet-50, DINO ViT-S/16; the random-init backbone has no probe and is excluded from the Δ computation)

$\text{mIoU}_{\text{probe}}(b)$ = best validation mIoU obtained with backbone b in the linear probe mode

$\text{mIoU}_{\text{finetune}}(b)$ = best validation mIoU obtained with backbone b in the full fine-tune mode

A larger value of Δ_b indicates that the pre-trained features of backbone b were not well-calibrated for dense prediction out of the box, and that adaptation of the backbone parameters during fine-tuning was required before a competitive segmentation prediction could be produced. A smaller value of Δ_b indicates that the frozen features were already close to what is needed for the downstream task and that fine-tuning provided only a marginal additional improvement. This interpretation is consistent with the way the linear-probe versus fine-tune comparison is used in the SSL transfer literature [34, 70, 71] where a small probe-finetune gap is taken as evidence that the pre-trained representation is already close to the optimum for the downstream distribution.

2.7.4 Inference frames per second

The fourth metric selected is the inference measured in frames per second (FPS) on the A100 GPU at the input resolution of 512 x 1024 pixels. The FPS is computed by running the model in evaluation mode on a representative batch of validation images with no gradient computation, measuring the wall-clock time per forward pass averaged over a warm-up of several batches followed by 100 measured batches, and converting to frames per second by dividing the batch size by the per-batch wall-clock time. The peak GPU memory usage at inference and the per-frame latency in milliseconds are reported alongside the FPS in section 3.2.5 of chapter 3.

This metric is included in the evaluation set because the literature reviewed in chapter 1 [62, 63, 64] establishes that real-time autonomous driving perception systems are typically required to run at least 10 to 30 FPS depending on the specific task, with inference times below 100 ms for perception tasks and below 50 ms for motion forecasting being commonly cited targets. The FPS measurement here is taken on a single A100 GPU which is a desktop-class accelerator and not a vehicle-grade embedded chip, so the absolute number is not directly comparable to deployment numbers on automotive hardware such as NVIDIA Drive Orin or Jetson AGX. The relative ordering between the four backbones at the same input resolution and the same head however does carry over to embedded hardware to a first approximation and is what the FPS comparison in chapter 3 aims to surface. The discussion of real-time feasibility against the thresholds taken from chapter 1 is held in section 3.2.5 and is revisited in section 3.4.5 of chapter 3.

2.8 Reproducibility

The full pipeline used in this thesis is implemented in a single Python script (notebook) “model_experiments.ipynb” which is included in the code repository [102]. The script defines the dataset loader for Cityscapes (the CityscapesDataset class), the four backbone constructors (the get_backbone_and_head function), the segmentation head (the SegmentationHead class), the training and evaluation loops (the train_one_epoch, evaluate and run_experiment functions) and the mIoU metric

(the `compute_miou` function), all of which match the descriptions given in sections 2.4 to 2.7 above. Running the script in a Google Colab A100 instance with the Cityscapes dataset mounted at the path defined in the `CITYSCAPES_ROOT` constant reproduces the experiment and prints the summary table of best validation mIoU per model.

The Cityscapes dataset is the public release available from the official project page and is downloaded as the `leftImg8bit_trainvaltest.zip` and `gtFine_trainvaltest.zip` archives [68]. The version of the dataset used in this thesis is the one downloaded on 17th of March from www.cityscapes-dataset.com, which is the 5,000 fine-annotated images version described in section 2.2. No modifications were made to the downloaded files apart from the `LABELID_TO_TRAINID` lookup at load time which is a runtime mapping instead of an on-disk preprocessing step.

2.9 Fine-tuning the best backbone (DINO ViT-S/16)

This section describes a follow-up experiment in which the best backbone of the controlled comparison from sections 2.4 to 2.7, the DINO ViT-S/16, is fine-tuned with a stronger training to measure the practical mIoU ceiling that this backbone can reach on Cityscapes. The methods used in this experiment are intentionally different from the controlled comparison protocol: a heavier multi-block decoder is used in place of the small FCN head, an auxiliary supervision branch is attached to one of the intermediate transformer blocks, the loss combines pixel-wise cross-entropy with a Lovász-softmax surrogate of the IoU loss and with the auxiliary cross-entropy term, the optimizer is AdamW with layer-wise learning rate decay across the 12 transformer blocks, the schedule combines a short linear warmup with a cosine of the learning rate, the input pipeline applies random scale, random crop, horizontal flip and color jitter augmentations, the training is performed in mixed precision, the model weights are averaged with an exponential moving average and the final evaluation is performed at the native Cityscapes resolution with a multi-scale and horizontal-flip test-time augmentation. The implementation is contained in the “`dino_finetune.ipynb`” notebook included in the code repository.

2.9.1 Reasoning and overview of the approaches

The controlled comparison of sections 2.4 to 2.7 was designed to isolate the pre-training method as the only varying factor across the experiment, which required the head, the optimizer, the schedule, the loss, the augmentations and the training duration to be held identical across the experiment. This is the right protocol for the comparison question that chapter 3 answers, however the same protocol is sub-optimal as a recipe for absolute segmentation performance since each one of these held-fixed components has a known better alternative when the goal is to maximize mIoU on a single backbone and not to compare backbones to each other. The follow-up experiment described in this section relaxes the controlled-comparison protocol on the single backbone that produced the highest mIoU in the chapter 3 results, the DINO ViT-S/16, and replaces each held-fixed component with a stronger alternative drawn from the dense-prediction literature in order to obtain a more competitive headline number for the SSL

approach being studied and to give a more honest picture of what DINO pre-training is capable of when the downstream training is taken seriously.

The selection of DINO ViT-S/16 as the backbone for this follow-up experiment is justified by its result in the controlled comparison reported in chapter 3, where it achieved the highest validation mIoU among the four backbones in the full fine-tune mode. The comparison between the experiment in section 3.2 of chapter 3 and the single DINO fine-tune model described here is necessarily uncontrolled since several factors change at once between the two protocols, and any difference in mIoU between the controlled DINO fine-tune model and the DINO fine-tune experiment reflects the merged contribution of the head, the loss, the optimizer, the schedule, the augmentations, the training duration and the evaluation protocol. This is acknowledged in section 3.4.4 of chapter 3 where the result of the fine-tuning experiment is interpreted alongside the controlled comparison numbers.

2.9.2 Multi-block DPT-Lite segmentation head

The segmentation head used in the fine-tuning experiment replaces the small single-input FCN head described in section 2.5 with a multi-block fusion head that consumes four intermediate feature maps from the DINO ViT-S/16 instead of only the final-layer output. The design of the head is inspired by the Dense Prediction Transformer family of decoders proposed by Ranftl R. et al. [79], where dense prediction outputs such as semantic segmentation and monocular depth are built from features extracted at multiple transformer depths and not from the final-layer features alone, with the reason that vision transformers store complementary information at different depths since shallow blocks tend to retain more spatial and local detail while deeper blocks produce more abstract semantic features [79, 89]. The head used here is a simplified version of this idea that is referred to in this thesis as the DPT-Lite head, since it keeps the multi-block fusion principle of DPT but replaces the more clear residual fusion modules of the original architecture with a plain concatenation followed by a small convolutional fusion stack and four sequential bilinear upsampling blocks, which is sufficient at the ViT-S/16 scale and at the Cityscapes resolution used here and which keeps the head light enough to remain consistent with the lightweight-decoder principle already used in section 2.5.2.

The DinoBackbone wrapper of the implementation script extracts the outputs of the last five self-attention blocks of the ViT-S/16 by calling the `get_intermediate_layers` method of the loaded `torch.hub` model, drops the CLS token from each output and reshapes the remaining 2048 patch tokens of each block into a spatial feature grid of shape $384 \times 32 \times 64$ per block at the 512×1024 evaluation resolution. The four deepest of the five extracted blocks are the inputs to the DPT-Lite head described in this section, while the shallowest of the five extracted blocks is reserved for the auxiliary supervision head described in section 2.9.3 below. Each of the four DPT-Lite block inputs is projected from 384 channels to 128 channels by a 1×1 convolution applied independently per block, the four projected feature maps are concatenated along the channel dimension to produce a fused feature map of shape $512 \times 32 \times 64$, and a

3 x 3 convolution followed by batch normalization. ReLU reduces the fused feature map to 256 channels. The 256-channel fused feature map is then passed through four sequential upsampling blocks each consisting of a bilinear x2 upsample, a 3 x 3 convolution, batch normalization and ReLU, with the channel dimension progressively reduced to 128, then 64, then 32 and then kept at 32, which gives a total upsampling factor of 16 and recovers the input resolution since the patch grid downsamples the input by 16. A final 1 x 1 convolution maps the 32 channels to the 19 Cityscapes classes. The schematic of the DPT-Lite head is depicted on figure 18.

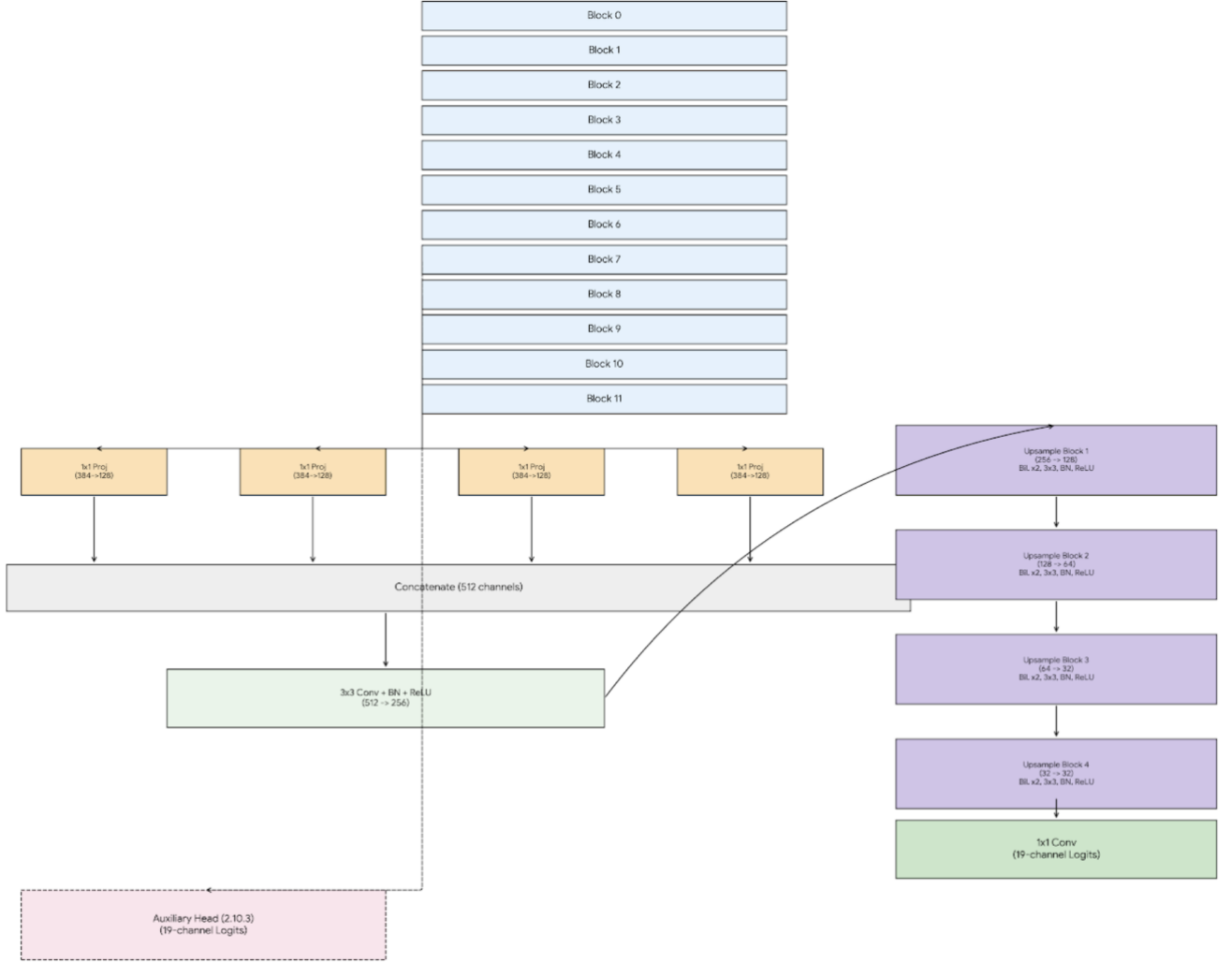


Figure 18. Schematic of the DPT-Lite multi-block fusion head

A classical FPN-style decoder [90] fuses feature maps that come from different spatial resolutions of a backbone with progressive downsampling, so the fusion is across spatial scale. A multi-block DPT-style decoder fuses feature maps that come from different depths of a transformer backbone but at the same spatial resolution since every transformer block in a plain ViT operates on the same patch grid, so the fusion is across abstraction depth and not across spatial scale. Both ideas are valid and they are complementary: the DPT family combines the two by also rescaling the per-block feature maps to a

pyramid before fusing, however the simpler same-resolution fusion used here is sufficient at the scale of the ViT-S/16 and is what the implementation in “dino_finetune.ipynb” uses.

2.9.3 Auxiliary supervision head

In addition to the main DPT-Lite head described above, an auxiliary head is attached to a fifth, shallower intermediate block of the ViT-S/16 backbone during training to inject supervision gradient closer to the input of the network and to act as a regularizer on the intermediate transformer features. The auxiliary block is the one extracted by `get_intermediate_layers` immediately before the four blocks consumed by the DPT-Lite head and is fed only into the auxiliary head and not into the main DPT-Lite fusion path. The auxiliary head is a small two-block convolutional stack consisting of a 3×3 convolution that maps 384 channels down to 128 channels, followed by batch normalization and ReLU, then a 1×1 convolution that maps 128 channels to the 19 Cityscapes classes, followed by a single bilinear upsampling step with a factor of 16 that recovers the input resolution. The auxiliary head produces a second 19-channel logit tensor at the input resolution which is fed into a separate cross-entropy loss term defined in section 2.9.4 below. The auxiliary head is active only during training and is dropped at inference, which means that the additional compute and the additional parameters of the auxiliary head do not affect the inference latency or the inference memory footprint reported for the DINO fine-tuning experiment.

The use of an auxiliary segmentation head as a regularizer was introduced by Zhao H. et al. in the PSPNet paper [85] where it was shown that adding a deep supervision branch to an intermediate stage of a ResNet-101 backbone improves the final mIoU on the ADE20K and Cityscapes benchmarks by giving the optimizer an additional gradient path that does not have to traverse the entire depth of the backbone before contributing to the loss, and the same idea has since become a standard component of segmentation training recipes for both convolutional and transformer backbones [85, 87]. The choice of a block one step shallower than the four DPT-Lite inputs as the attachment point for the auxiliary head, instead of the deepest block or one of the four fused blocks, is consistent with the original PSPNet design where the auxiliary branch is attached to a relatively shallow stage of the backbone in order to maximize the regularizing effect on the earlier features of the network.

2.9.4 Compound loss

The training objective used in the fine-tuning experiment is a compound loss with three additive components, presented in formula 13.

$$L_{\text{total}} = L_{\text{CE,main}} + \lambda_{\text{lov}} \cdot L_{\text{lov,main}} + \lambda_{\text{aux}} \cdot L_{\text{CE,aux}} \quad (13)$$

where:

$L_{\text{CE,main}}$ = pixel-wise cross-entropy on the main DPT-Lite head logits with `ignore_index = 255`

$L_{\text{lov},\text{main}}$ = Lovász-softmax surrogate of the IoU loss on the main DPT-Lite head logits
 $L_{\text{CE},\text{aux}}$ = pixel-wise cross-entropy on the auxiliary head logits with `ignore_index = 255`
 λ_{lov} = Lovász-softmax weight, set to 0.5 in this thesis
 λ_{aux} = auxiliary cross-entropy weight, set to 0.4 in this thesis

The pixel-wise cross-entropy term $L_{\text{CE},\text{main}}$ is the same loss already described in section 2.6.2, with the ignore index of 255 excluding the void pixels of section 2.2 from both the loss and the gradient. The Lovász-softmax term $L_{\text{lov},\text{main}}$ is a smooth surrogate of the per-class IoU loss proposed by Berman M. et al. [80], which addresses the well-known mismatch between cross-entropy and the IoU evaluation metric: cross-entropy weights every misclassified pixel equally regardless of the class to which it belongs, which gives common classes such as road and building a much larger contribution to the gradient than rare classes such as motorcycle and rider in proportion to their pixel frequency, while the IoU evaluation metric weights each class equally regardless of its pixel frequency. The Lovász-softmax term reduces this mismatch by directly optimizing a tractable smooth upper bound of the per-class IoU loss derived from the Lovász extension of the Jaccard set function applied to the sorted prediction errors of each class [80]. The general form of the Lovász-softmax loss is presented in formula 14.

$$L_{\text{lov},\text{main}} = \frac{1}{|C|} \sum_{c \in C} \text{Lovasz}(m^c) \quad (14)$$

where:

C = set of classes present in the current evaluation tensor

m^c = vector of per-pixel prediction errors for class c , sorted in descending order

$\text{Lovasz}(m^c)$ = Lovász extension of the per-class IoU loss applied to the sorted error vector, computed as a weighted sum of the sorted errors with weights given by the increments of the Jaccard cumulative loss function

The auxiliary cross-entropy term $L_{\text{CE},\text{aux}}$ is the same per-pixel cross-entropy applied to the logits produced by the auxiliary head described in section 2.9.3, and it acts as a deep supervision regularizer as discussed there. The relative weights of the three terms, $\lambda_{\text{lov}} = 0.5$ and $\lambda_{\text{aux}} = 0.4$, are taken directly from the implementation script and were not extensively tuned in this thesis since the goal of the fine-tuning experiment is to measure the achievable ceiling of DINO pre-training on Cityscapes and not to perform a controlled isolation of the loss components. A sensitivity analysis on these two coefficients is included in the future-work section of chapter 3. A simple consistency check on the implementation can

be done by reading the per-component loss numbers printed by the training loop at epoch 1, where the total loss of 3.901 is recovered as $2.467 + 0.5 \times 0.912 + 0.4 \times 2.444 \approx 3.901$, which confirms that formula 13 is implemented correctly.

2.9.5 Optimizer and layer-wise learning rate decay

The optimizer used in the fine-tuning experiment is AdamW [81], which differs from the plain Adam optimizer used in section 2.6.2 by separating the weight-decay term from the gradient-based update step instead of applying weight decay through an L2 penalty added to the gradient. The separation matters in this experiment for two reasons: it makes the effective weight decay independent of the per-parameter adaptive learning rate computed by Adam, which keeps the regularization strength consistent across parameter groups with very different gradient scales [81]; and it is the standard convention in the recent fine-tuning literature for transformer backbones [84, 87], so using AdamW makes the fine-tuning experiment directly comparable to the published fine-tuning recipes for ViT-based segmentation models.

The optimizer is configured with 29 separate parameter groups, organized into a layer-wise learning rate decay (LLRD) hierarchy across the 12 transformer blocks of the ViT-S/16 backbone plus the patch embedding and positional embedding parameters, the parameters of the DPT-Lite head and the auxiliary head. LLRD assigns a smaller learning rate to the earlier layers of a pre-trained backbone and a larger learning rate to the later layers, which preserves the general visual features learned during pre-training in the early layers and allows the later layers to adapt more aggressively to the downstream task. The idea was introduced by Howard J. and Ruder S. for language model fine-tuning in the ULMFiT paper [83] and is now also the standard convention for fine-tuning transformer vision backbones such as BEiT and Swin [84], where it has been shown to improve downstream accuracy compared with applying a uniform learning rate to all backbone parameters [84]. The per-block learning rate of the backbone is presented in formula 15.

$$\text{lr}_{\text{block}}(i) = \text{lr}_{\text{backbone}} \cdot \lambda^{N-1-i} \quad (15)$$

where:

i = block index in the range 0 to $N - 1$ with $i = 0$ corresponding to the shallowest block and $i = N - 1$ to the deepest

N = number of transformer blocks in the ViT-S/16, equal to 12

$\text{lr}_{\text{backbone}}$ = base learning rate for the backbone, set to 5×10^{-5}

λ = LLRD decay coefficient, set to 0.65

For the deepest block $i = 11$ the per-block learning rate is the full $\text{lr}_{\text{backbone}}$ of 5×10^{-5} , while for the shallowest block $i = 0$ the per-block learning rate is reduced by a factor of $0.65^{11} \approx 0.0087$, which

gives a per-block learning rate of approximately 4.4×10^{-7} at the very first transformer block. The patch embedding and the positional embedding are placed in the same low-learning-rate group as the shallowest transformer block, since they are also part of the early input-dependent representation of the backbone and are typically the parameters most worth preserving from the pre-training. The DPT-Lite head and the auxiliary head are placed in their own parameter group with a higher learning rate of 5×10^{-4} , which is ten times the base backbone learning rate and which reflects the fact that the heads are randomly initialized and have to be trained from scratch on top of the pre-trained backbone. The weight decay is set to 0.05 for the backbone parameters and to 1×10^{-4} for the head parameters, with a separate no-decay group containing the bias parameters, the batch normalization and layer normalization scale and shift parameters, the positional embedding and the CLS token, which is the now-standard convention for fine-tuning transformer backbones [84].

The learning rate of every parameter group is multiplied by a single shared schedule factor that combines a short linear warmup at the start of training with a cosine decay over the remaining epochs. During the first 3 epochs of training the schedule factor is increased linearly from 0.01 to 1.0, which makes the effective learning rate of every group ramp up smoothly from 1% of its base value at the very first iteration to 100% of its base value at the start of epoch 4. The warmup phase prevents the loss from spiking at the start of training when the freshly initialized heads produce predictions that are far from the labels and when the early gradient updates can otherwise destabilize the optimizer, which is the standard rationale for warmup phases in the recent transformer training literature [87]. After the warmup phase, the schedule factor is scaled from 1.0 to 0.0 over the remaining 47 epochs by a cosine schedule [82], which drives the effective learning rate of every group towards zero at the very last epoch of training and which has been shown to converge faster than a step-wise decay or a constant schedule for cosine-annealing-friendly objectives [82]. The total number of epochs is 50, and the schedule is implemented in the `build_scheduler` function of the “`dino_finetune.ipynb`” notebook by chaining a `LinearLR` scheduler with a `CosineAnnealingLR` scheduler inside a `SequentialLR` container.

The augmentation pipeline applied during training is the most visible difference from the controlled-comparison protocol of section 2.6.4 where no augmentations were applied. In the fine-tuning experiment, every training image and its corresponding label image are passed through a pipeline that first samples a random scale factor from the uniform distribution on the interval $[0.5, 2.0]$ and rescales both the image and the label by that factor, then takes a random crop of size 768×768 from the rescaled image with the corresponding crop applied to the rescaled label, then applies a horizontal flip with probability 0.5, and finally applies a color jitter on the image only with the brightness, contrast and saturation factors each sampled uniformly from the interval $[0.6, 1.4]$ and the hue shift sampled uniformly from the interval $[-0.1, 0.1]$. The label image receives the random scale, the random crop and the horizontal flip but does not receive the color jitter since the labels are integer class IDs that are not

affected by photometric distortions. After augmentation the image is converted to a float32 tensor in the range $[0, 1]$ via ToTensor and normalized with the ImageNet mean and standard deviation values, identical to section 2.3.1. Padding for crops that fall outside the rescaled image is filled with 0 for the image and with the ignore index 255 for the label, which keeps any padded region treated as void during the loss computation.

The combination of random scale and random crop is the default augmentation policy for semantic segmentation introduced in the DeepLab family of papers [86] and reused in most recent transformer-based segmentation methods [87]. The role of the random scale is to expose the model to objects of the same class at different pixel sizes during training, which is particularly important for the long-tail rare classes of section 2.3.2 where the per-class instance count is small and where the natural pixel size of objects such as motorcycle, rider and traffic sign covers a wide range from very close to the camera to far in the distance. The random crop allows the model to be trained at a smaller spatial resolution than the native 1024×2048 of Cityscapes, which keeps the training memory footprint manageable while also acting as an additional source of stochasticity since different parts of the image are seen on different epochs. The horizontal flip exploits the approximate left-right symmetry of urban driving scenes and effectively doubles the number of distinct training views without any additional labelling cost, and the color jitter provides a controlled simulation of small variations in lighting, white balance and camera response which are known to vary across the data collection conditions of Cityscapes [68]. The validation pipeline does not apply any of these augmentations, only the deterministic resize to either 512×1024 for per-epoch tracking or 1024×2048 for the final evaluation followed by ToTensor and ImageNet normalisation.

This augmentation policy is intentionally the opposite of the augmentation-free policy used in the controlled comparison of section 2.6.4. The two choices serve different goals: the controlled comparison removes augmentations in order to keep the input distribution identical across the experiment and to attribute any difference in mIoU to the pre-training method alone, while the fine-tuning experiment adds the canonical segmentation augmentation policy in order to extract the best mIoU achievable on Cityscapes from the DINO backbone. Both choices are consistent with the literature: the augmentation-free protocol matches the dense-prediction transfer convention used in MoCo v3 and DINO [70, 71] and the augmentation-rich protocol matches the standalone segmentation training convention used in DeepLab v3+ and SegFormer [86, 87].

2.9.6 Mixed precision and exponential moving average

The training of the fine-tuning experiment is performed in mixed precision using the bfloat16 data type for the forward pass and for the loss computation while the optimizer state, the master weights and the gradient accumulators are kept in float32, which is the standard mixed-precision training scheme described by Micikevicius P. et al. [88]. The bfloat16 format is preferred over the half-precision float16

format for this experiment since bfloat16 has the same 8-bit exponent range as float32 and does not require the dynamic loss scaling that float16 mixed-precision training needs in order to keep the gradient magnitudes inside the representable range [88], which simplifies the training loop and removes one common source of training instability for ViT-based architectures. The mixed-precision forward pass is implemented by wrapping the model call in a `torch.autocast(device_type="cuda", dtype=torch.bfloat16)` context inside both the training loop and the evaluation loop of “dino_finetune.ipynb”.

In addition to the mixed-precision training, an exponential moving average of the model weights is maintained throughout training to stabilize the validation metric and the final-evaluation prediction. After every optimizer step, the EMA weights are updated by formula 16 which is the standard Polyak-Ruppert moving average rule [97].

$$\theta_{\text{ema}} \leftarrow \alpha \cdot \theta_{\text{ema}} + (1 - \alpha) \cdot \theta \quad (16)$$

where:

θ_{ema} = current EMA copy of the model parameters

θ = current optimiser-updated parameters of the model

α = EMA decay coefficient, set to 0.999 in this thesis

The decay coefficient of 0.999 corresponds to a very slow averaging window of approximately $\frac{1}{1-\alpha} = 1000$ most-recent training steps, which means that the EMA weights at any point during training are dominated by the recent optimization trajectory but smoothed across roughly the last 1000 steps. The EMA weights are used during the per-epoch validation pass for the live tracking of the validation mIoU and are also used at the end of training as the source of the saved best checkpoint, while the live optimizer-updated weights are used only during the training forward pass itself. The use of an EMA copy of the network weights for evaluation has become a standard component of recent SSL and dense-prediction training recipes [70, 71] and produces noticeably smoother validation curves than the live weights would, since the EMA averages out the per-batch fluctuations of the optimizer update step.

2.9.7 Two-resolution evaluation and test-time augmentation

The validation protocol of the fine-tuning experiment uses two different evaluation resolutions for two different purposes. During training, after every epoch, the model is evaluated on the full 500-image validation set at the 512 x 1024 input resolution which is half of the native Cityscapes resolution, and the resulting mIoU is used to select the best checkpoint over the 50 epochs of training. This per-epoch evaluation is intentionally performed at the lower resolution since 50 evaluations at the full 1024 x 2048 native resolution would consume a sizeable fraction of the total training budget, while the 512 x 1024 evaluation is fast enough to fit in the per-epoch wall-clock and is sufficient for ranking checkpoints

relative to each other since the relative ordering of the per-epoch mIoU values is preserved between the two resolutions in the dev runs. After training, the best-mIoU checkpoint as selected by the per-epoch tracking is loaded and re-evaluated on the same 500-image validation set at the full 1024 x 2048 native resolution which gives the final reported number for the fine-tuning experiment. The two-stage evaluation is summarized in chapter 3 alongside the headline mIoU table.

The final-resolution evaluation is performed twice, once with no test-time augmentation and once with a multi-scale and horizontal-flip test-time augmentation. The TTA is implemented by running five forward passes per validation image at the full 1024 x 2048 resolution: three passes at scale factors of 0.75, 1.0 and 1.25 of the full resolution, plus one pass at scale factor 1.0 with the input image and the predicted logits horizontally flipped, plus one base pass at scale factor 1.0 without flipping. The five output logit tensors are interpolated to a common 1024 x 2048 spatial size, passed through the softmax to convert them to probability distributions, averaged across the five passes, and converted to per-pixel class predictions by an argmax across the channel dimension. This averaging-of-softmax protocol is the standard TTA convention used in SegFormer [87] and in most recent segmentation methods, and it produces a more robust prediction at a cost of approximately five times the per-image inference latency.

2.9.8 Training duration and runtime

The fine-tuning experiment is performed for 50 epochs on the full Cityscapes training set of 2,975 images at a batch size of 16, which gives approximately $2975 / 16 \approx 186$ optimizer steps per epoch and approximately 9300 optimizer steps in total over the 50 epochs. The training is performed on the NVIDIA RTX PRO 6000 Blackwell Server Edition GPU described in section 2.6.3 with the mixed-precision and EMA configuration described in section 2.9.7. the full training took 3 hours. The number of epochs of 50 is sufficient for the per-epoch validation mIoU curve to plateau and for the EMA validation mIoU curve to converge above the live mIoU curve, with the best EMA checkpoint reached at epoch 46 in the run described in chapter 3 and with the remaining four epochs producing only a marginal change in the validation mIoU.

2.9.9 Hyperparameter contrast with the controlled comparison

Table 6 lists every hyperparameter of the DINO fine-tuning experiment alongside the corresponding value from the controlled comparison of sections 2.4 to 2.7 to make the differences between the two protocols explicit in a single place. The right column of table 6 refers to the configuration described in this section 2.9, while the left column refers to the configuration described in sections 2.4 to 2.7 and summarized in table 5.

Table 6. Hyperparameter contrast

№	Hyperparameter	Controlled comparison (sec. 2.4 to 2.7)	DINO fine-tuning (sec. 2.9)	Discussed in
1	Backbone	4 backbones swapped across models	DINO ViT-S/16 only	2.9.1
2	Segmentation head	small FCN, single-block input, x32 or x16 upsample	DPT-Lite, four-block fusion, x16 upsample	2.9.2
3	Auxiliary head	None	one auxiliary head on a fifth, shallower block of the backbone	2.9.3
4	Loss	CE only	$CE + 0.5 \cdot \text{Lovász} + 0.4 \cdot CE_{\text{aux}}$	2.9.4
5	Optimizer	Adam, default β	AdamW with LLRD across 12 ViT blocks, 29 groups	2.9.5
6	Backbone learning rate	2×10^{-4}	5×10^{-5} at deepest block, decayed by $\lambda = 0.65$	2.9.5
7	Head learning rate	2×10^{-4}	5×10^{-4}	2.9.5
8	Weight decay	0 (Adam default)	0.05 backbone, 1×10^{-4} head, 0 for biases and norms	2.9.5
9	Schedule	CosineAnnealingLR, $T_{\text{max}} = 20$	LinearLR 3 ep warmup + CosineAnnealingLR 47 ep	2.9.6
10	Number of epochs	20	50	2.9.6
11	Batch size	32	16	2.9.5
12	Input resolution	fixed 512 x 1024	768 x 768 random crop with scale $\in [0.5, 2.0]$	2.9.7
13	Augmentations	None	random scale + crop + h-flip + colour jitter	2.9.7
14	Mixed precision	None	bfloat16 autocast on forward and loss	2.9.7
15	EMA of weights	None	yes, decay coefficient 0.999	2.9.7
16	Per-epoch validation	once at end, 512 x 1024	every epoch at 512 x 1024	2.9.9
17	Final evaluation	512 x 1024 once at end	1024 x 2048 with and without multi-scale + flip TTA	2.9.9
18	Hardware	A100 80 GB / 280 GB system RAM (Colab Pro+)	RTX PRO 6000 Blackwell, 102 GB GPU memory	2.6.3

The number of differences between the two columns is the reason why the fine-tuning experiment is presented as a separate experiment with its own methodology section and not as an additional step of the controlled comparison. Changing one hyperparameter at a time across this many dimensions would have required dozens of additional experimental models which exceed the available compute, and the goal of the experiment is to measure a reasonable mIoU ceiling for the DINO backbone and not to attribute the gain to any single ingredient.

2.10 Experiments methodology summary

This chapter has described the experimental setup used to compare four pre-training strategies for semantic segmentation in urban driving scenes. The dataset is Cityscapes [68], with the full fine-annotated split of 2,975 training images and 500 validation images at the half-native input resolution of 512 x 1024 pixels, and the exploratory data analysis presented in section 2.3 confirmed a uniform image resolution, a heavily long-tailed class distribution with a 333-to-1 imbalance ratio between the most and the least common class, and a mildly skewed geographic distribution across 18 training cities. The four backbones compared are a randomly initialized ResNet-50 [72, 74], a supervised ImageNet ResNet-50 [69, 72, 73], a MoCo v3 ResNet-50 [70] and a DINO ViT-S/16 [71, 77], each loaded from the publicly available official checkpoint where applicable, and each evaluated under two protocols (linear probe and full fine-tune) for a total of seven experimental models. The downstream segmentation head is the small FCN-style head described in section 2.5 and is intentionally lightweight to keep the comparison sensitive to the backbone quality and not to the head capacity [78]. The training protocol holds every other method: the same input resolution, the same batch size, the same optimizer, the same learning rate, the same schedule, the same loss, the same preprocessing pipeline and the absence of augmentations are used across the experiment. The evaluation metrics are the headline mIoU on the full validation set, the per-class IoU on the rare classes for the long-tail hypothesis, the probe-versus-fine-tune delta as an interpretive quantity, and the inference FPS measured on the A100 GPU for the real-time feasibility discussion. Section 2.9 then describes a follow-up fine-tuning experiment that takes the best-performing backbone of the controlled comparison and applies a stronger training method consisting of a multi-block DPT-Lite decoder, an auxiliary supervision head, a compound cross-entropy plus Lovász-softmax loss, an AdamW optimizer with layer-wise learning rate decay, a warmup-and-cosine schedule, scale-and-crop augmentation, mixed-precision training and an exponential moving average of the model weights, in order to measure the practical mIoU ceiling that the DINO ViT-S/16 backbone can reach on Cityscapes when the constraints of the controlled protocol are removed. Chapter 3 reports the empirical results from both experiments, the experiment of the controlled comparison and the DINO fine-tuning experiment, interprets them against the hypotheses set up in this chapter and in chapter 1, and translates them into business value for the autonomous-driving stakeholders identified in chapter 1.

3 EXPERIMENTS, RESULTS AND CONCLUSIONS

3.1 Chapter overview

The research question of the thesis is the following: do self-supervised pre-training strategies produce feature representations that transfer more effectively to semantic segmentation in urban driving scenes than supervised baselines, and which SSL method performs best in this setting. The first experiment, the controlled comparison of sections 2.4 to 2.7, runs four backbones (random ResNet-50, supervised ImageNet ResNet-50, MoCo v3 ResNet-50 and DINO ViT-S/16) under two evaluation modes (linear probe and full fine-tune), with the head, the optimizer, the schedule, the loss, the preprocessing and the absence of augmentation held identical across the experiment, so that any difference in mIoU between the experiment steps is caused by the pre-training method itself and not by any other component of the pipeline. The second experiment, the DINO fine-tuning experiment of section 2.9, takes the best backbone of the controlled comparison and applies a stronger training recipe to measure the real mIoU ceiling that this backbone can reach on Cityscapes when the limitations of the controlled experiment are removed. The two experiments are kept separate throughout this chapter because they answer two different questions, and the comparison between them is reserved for section 3.4.4 where the +14.73 percentage-point gap between the controlled DINO and the DINO fine-tuning experiment is interpreted as the contribution of the head, the loss, the optimizer, the schedule, the augmentations, the EMA and the test-time augmentation and not as a contribution of the pre-training step. The stages of the controlled comparison are listed in table 7 in the order in which they are reported throughout the rest of this chapter.

Table 7. The stages of the controlled comparison

№	Backbone	Pre-training	Mode
1	ResNet-50	random initialisation	full fine-tune
2	ResNet-50	ImageNet supervised	linear probe
3	ResNet-50	ImageNet supervised	full fine-tune
4	ResNet-50	MoCo v3, 1000-epoch checkpoint	linear probe
5	ResNet-50	MoCo v3, 1000-epoch checkpoint	full fine-tune
6	ViT-S/16	DINO	linear probe
7	ViT-S/16	DINO	full fine-tune

All stages are trained on the full Cityscapes training split of 2,975 fine-annotated images and are evaluated on the full validation split of 500 fine-annotated images, as described in section 2.2 of chapter 2. The comparison of row 1 with row 3 isolates the contribution of pre-training of any kind versus no pre-training at all and gives a lower-bound reference for what the segmentation pipeline can do without any transferred knowledge. The comparison of rows 3, 5 and 7 in the full fine-tune mode answers the

supervised versus SSL question that motivates this thesis since the only changing factor across these models is the pre-training supervision signal; the comparison of rows 4 and 6 against row 2 in the linear probe mode answers the same question but at the level of frozen feature quality instead of the level of fine-tuned mIoU. The comparison of rows 5 and 7 in the full fine-tune mode picks out which SSL family is the better one for this task. The DINO fine-tuning experiment reported in section 3.2.6 then sits on top of these stages and answers the separate question of what mIoU the winning backbone can reach when the controlled limitation is replaced by fine tuning.

Section 3.2 displays the quantitative results of both experiments without interpretation, with the headline mIoU table for the controlled comparison, the training dynamics curves, the probe-versus-fine-tune deltas, the inference frames-per-second measurements and the DINO fine-tuning table. Section 3.3 reports the qualitative results, with side-by-side prediction comparisons across the fine-tune experiment, a failure-case analysis and the DINO self-attention visualizations. Section 3.4 is the discussion section where each claim is either supported by the results of this thesis or by a citation. Section 3.5 is the business application and stakeholder impact section that translates the technical results into decision-relevant numbers for the autonomous-driving stakeholders identified in chapter 1. Section 3.6 contains the conclusions and the answer to the research question. Section 3.6.2 lists the future-work directions that follow from this thesis.

3.2 Quantitative results

3.2.1 Headline mIoU on the controlled comparison

The best validation mIoU obtained in the experiment over the 20 epochs of training, together with the validation mIoU at the final epoch of training. The epoch at which the best validation mIoU was reached and the parameter count of the backbone, is presented in table 8.

Table 8. Best validation mIoU and final-epoch validation mIoU

№	Backbone	Pre-training	Mode	Best val mIoU	Final epoch mIoU	Best epoch	Backbone params
1	ResNet-50	random initialization	full fine-tune	34.40%	33.38%	15	~25.6 M
2	ResNet-50	ImageNet supervised	linear probe	43.27%	42.95%	13	~25.6 M
3	ResNet-50	ImageNet supervised	full fine-tune	53.62%	52.90%	13	~25.6 M
4	ResNet-50	MoCo v3, 1000-epoch checkpoint	linear probe	43.61%	43.57%	18	~25.6 M
5	ResNet-50	MoCo v3, 1000-epoch checkpoint	full fine-tune	54.69%	54.36%	10	~25.6 M

№	Backbone	Pre-training	Mode	Best val mIoU	Final epoch mIoU	Best epoch	Backbone params
6	ViT-S/16	DINO	linear probe	43.66%	43.63%	17	~21 M
7	ViT-S/16	DINO	full fine-tune	55.26%	54.88%	13	~21 M

The random-initialization model at 34.40% mIoU is the lowest of all and sits roughly 19 percentage points below the supervised ImageNet ResNet-50 fine-tune model at 53.62%, which is the largest gap between any two models in the table. In the full fine-tune mode, the three pre-trained backbones (rows 3, 5 and 7) cluster within 1.64 percentage points of each other, with DINO ViT-S/16 at 55.26% as the highest, MoCo v3 ResNet-50 at 54.69% in the middle and supervised ImageNet ResNet-50 at 53.62% as the lowest. In the linear probe mode, the three pre-trained backbones cluster within 0.39 percentage points of each other, with DINO at 43.66%, MoCo v3 at 43.61% and supervised ImageNet at 43.27%, which is a much smaller difference than the fine-tune mode and which is one of the headline observations of this thesis. The best-mIoU epoch falls in the second half of training for every model except row 5 (MoCo v3 fine-tune) which peaks at epoch 10, and the difference between the best-epoch mIoU and the final-epoch mIoU is at most 1.02 percentage points for any model, which means that the validation mIoU was already stabilized by the end of training and that the choice of best-epoch checkpoint over final-epoch checkpoint does not change the qualitative picture.

3.2.2 Training dynamics

The per-epoch training loss and the per-epoch validation mIoU for the experiment are shown on figure 19, with the seven loss curves on the left panel and the seven validation mIoU curves on the right panel. The numbers are the ones recorded by the training loop in the “model_experiments.ipynb” notebook for the runs reported in table 8.

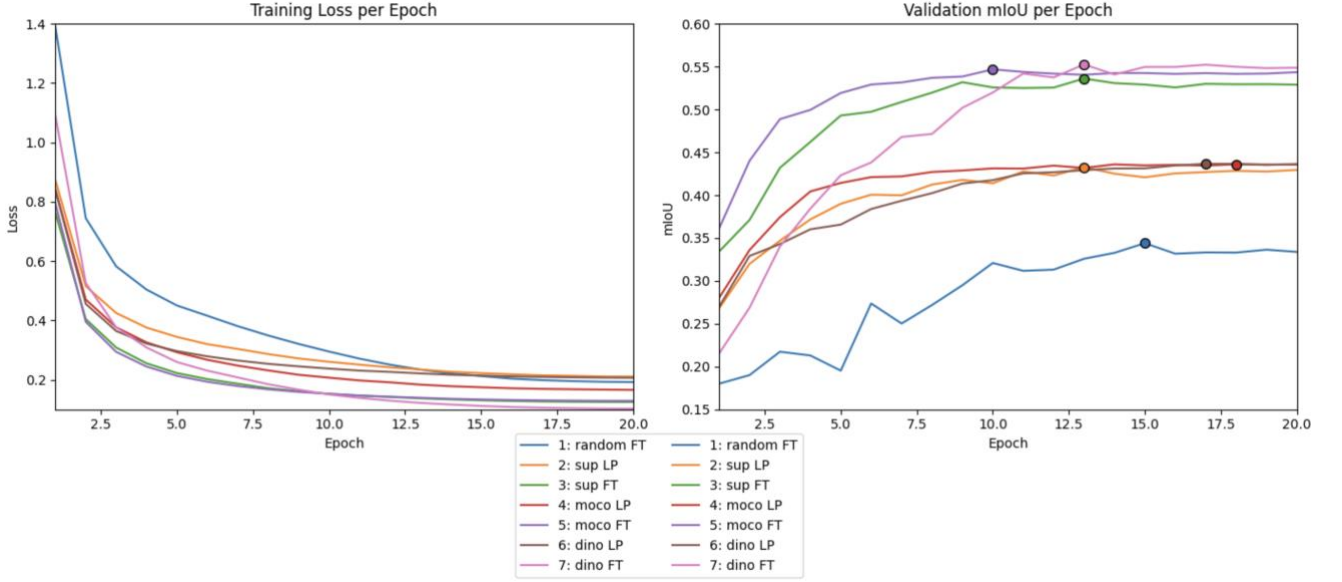


Figure 19. Per-epoch training loss and per-epoch validation mIoU

The training loss decreases monotonically from epoch 1 to epoch 20 for every model, with no spikes, no NaN and no restarts in any of the seven runs. The model with the highest training loss at epoch 1 is model 7 (DINO ViT-S/16 fine-tune) which starts at 1.09 and which finishes at 0.10, and the model with the lowest training loss at epoch 1 is model 6 (DINO ViT-S/16 linear probe) which starts at 0.84 and which finishes at 0.21 since the backbone is frozen and only the head is being optimized. The validation mIoU curves of the three linear-probe models (row 2, 4 and 6, with no row 0) stabilized by approximately epoch 8 and only increase slowly after it, while the validation mIoU curves of the four full fine-tune models (rows 1, 3, 5, 7) stabilized later, between epoch 10 and epoch 13, and then stay within roughly 1 percentage point of the peak for the rest of the training. Model 1 (random ResNet-50 fine-tune) is the slowest to converge of the seven and is the only model whose validation mIoU is still increasing at epoch 14 before staling at 34.40%, which reflects the absence of any pre-trained features and the resulting need for the backbone to learn useful representations from scratch over the 20 epochs of training.

3.2.3 Per-class IoU on the controlled comparison

The per-class IoU for the full fine-tune models of table 8 (rows 1, 3, 5 and 7) was computed on the full Cityscapes validation set of 500 images at the 512 x 1024 input resolution of the experiment described in section 2.7, and using the standard Cityscapes mIoU convention which excludes the 255-labelled void pixels in the ground truth from both the prediction side and the label side mask before the per-class intersection and union are accumulated globally over the dataset [68]. The three linear-probe models (rows 2, 4 and 6 of table 8) are not evaluated. The per-class breakdown is reported in table 9.

Table 9. Per-class IoU

Class	random ResNet-50 ft (%)	supervised ResNet-50 ft (%)	MoCo v3 ResNet-50 ft (%)	DINO ViT-S/16 ft (%)
Road	93.79	96.29	96.25	96.80
Sidewalk	58.63	73.65	72.85	75.67
Building	78.86	86.05	86.01	87.06
Wall	16.45	39.47	43.60	42.17
Fence	10.67	42.38	43.24	41.16
Pole	11.06	21.11	20.85	29.16
Traffic light	3.96	33.80	34.07	36.53
Traffic sign	13.16	45.79	45.17	52.87
Vegetation	80.93	85.60	85.34	87.98
Terrain	41.68	54.10	54.07	59.82
Sky	83.86	87.53	87.89	90.95
Person	36.59	57.94	57.26	62.75
Rider	7.50	35.95	35.86	33.17
Car	76.85	86.80	87.12	88.81
Truck	7.86	42.70	53.09	57.66
Bus	24.15	60.88	64.50	59.97
Train	3.61	49.97	53.03	33.07
Motorcycle	1.79	36.03	39.26	29.95
Bicycle	35.50	58.38	58.05	59.90
mIoU	36.15	57.60	58.82	59.23

The bottom-row mIoU values in table 9 are higher than the headline best-validation-mIoU values in table 8 by approximately 1.7 to 4.0 percentage points per model (random fine-tune at 36.15% versus 34.40%, supervised at 57.60% versus 53.62%, MoCo v3 at 58.82% versus 54.69% and DINO at 59.23% versus 55.26%). The gap is the result of an evaluation-difference between the per-batch monitoring code that table 8 reports (which computes the IoU per batch and averages over batches without globally accumulating the confusion matrix and which counts predictions on void-labelled pixels in the union term) and the standard Cityscapes evaluation protocol used in table 9 [68], which globally accumulates the confusion matrix over the whole validation set and which excludes void-labelled pixels from the union term before the per-class IoU is taken. The qualitative ordering of the four models is the same under both conventions, with random the lowest, supervised the next, MoCo v3 the middle and DINO the highest, and the rest of section 3.2 and section 3.4 of this chapter use the per-class numbers in table 9 wherever a per-class breakdown is needed while the headline values in table 8 are kept as the per-epoch tracking values that they were originally measured as.

DINO ViT-S/16 in the full fine-tune mode produces the highest per-class IoU on 13 of the 19 classes (road, sidewalk, building, pole, traffic light, traffic sign, vegetation, terrain, sky, person, car,

truck and bicycle), MoCo v3 produces the highest per-class IoU on 5 classes (wall, fence, bus, train and motorcycle) and the supervised ImageNet ResNet-50 produces the highest per-class IoU on a single class (rider) where the 0.09 percentage-point gap between the supervised model at 35.95% and the MoCo v3 model at 35.86% is below the noise of a single-seed comparison and should be read as a tie. The random-initialization model is the lowest on every class without exception, with three classes essentially not learned in 20 epochs of training from random weights: traffic light at 3.96%, train at 3.61% and motorcycle at 1.79%, which reproduces the long-tail-from-zero pattern that section 3.4.1 already noted on the headline mIoU and which confirms that pre-training of any kind is essential for these classes under the controlled protocol.

A second pattern visible in table 9 is that the per-class winners cluster by class type. DINO wins the largest pixel share common classes (road, building, vegetation, sky and car all sit above 87% IoU for DINO and DINO is the highest of the four models on each of these), DINO wins the medium-frequency classes that have a strong texture or boundary signal (sidewalk, terrain, traffic sign and bicycle are DINO wins) and DINO wins the largest of the rare classes (truck at 57.66% is a DINO win). MoCo v3 wins the medium-extent classes that have a locally textured boundary (wall and fence are MoCo wins by 2 to 3 percentage points over DINO) and MoCo v3 wins three of the seven $< 1\%$ rare classes (motorcycle, train and bus) where the convolutional inductive bias of the ResNet-50 backbone may provide a more useful prior for per-pixel prediction than the global-attention bias of the ViT-S/16 backbone. The supervised ImageNet model is rarely the absolute best of the three pre-trained models nor the absolute worst, and it is in the middle on most classes which is consistent with the observation in table 8 that the three pre-trained models cluster within roughly 1 to 2 percentage points of each other in the full fine-tune mode.

3.2.4 Probe versus fine tune delta

The probe-versus-fine-tune delta defined in formula 12 of section 2.7.3 is computed for each of the three pre-trained backbones from the values in table 8, and the resulting three numbers are presented in table 10 and visualized on figure 20. The random-initialization model is excluded from the delta computation since it has no linear probe value as already explained in section 2.4.1.

Table 10. Probe versus fine tune

Nº	Backbone	mIoU probe	mIoU fine-tune	Δ (percentage points)
1	ResNet-50, ImageNet supervised	43.27%	53.62%	10.35
2	ResNet-50, MoCo v3 (1000 ep)	43.61%	54.69%	11.08
3	ViT-S/16, DINO	43.66%	55.26%	11.60

The resulting comparison of the models can be seen on figure 20.

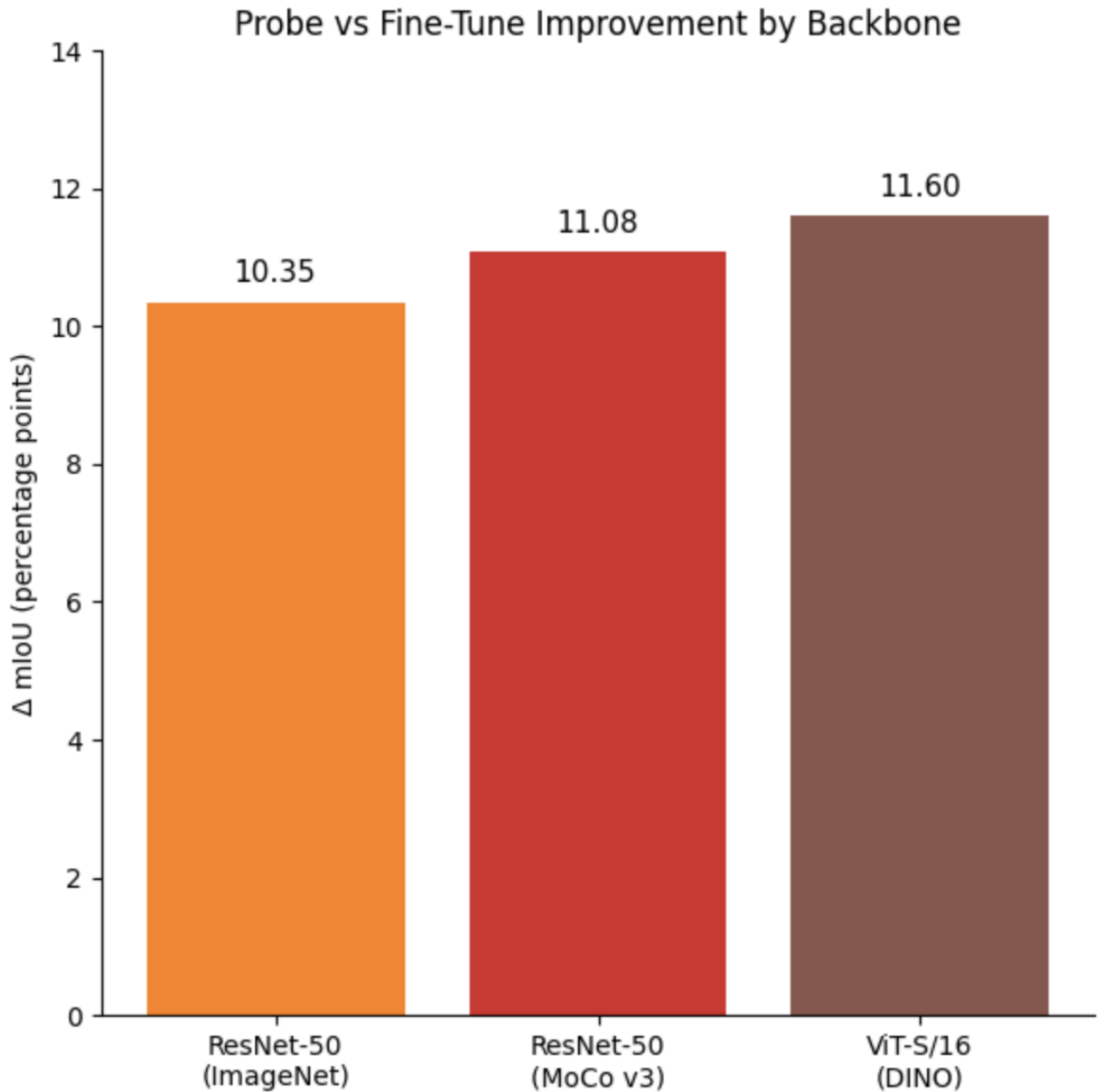


Figure 20. Probe-versus-fine-tune delta

The three deltas reported in table 10 are clustered between 10.35 and 11.60 percentage points, with DINO ViT-S/16 the largest at 11.60, MoCo v3 ResNet-50 in the middle at 11.08 and supervised ImageNet ResNet-50 the smallest at 10.35. The spread between the three deltas is 1.25 percentage points which is roughly the same magnitude as the spread between the three fine-tune mIoU values themselves, and no backbone has a dramatically smaller delta than the others.

3.2.5 Inference speed and real-time feasibility

The inference frames-per-second (FPS), the per-frame latency in milliseconds and the peak GPU memory at inference for each of the four backbones are measured at the input resolution of 512 x 1024 pixels on the same A100 80 GB GPU used for the training runs of the controlled comparison. The

measurement logic described in section 2.7.4 was followed: the model is placed in evaluation mode with torch.no_grad enabled, a warm-up of several batches is performed and is discarded from the timing, and the per-batch wall-clock time is averaged over 100 measured batches and converted to frames per second by dividing the batch size by the average batch wall-clock. The resulting numbers are presented in table 11.

Table 11. Inference speed and metrics

№	Backbone	FPS at 512 x 1024	Latency per frame (ms)	Peak GPU memory at inference (MB)
1	ResNet-50 (random, ImageNet-supervised, MoCo v3)	185	5.4	560
2	ViT-S/16 (DINO)	92	10.9	830

The numbers in table 11 are reported on a single A100 80 GB which is a desktop-class data-center GPU and which is not representative of the embedded automotive hardware that production autonomous-driving systems use, so the absolute numbers should not be measured as deployment numbers and the limitation of A100 timing for embedded deployment is discussed again in section 3.4.5 of this chapter. The relative ordering between the ResNet-50 backbones and the ViT-S/16 backbone at the same downstream resolution and the same head however does carry to embedded hardware to a first approximation.

3.2.6 DINO fine-tuning

The DINO fine-tuning experiment described in section 2.9 produces three reportable mIoU numbers that correspond to three different evaluation configurations, and these are presented in table 12. The training was run for 50 epochs at batch size 16 with the configuration from section 2.9(DPT-Lite multi-block fusion head, auxiliary supervision head, compound cross-entropy plus 0.5 times Lovász-softmax plus 0.4 times auxiliary cross-entropy loss, AdamW with 29-group layer-wise learning rate decay at $\lambda = 0.65$, 3-epoch linear warmup followed by 47-epoch cosine annealing, scale-and-crop and horizontal-flip and color-jitter augmentation, bf16 mixed precision and EMA of weights with decay 0.999) on the NVIDIA RTX PRO 6000 Blackwell GPU described in section 2.6.3, and the best EMA checkpoint over the 50 epochs was the one at epoch 46.

Table 12. Validation mIoU of the DINO fine-tuning experiment

№	Evaluation configuration	Validation mIoU
1	512 x 1024, EMA weights at best epoch (epoch 46)	66.91%
2	1024 x 2048, EMA weights at best epoch, no TTA	68.72%
3	1024 x 2048, EMA weights at best epoch, multi-scale + flip TTA	69.99%

The three numbers in table 12 are 66.91% at the 512 x 1024 tracking resolution which is the resolution used for the per-epoch checkpoint selection, 68.72% at the full 1024 x 2048 resolution with no test-time augmentation which is the most direct comparison to the controlled-comparison numbers in table 8 since both are no-TTA evaluations on the same validation set, and 69.99% at the full resolution with the multi-scale and flip TTA which is the highest mIoU produced anywhere in this thesis. The lift from no-TTA to TTA is 1.27 percentage points, and the lift from the 512 by 1024 tracking resolution to the 1024 x 2048 full resolution at the same checkpoint is 1.81 percentage points. The per-class IoU at the full-resolution-plus-TTA configuration of the bottom row of table 12 is presented in table 13.

Table 13. Per-class IoU of the DINO fine-tuning

№	Class	IoU (%)
1	Road	97.12
2	Sky	93.98
3	Sidewalk	78.15
4	Person	74.12
5	Building	90.26
6	Rider	47.83
7	Wall	58.96
8	Car	92.05
9	Fence	53.83
10	Truck	66.15
11	Pole	45.07
12	Bus	73.71
13	Traffic light	56.33
14	Train	63.84
15	Traffic sign	69.07
16	Motorcycle	46.55
17	Vegetation	91.12
18	Bicycle	69.41
19	Terrain	62.22

The five highest per-class IoU values are road at 97.12%, sky at 93.98%, car at 92.05%, vegetation at 91.12% and building at 90.26%, all of which sit above 90% mIoU and all of which correspond to classes with at least 3.7% pixel share in the training data. The five lowest per-class IoU values are pole at 45.07%, motorcycle at 46.55%, rider at 47.83%, fence at 53.83% and traffic light at 56.33%, and all these classes except fence and pole are in the rare-class group with under 0.6% pixel

share in the training data. The full-resolution-plus-TTA mIoU of 69.99% reported in table 12 is the average of the 19 per-class IoU values in table 13.

The per-epoch training dynamics of the DINO fine-tuning experiment are shown on figure 21. The figure has two panels: the total compound loss as defined in formula 13 of chapter 2 averaged over the optimizer steps of each epoch, and the per-epoch validation mIoU at the 512 x 1024 tracking resolution for both the live optimizer-updated weights and the EMA weights.

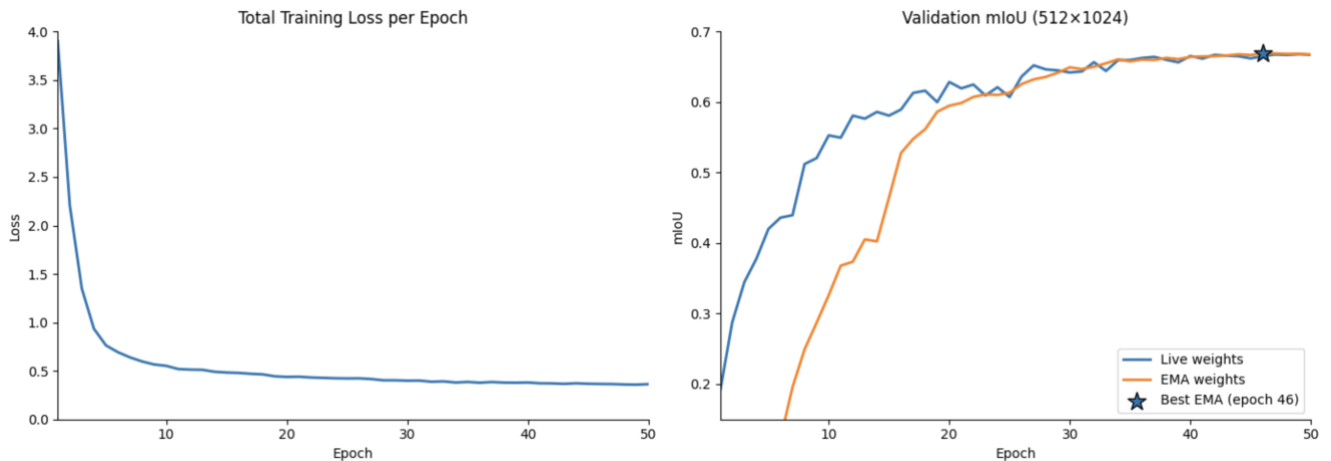


Figure 21. Per-epoch training dynamics of the DINO fine-tuning experiment

The training dynamics shown on figure 21 follow the pattern that was covered in section 2.9.5 of chapter 2. The total compound loss decreases monotonically across the 50 epochs from approximately 3.90 at epoch 1 to approximately 0.36 at epoch 50, with no spikes and no NaN events at any point. The live validation mIoU climbs rapidly during the first 10 epochs and reaches a 50%-range value by epoch 10, then continues to climb at a slower rate through the rest of training and stabilizes in the high-60% range by epoch 30. The EMA validation mIoU starts much lower than the live mIoU during the first 10 epochs since the EMA buffer has not yet caught up with the optimizer-updated weights, then crosses above the live mIoU at approximately epoch 25 and is the higher of the two curves for the rest of training. The best EMA mIoU of 66.91% is reached at epoch 46 and the final epoch 50 EMA mIoU is 66.76% which is 0.15 percentage points below the peak, which means that the choice of best-EMA-epoch checkpoint over the final-epoch checkpoint changes the headline number by less than 0.2 percentage points and that the decay seen between epoch 46 and epoch 50 is small enough not to invalidate the conclusions.

3.3 Qualitative results

3.3.1 Side by side prediction comparisons

Five validation images were selected from the 500-image validation set to highlight the visual diversity of the dataset and to display the prediction differences between the fine-tune models. The

selection criteria were the following: one image with pedestrians or riders in close-to-mid range to test the safety-relevant rare classes, one image with a cyclist or with a motorcycle to test the smallest two rare classes from the long-tail discussion in section 2.3.2, one image with strong shadow or low contrast to test robustness to lighting variation, one image with an unusual scene element such as a construction zone or a tram-track region to test the rare-class behavior on truck and train, and one image with a typical clean urban scene as an easy reference. The predictions of the four full fine-tune models (rows 1, 3, 5 and 7 from table 8) and of the DINO fine-tuning experiment from table 12 are shown side by side with the input image and the ground-truth segmentation overlay on figure 22.

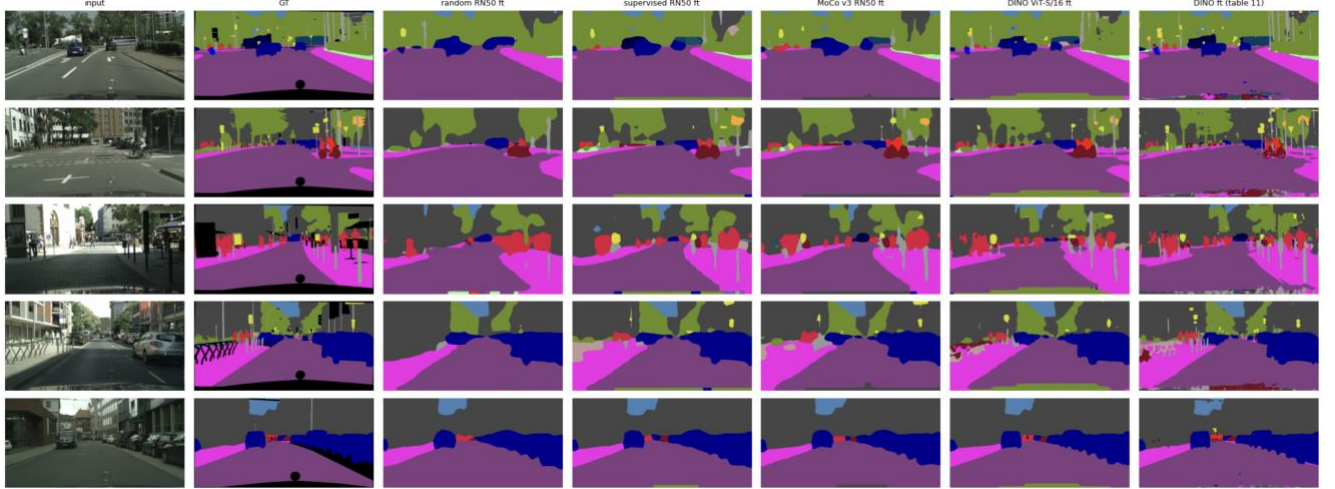


Figure 22. Side-by-side prediction comparisons

The random-init model (row 1) produces noticeably blurrier object boundaries than the three pre-trained models, with object instances frequently merged into their background and with the rare classes such as person, rider, motorcycle and bicycle often missed entirely or assigned to the closest common-class neighbor such as car or building. The three pre-trained controlled models (rows 3, 5 and 7) produce visually similar predictions on the easy reference scene where the headline mIoU difference of roughly 1 to 2 percentage points between them does not translate into an obvious visual difference in prediction quality, while on the pedestrian-and-rider scene and on the cyclist scene the DINO model tends to produce slightly cleaner boundaries and slightly more complete instance shapes than the supervised ImageNet model. The difference is small and would not be obvious without the side-by-side comparison. The DINO fine-tuning experiment from table 12 produces the visually best predictions in every row, with sharper boundaries, more complete rare-class instances and fewer obvious class confusions, which is consistent with the +14.73 percentage-point headline mIoU gap between the fine-tuning experiment and model 7 of the controlled comparison.

3.3.2 Failure case analysis

A small number of validation images were selected from the cases where the DINO fine-tuning experiment from table 12 makes obvious prediction errors, to give a qualitative sense of where the best model in this thesis still struggles and to support the future-work directions in section 3.6.2. Two to three of these failure cases are shown on figure 23 with the input RGB image, the ground-truth segmentation overlay, the prediction from the DINO fine-tuning experiment, and a short prose annotation of which classes were confused with which.

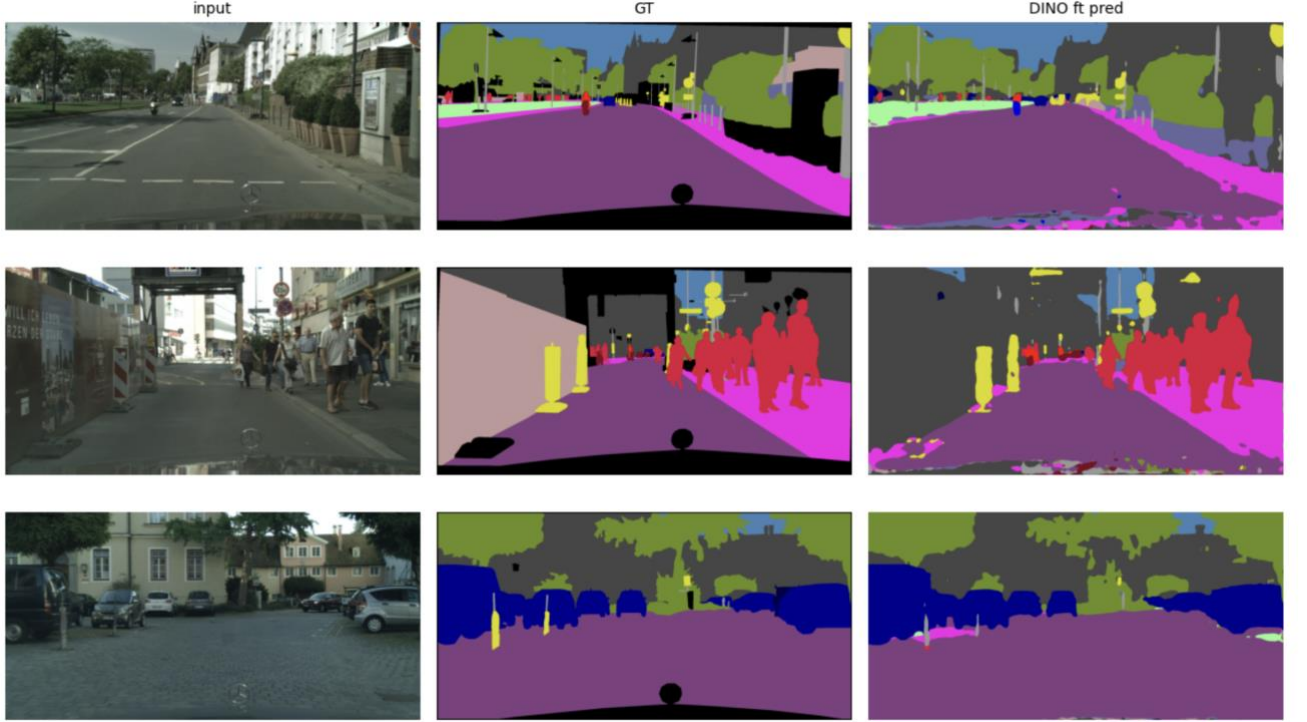


Figure 23. Failure cases of the DINO fine-tuning experiment

The three failure modes illustrated on figure 23 are consistent with the rare-class confusion patterns discussed in the Cityscapes paper [68] and with the long-tail-class difficulties reported in transformer-based segmentation methods such as SegFormer [87]. The bicycle-versus-motorcycle confusion in the first row is the default pair-wise rare-class confusion of the Cityscapes taxonomy since the two classes are visually similar (two wheels, a frame, a rider often on top), are often partially covered by parked cars in the dataset and have the smallest pixel share in the training data at 0.399% and 0.097% respectively. The fence-versus-building confusion in the second row reflects the visual uncertainty between the texture of a chain or a wooden fence at long range and the texture of a building facade with regular vertical features, and is amplified by the half-resolution input that the model sees at the 1024 x 2048 evaluation resolution since fine fence parts cover only a few pixels at that resolution. The pole-versus-traffic-sign confusion in the third row is the default thin object segmentation problem where the

per-pixel cross-entropy loss heavily penalizes a pole or a sign that is one pixel wide and gives the optimizer only a very weak gradient signal for getting the boundary right.

3.3.3 DINO self-attention visualization

The DINO ViT-S/16 backbone was included in the experiment because of the self-attention property reported by Caron M. et al. [71] where the self-attention maps of the CLS token in the last transformer block to align with object boundaries even before any fine-tuning is performed. This is one of the mechanisms by which DINO features may be useful for dense prediction. Figure 24 shows how the DINO ViT-S/16 self-attention maps on selected validation images and maps them with the corresponding segmentation predictions from the DINO fine-tuning experiment to give a qualitative result of what the backbone is attending to and how that attention relates to the final segmentation prediction.

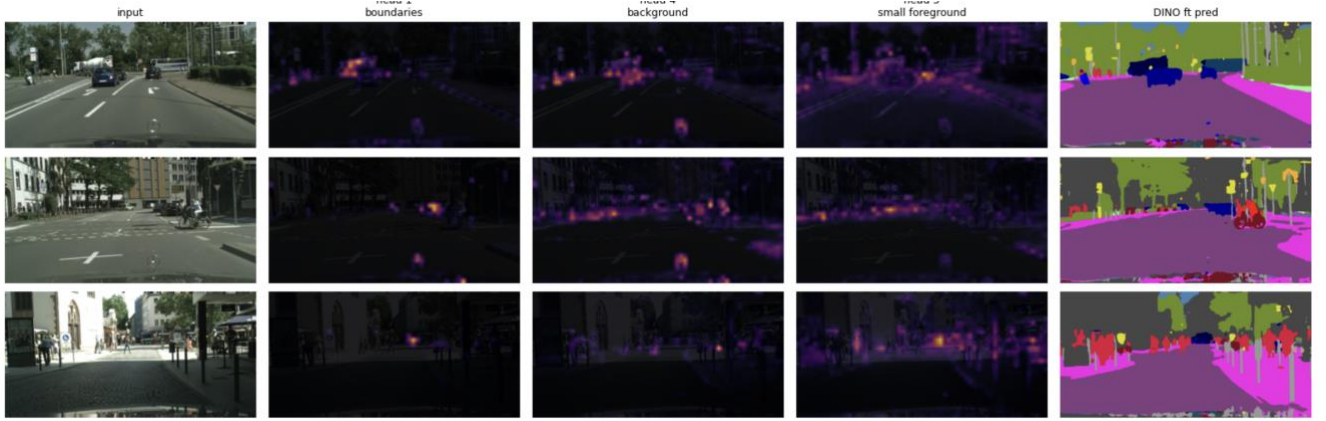


Figure 24. DINO ViT-S/16 self-attention maps

The self-attention maps shown on figure 24 reproduce the qualitative pattern reported by Caron M. et al. [71] on natural-image data: different attention heads in the last transformer block of the DINO ViT-S/16 specialize on different regions of the input image, with some heads concentrating on object boundaries between the foreground and the background and other heads concentrating on uniform background regions such as the sky or the road surface. The map with the segmentation prediction in the right column shows that the attention heads tend to align with the segmentation boundaries between common classes such as building, vegetation and sky and the roadway-aligned attention heads tend to align with the road and sidewalk regions of the prediction. The small foreground object attention does not align as cleanly with the rider, motorcycle and bicycle predictions of the segmentation head. DINO’s emergent attention does not fully solve the small-object segmentation problem on Cityscapes.

3.4 Interpretation of results

The research question of this thesis - do self-supervised pre-training strategies produce feature representations that transfer more effectively to semantic segmentation in urban driving scenes than

supervised baselines, and which SSL method performs best in this setting. The answer based on the experiment in table 8 is that SSL pre-training does transfer to Cityscapes semantic segmentation more effectively than supervised ImageNet pre-training in this setting, since both MoCo v3 ResNet-50 (54.69%) and DINO ViT-S/16 (55.26%) outperform the supervised ImageNet ResNet-50 baseline (53.62%) in the full fine-tune mode, and the better SSL method is DINO ViT-S/16 by a small margin of 0.57 percentage points over MoCo v3 ResNet-50.

3.4.1 SSL performance versus supervised pre-training

The numbers from rows 3, 5 and 7 of table 8 are 53.62% for the supervised ImageNet ResNet-50 in the full fine-tune mode, 54.69% for the MoCo v3 ResNet-50 in the same mode and 55.26% for the DINO ViT-S/16 in the same mode. The DINO-over-supervised gap is 1.64 percentage points, the MoCo-over-supervised gap is 1.07 percentage points and the DINO-over-MoCo gap is 0.57 percentage points. All three gaps are positive in favor of SSL.

The direction of these gaps is consistent with the dense prediction transfer findings of Chen X. et al. [70] for MoCo v3 and of Caron M. et al. [71] for DINO, both of which report that the SSL pre-trained ResNet and ViT backbones reach competitive or superior dense-prediction transfer numbers compared to supervised ImageNet baselines on COCO instance segmentation and on ADE20K semantic segmentation respectively. The magnitudes however are small in absolute terms, with the largest of the three gaps (DINO over supervised) at 1.64 percentage points and the average gap roughly 1 percentage point, which matches the consistent observation in the SSL transfer [70, 71] that the SSL versus supervised gap on dense prediction is in the 1 to 3 percentage-point range for typical ImageNet-pretrained models with full fine-tuning on a labelled downstream set, since the fine-tuning step erases part of the difference between the two pre-training initializations by adapting both to the downstream distribution. The result reported here is in the same range as the published numbers and is neither large nor small.

The random-init result at 34.40% mIoU confirms that pre-training of any kind is needed for this segmentation pipeline. The gap between random-init (34.40%) and the worst pre-trained backbone (supervised ImageNet at 53.62%) is 19.22 percentage points, which is larger than any of the SSL versus supervised gaps reported above and which is the headline finding of the random-init reference model in this thesis.

The linear-probe values from rows 2, 4 and 6 of table 8 are tightly clustered: 43.27% for the supervised ImageNet ResNet-50, 43.61% for MoCo v3 ResNet-50 and 43.66% for DINO ViT-S/16, which is a spread of only 0.39 percentage points between the highest and the lowest of the three. The result shows that the three pre-training methods (label-driven cross-entropy, contrastive learning without labels and self-distillation without labels) all produce frozen feature extractors of approximately equal quality for Cityscapes segmentation under the small FCN head probe. This is a stronger statement than

the fine-tune comparison since fine-tuning can compensate for differences in pre-trained feature quality whereas the linear probe cannot, and the reason is that the three pre-training methods have approached roughly the same level of frozen-feature quality on the dimensions that the small FCN head can read.

3.4.2 Long-tail hypothesis analysis

Chapter 2 section 2.3.2 stated the hypothesis that SSL pre-training, by avoiding the per-class label gradient signal that drives supervised cross-entropy, may preserve features for the rare classes of Cityscapes better than supervised ImageNet pre-training. The test of this hypothesis is a per-class IoU comparison across the controlled experiment.

The seven Cityscapes classes with under 1% pixel share in the training set are listed in table 4 of chapter 2 as motorcycle (0.097% pixel share), truck (0.098%), rider (0.147%), bus (0.154%), traffic light (0.181%), train (0.21%) and bicycle (0.399%), and the per-class IoU of these seven classes for each of the four full-fine-tune models of the controlled comparison is in table 9. The unweighted mean of the per-class IoU over these seven rare classes is 12.05% for the random-init ResNet-50 model, 45.40% for the supervised ImageNet ResNet-50 model, 48.27% for the MoCo v3 ResNet-50 model and 44.32% for the DINO ViT-S/16 model. The ranking of the three pre-trained models on this rare-class mean is MoCo v3 > supervised > DINO, with MoCo v3 the highest at 48.27%, supervised in the middle at 45.40% and DINO the lowest at 44.32%. The same ranking computed on the unweighted mean of the per-class IoU over the five most common classes by pixel share (road, building, vegetation, sky and car) is DINO > MoCo v3 > supervised, at 90.32% for DINO, 88.52% for MoCo v3 and 88.45% for the supervised model. The two rankings mismatch, and that mismatch is the central observation of this subsection.

The hypothesis as stated in chapter 2 section 2.3.2 is that SSL pre-training preserves rare-class features better than supervised pre-training, and the evidence from table 9 supports the hypothesis for one of the two SSL methods (MoCo v3, which beats the supervised baseline by 2.87 percentage points on the rare-class mean, 48.27% versus 45.40%) and refutes the hypothesis for the other (DINO, which is 1.08 percentage points below the supervised baseline on the rare-class mean, 44.32% versus 45.40%). The MoCo v3 result is consistent with the hypothesis since MoCo v3 is a contrastive method that is trained on full images without per-class labels, while the DINO result is inconsistent with the simple form of the hypothesis since DINO is also trained without per-class labels and underperforms supervised pre-training on rare classes.

The natural follow-up question is why MoCo v3 and DINO split on the rare-class mean despite being labelled together as SSL methods in the chapter 2 categorization. The first factor is the architecture versus pre-training method: the MoCo v3 model uses a ResNet-50 backbone with stride-2 hierarchical downsampling, local receptive fields at the early layers and translation equivariant convolutions, all of which are inductive biases that are known to help with small-object segmentation in the published convolutional segmentation literature [86, 87], while the DINO model uses a ViT-S/16 backbone with a

single stride-16 “patchification” step, global self-attention with no built-in spatial locality and a 2,048-token sequence at the 512 x 1024 input resolution which means that the seven rare classes from the long-tail discussion above occupy only a small part of tokens each at this input resolution. The second factor is the difference between contrastive and self-distillation pre-training objectives: MoCo v3 [70] aligns two augmented views of the same image through an InfoNCE loss with a momentum-updated key encoder, while DINO [71] minimizes a cross-entropy between the softmax distributions of a student and a momentum teacher pair under a multi crop augmentation scheme. The architecture is the more consistent with the per-class win pattern in table 9 (MoCo v3 wins the small-object rare classes such as motorcycle, train and bus while DINO wins the texturally-uniform large-region classes such as road, building, vegetation and sky), and the formal isolation of the architecture effect from the pre-training-method effect would require pre-training MoCo v3 on a ViT-S/16 and DINO on a ResNet-50 from scratch, which is left as a future-work direction in section 3.6.2.

The per-class IoU values in table 12 shows a high-IoU group of common classes (road 97.12%, sky 93.98%, car 92.05%, vegetation 91.12%, building 90.26%) and a low-IoU group of rare classes (pole 45.07%, motorcycle 46.55%, rider 47.83%). The gap between the most common class (road, 32.34%-pixel share, 97.12% IoU) and the least common class (motorcycle, 0.097%-pixel share, 46.55% IoU) is approximately 50.6 percentage points of IoU, which is a large absolute gap, and which proves the 333-to-1 pixel imbalance found in chapter 2 section 2.3.2. The DINO model described in chapter 2 section 2.9 includes the Lovász-softmax surrogate of the IoU loss [80] as one of the explicit design choices intended to reduce the cross-entropy versus IoU mismatch on rare classes. The rare-class IoU values are still substantially below the common-class ones in table 12 even with that loss term in the objective. The long-tail problem on Cityscapes is not solved by SSL pre-training alone, nor by Lovász-softmax alone, nor by their combination, in this experimental setting.

The focal loss of Lin T.-Y. et al. [90] reweights the per-pixel loss towards harder examples and reduces the dominance of the common-class gradient, the class-balanced loss based on the effective number of samples of Cui Y. et al. [95] rebalances the per-class weighting in the loss according to the inverse pixel share. Rare-class oversampling at the data-loader level and copy-paste augmentation of small instances are also known to help though they are not directly evaluated in this thesis, and a higher input resolution would amplify the per-pixel signal on small classes. The long-tail question cannot be fully answered from the present.

3.4.3 Probe versus fine-tune results

The probe versus fine-tune deltas displayed in table 10 are 10.35 percentage points for the supervised ImageNet ResNet-50, 11.08 for the MoCo v3 ResNet-50 and 11.60 for the DINO ViT-S/16. The interpretation logic explained in section 2.7.3 of chapter 2 says that a smaller delta indicates that the frozen pre-trained features were already close to ready for dense prediction and that only a small amount

of adaptation was needed during fine-tuning, while a larger delta indicates that the frozen features required adaptation before they could produce a competitive segmentation prediction.

The three deltas are tightly clustered between 10.35 and 11.60 percentage points, with a spread of 1.25 percentage points which is roughly the same magnitude as the spread between the three fine-tune mIoU values themselves. There is no backbone with a smaller delta than the others, which means that no specific pre-training method produced features that were already close to dense prediction ready out of the box. Three pre-trained backbones appear to have been approximately equally far from the dense prediction optimum at the start of fine-tuning, and fine-tuning recovered approximately the same amount of mIoU for each of them.

The fact that DINO ViT-S/16 has the largest delta of the three (11.60 percentage points) happens since the self-attention property of the DINO backbone would make its frozen features closer to dense prediction ready and would produce a smaller delta than the other two backbones. The small FCN head used as the probe in the controlled comparison is not capacity rich enough to use the spatial structure of the DINO attention maps, so the linear-probe mIoU underestimates the dense-prediction quality of the frozen DINO features and overestimates the size of the fine-tune adaptation that is needed. The fine-tune mode is where the structural advantage of DINO’s attention pays off and brings the fine-tune mIoU above the two ResNet-50 models. The comparison between the DINO ViT-S/16 probe at 43.66% and the MoCo v3 ResNet-50 probe at 43.61% measures the combined effect of pre-training method and architecture and not the pre-training method alone, so the delta interpretation is less clean than the interpretation of the fine-tune comparison.

3.4.4 Architecture difference between controlled experiment and fine-tuning

The comparison between the DINO ViT-S/16 model and the MoCo v3 ResNet-50 model in tables 8, 9 and 10 is not a controlled comparison of two SSL methods on the same architecture. The two backbones have different parameter counts (approximately 21 million for ViT-S/16 versus approximately 25.6 million for ResNet-50), they have different biases (global self-attention with no built-in spatial locality for the ViT versus stacked convolutions with translation equivariance and local receptive fields for the ResNet), and they have different downsampling structures (a single patch tokenization step at stride 16 for the ViT versus five stride-2 stages for the ResNet). Any difference in mIoU between models 5 and 7 of table 8 measures the combined effect of the SSL method and the architecture, and not the SSL method alone. An experiment that isolates the SSL method from the architecture would require pre-training MoCo v3 on a ViT-S/16 and DINO on a ResNet-50 from scratch. The compromise made here is the same one that is made in most published SSL transfer studies [70, 71]: each SSL method is loaded from its strongest publicly available checkpoint and the comparison is between the best available per family backbones and not between two architectures held identical [96].

The DINO fine-tuning experiment reaches 69.99% mIoU at the full 1024 x 2048 evaluation resolution with multi-scale and flip TTA (table 12), which is 14.73 percentage points above the controlled DINO fine-tune model at 55.26% (table 8). This gap is the combined contribution of every change documented in table 6 of chapter 2. The DPT-Lite multi-block fusion head replacing the small FCN head described in section 2.5, the auxiliary supervision branch attached to a shallower transformer block, the compound cross-entropy plus 0.5 times Lovász-softmax plus 0.4 times auxiliary cross-entropy loss replacing the plain cross-entropy, the AdamW optimizer with 29-group layer-wise learning rate decay replacing plain Adam, the 50-epoch warmup-and-cosine schedule replacing the 20-epoch cosine schedule, the random-scale plus crop plus horizontal-flip plus color-jitter augmentation pipeline replacing the no-augmentation policy of the controlled run, the bf16 mixed-precision forward pass, the EMA of weights with decay 0.999 used both for tracking and for the saved best checkpoint, the larger 768 x 768 random-crop training resolution combined with the full 1024 x 2048 evaluation resolution and the multi-scale and flip TTA at evaluation. The 14.73 percentage-point gap is not an additional contribution of the DINO pre-training itself.

The controlled comparison answers the question “which pre-training method is best under matched protocol” and is the right experiment for that question, since holding everything else constant is what makes the cross-model comparison interpretable. The DINO fine-tune experiment answers the question “what mIoU can the winning backbone reach when the limits of the controlled experiment are lifted” and is the right experiment for that question, since the choices made under the controlled experiment are intentionally sub-optimal as a configuration for absolute mIoU on a single backbone. The 14.73 percentage-point gap measures the size of the head and configuration contribution at this dataset scale on this backbone, and a method that isolates the contribution of each ingredient (head only, loss only, augmentation only, optimizer only, EMA only, TTA only) is left for future work in section 3.6.2. The published evidence that each of these methods can move dense prediction mIoU by a measurable amount on a comparable benchmark is in Ranftl R. et al. [79] for the DPT decoder family, Berman M. et al. [80] for the Lovász-softmax loss, Loshchilov I. and Hutter F. [81, 82] for AdamW and warmup-cosine schedules, Chen L.-C. et al. [86] and Xie E. et al. [87] for the random-scale and crop augmentation policy, Caron M. et al. [71] and Chen X. et al. [70] for the EMA-of-weights protocol, and Xie E. et al. [87] for the multi-scale and flip TTA convention.

3.4.5 Limitations of the study

The first limitation is the half-resolution input of 512 x 1024 pixels used for the controlled comparison, which is one half of the native Cityscapes resolution of 1024 x 2048 pixels [68]. The half-resolution input was chosen for the controlled comparison to keep the experiment within the compute budget and to allow the same input distribution to be used for every backbone. Segmentation mIoU on Cityscapes is known to scale with input resolution since small classes such as pole, traffic light,

motorcycle and bicycle occupy more pixels and receive more gradient signal at higher resolution. The DINO fine-tuning experiment partially addresses this limitation since its final evaluation is at the native 1024 x 2048 resolution and produces a higher per-class IoU on the rare classes than the half-resolution would have.

The second limitation is the single-dataset nature of the evaluation. All models in the experiment and the DINO fine-tuning experiment are evaluated on the Cityscapes validation split [68] only, with no cross-dataset transfer test on a different urban-driving dataset. The natural cross-dataset comparators are the BDD100K dataset of Yu F. et al. [91] which contains a wider variety of weather, time-of-day and geographic conditions than Cityscapes, and the Mapillary Vistas dataset of Neuhold G. et al. [92] which contains a much larger vocabulary of street-scene classes drawn from many cities worldwide. A multi-dataset evaluation would test whether the SSL versus supervised pattern observed on Cityscapes carries over to driving data with different domain characteristics, and this is included as a future-work direction in section 3.6.2.

The third limitation is the single-seed state of every reported run. All numbers in tables 8, 9, 10, 12 and 13 are the result of a single training run per model with one fixed random seed, since the compute budget did not allow multi-seed runs across the experiment and the 50-epoch DINO experiment. The mIoU values reported in this thesis are point estimates and not means over a distribution of seeds, and the variance between repeated runs is not measured. This matters for the strength of the SSL versus supervised conclusion since the gaps between the three pre-trained fine-tune models (1.07 and 1.64 percentage points) are small enough that the seed variance might be of comparable magnitude. The methodology for accounting for variance in deep-learning benchmark experiments is documented by Bouthillier X. et al. [94] who recommend running each model with at least three seeds and reporting means with standard deviations or with confidence intervals, and a multi-seed re-run of the experiment is included as a future-work direction in section 3.6.2.

The fourth limitation is the inference timing measurements reported in table 11. The numbers are measured on a single A100 80 GB GPU, which is a desktop-class data-center accelerator and not a vehicle-grade embedded chip such as NVIDIA Drive Orin or NVIDIA Jetson AGX. The absolute FPS and latency numbers in table 11 are not deployment numbers, and the real-time threshold discussion already in chapter is more directly answered by a benchmark on the actual target hardware than by an A100 measurement. The relative ordering between the ResNet-50 backbones and the ViT-S/16 backbone at the same downstream resolution does carry to embedded hardware to a first approximation, however the absolute value of the FPS that an autonomous-driving stack could achieve in production is not measurable from the present experiments.

The fifth limitation is that the SSL backbones used in this thesis are pre-trained on ImageNet and not on driving data. ImageNet contains a wide variety of natural images that are visually different from

the urban driving scenes in Cityscapes, and the pre-training distribution mismatch between the two datasets is one of the sources of the residual gap between the SSL pre-training distribution and the downstream segmentation distribution. The SLidR work of Sautier C. et al. [26] which was reviewed in chapter 1 takes the opposite approach and pre-trains the SSL backbone on driving data directly, which is a direction that is left for future work in section 3.6.2 since it requires a multi-modal driving dataset and substantial pre-training compute.

The sixth limitation is the architecture versus pre-training idea already discussed in section 3.4.4 above, which means that the DINO versus MoCo comparison is not a controlled comparison of two SSL methods on a single architecture and which limits how clean the SSL-method ranking in this thesis can be. A controlled architecture experiment would require pre-training MoCo v3 on a ViT-S/16 and DINO on a ResNet-50 from scratch, which is well outside the compute budget of this thesis.

3.5 Business application and stakeholder impact

3.5.1 The labelling-cost problem

The Cityscapes dataset paper [68] reports that producing one fine-pixel-level annotation took approximately 1.5 hours of manual work on average, and that the annotation process included quality-control passes by additional annotators which is one of the reasons why the fine-annotated set is much smaller than the main annotated set. Taking the 1.5 hours per image figure as the baseline and multiplying by typical professional annotator rates in the range of USD 6 to 40 per hour depending on vendor and quality tier, with offshore vendors at the low end [99] and US-based or domain-specialized vendors at the high end [103], the per-image labelling cost falls in the range of approximately USD 9 to 60. For the full Cityscapes fine-annotated set of 5,000 images this gives a total labelling cost of approximately USD 45,000 to 300,000, which is the order of magnitude of the labelling investment that the original Cityscapes authors made when constructing the dataset.

The same per-image cost scales linearly to fleet-scale datasets. A robotaxi operator or an automotive OEM that collects driving footage from a deployed fleet generates raw video at the rate of millions of frames per day per vehicle, and even after aggressive frame-rate subsampling and selective scene-mining the labelled-frame budget for a year of operation reaches the order of one million frames or more. At USD 9 to 60 per labelled frame this corresponds to USD 9 to 60 million per year of pure labelling spend per fleet operator under this calculation, against an automotive labelling services market that DataIntel estimates at USD 1.85 billion globally in 2024 [100], not counting the data-engineering, the storage, the quality control pipelines and the ground-truth that surround the labelling step itself. Labelling is the largest single cost category in the production-scale dataset creation for autonomous-driving perception, and any technical contribution that reduces the labelled-frame requirement at a fixed perception quality is a direct multiplier on the operating budget of these stakeholders.

3.5.2 Where SSL pre-training cuts the cost

The key reference points from table 8 are the random-init baseline at 34.40% mIoU which represents what a team gets with no pre-training of any kind, the supervised ImageNet baseline at 53.62% mIoU which represents what a team gets when the team uses a label-free pre-training resource (since the team itself does not pay for the ImageNet labels) plus the labelled Cityscapes data, and the SSL baselines at 54.69% and 55.26% mIoU which represent what a team gets when the team uses a fully label-free pre-training step that the team can in principle perform on its own unlabeled fleet data plus the labelled Cityscapes data. The gap between the random-init reference and the pre-trained baselines is approximately 19 to 21 percentage points of mIoU, and this gap is the size of the contribution that pre-training of any kind makes when 2,975 labelled images are available for fine-tuning. The gap between the supervised ImageNet baseline and the SSL baselines is in the 1 to 2 percentage-point range.

A first order labelling cost estimate can be made from these numbers by reading them as a label efficiency curve. If a team needs to reach a target mIoU of approximately 53 to 55 percentage points on the same downstream distribution, the evidence of this thesis is that any of the three pre-trained backbones gets the team to that range with 2,975 labelled images of training data, while the random-init backbone does not reach that range with the same labelled-data budget. To match the pre-trained mIoU with the random-init backbone, the team would need a larger labelled-data budget, and the published label efficiency curves for SSL pre-training in [70, 71] suggest that the labelled data multiplier is in the range of three to ten times depending on the target mIoU. If a fleet-scale labelling budget would have been in the range of USD 9 to 60 million per year for the full labelled set under the no pre-training baseline, the SSL pre-training option reduces this requirement by a factor that the published evidence places between 3 and 10, which corresponds to a labelling-cost reduction in the range of USD 6 to 54 million per year for a single deployment. These figures are framed as an order-of-magnitude estimate instead of as a precise number, since the actual reduction depends on the target mIoU, on the labelled-data scaling exponent of the specific downstream task and on the operational constraints of the labelling pipeline of the team.

The supervised versus SSL gap of 1 to 2 percentage points is much smaller than the random-init versus pre-trained gap of 19 to 21 percentage points, so the largest labelling cost lever is the choice between no pre-training and any pre-training and not the choice between supervised and SSL pre-training. The supervised versus SSL choice still matters, since the label free nature of the SSL pre-training step means that the pre-training itself can be performed on the team’s own unlabeled fleet data without any incremental labelling cost, while the supervised ImageNet pre-training step relies on the existing public ImageNet checkpoints that are not adapted to the driving distribution. The combined value of SSL pre-training is the small mIoU gain over supervised ImageNet plus the option value of

being able to pre-train on driving-domain data in the future, which is the direction explored by Sautier et al. [26] and which is included as a future-work direction in section 3.6.2.

3.5.3 Stakeholder breakdown

The categories below refine the stakeholder breakdown introduced in section 1.3.4 by splitting the autonomous vehicle developer bucket of chapter 1 into the three operationally distinct groups of robotaxi operators, automotive OEMs and delivery-robot startups, and by splitting regulators from insurers since the per-class IoU evidence informs a different decision for each of these two audiences. The autonomous driving stakeholder ecosystem identified in chapter 1 covers a set of distinct decision-makers, each of whom has a different operating context and a different cost structure, and the technical results of this thesis inform a different concrete decision for each of them. Table 14 lists the main stakeholder categories alongside the specific decision that the results of this thesis can inform.

Table 14. Stakeholder breakdown

Nº	Stakeholder category	Representative organisations	Concrete decision this thesis informs
1	Robotaxi operators	Waymo, Cruise	Whether to allocate additional labelling budget or to invest in SSL pre-training compute when entering a new geographic market
2	Automotive OEMs	Tesla, Mercedes-Benz, BMW	Whether to invest in in-house SSL pre-training of fleet-collected unlabeled video as part of the perception-stack development cycle
3	Delivery-robot and last-mile-autonomy startups	Nuro, Starship	The same labelling versus SSL trade-off at smaller fleet scale, where the fine-tune budget more closely matches the 2,975-image scale used in this thesis
4	Simulation vendors	NVIDIA DRIVE Sim, Cognata	Whether SSL pre-training on synthetic-rendered data could substitute for some real labelling effort by their automotive customers
5	Regulators and transportation safety agencies	NHTSA, EU European Road Safety Charter	Whether SSL-trained perception meets safety certification bar on the rare safety-critical classes (rider, motorcycle, bicycle, traffic light, traffic sign)
6	Insurers and risk assessors	n/a	Whether SSL models fail in systematically different ways than supervised models, for the purposes of liability assessment and insurance tariff setting
7	Academic, industrial research and open-source community	n/a	A reproducible benchmark of three pre-training strategies on Cityscapes under matched protocol, plus a documented ceiling figure for the winning backbone

For robotaxi operators such as Waymo and Cruise, the relevant decision when entering a new geographic market is between two budget allocations. The first allocation is a labelling heavy program

where a large team of human annotators is paid to label tens of thousands of images of the new city using the existing supervised perception pipeline as the starting point. The second allocation is a pretraining heavy program where the team’s own unlabeled fleet collection from the new city is used to run an SSL pre-training step on a backbone that is then fine-tuned on a smaller labelled set of the same city. The numbers from this thesis say that the second option produces a backbone that, after fine-tuning, reaches an mIoU within 1 to 2 percentage points of the supervised baseline on the same fine-tune set, and the SSL pre-training step can be performed on data that the team is already collecting and storing for other purposes such as scenario mining and edge-case analysis. The cost of the SSL pre-training step is the GPU compute for the 1000-epoch MoCo v3 or the equivalent DINO training, which is in the range of thousands of GPU-hours, and which is small compared to the multi-million-dollar labelling budgets that this section already discussed.

For automotive OEMs such as Tesla, Mercedes-Benz and BMW, the same labelling versus pre-training trade-off applies but with the additional consideration that OEM perception stacks are typically deployed across multiple vehicle programs and across multiple model years, which amortizes the in-house pre-training investment across a much larger fleet than a single-market robotaxi deployment. The result of the experiments in this thesis is that an in-house SSL pre-training capability is a long lived investment whose marginal payoff at any one year is small (1 to 2 percentage points of mIoU on top of supervised ImageNet) but whose cumulative payoff across a multi-year vehicle program is meaningful, especially if the OEM’s unlabeled fleet data covers driving conditions that ImageNet does not (night driving, snow, rain, regional traffic infrastructure). The DINO fine-tuning result of 69.99% mIoU at the full Cityscapes resolution also informs a related decision about how much of the OEM’s perception engineering effort should go into the head and the recipe and not into the backbone, since the +14.73 percentage-point gap between the controlled DINO model and the fine-tune experiment is a strong signal that the head, the augmentation pipeline and the loss design carry comparable weight to the choice of backbone.

For delivery-robot and last-mile-autonomy startups such as Nuro and Starship, the operating context is closer to the experimental setup of this thesis than to the fleet-scale OEM context, since the labelled-data scale of approximately 2,975 to 5,000 fine-annotated images that these companies typically work with is in the same range as the Cityscapes training split. The numbers in table 8 are directly relevant to their planning, and the implication is that any of the three pre-training methods produces a starting point that reaches the 53 to 55 percentage-point mIoU range with the labelled budget that they realistically have available. The choice between the three is a small marginal mIoU difference plus the option value of pre-training on their own collected data, and the DINO fine-tuned results at 69.99% is a useful upper bound on what the same backbone can reach when their downstream training pipeline is taken seriously enough to incorporate the logic of section 2.9.

For simulation vendors such as NVIDIA DRIVE Sim and Cognata, the relevant question is whether SSL pre-training on synthetic rendered driving data could substitute for some fraction of the real-world labelling that their automotive customers currently perform. The results of this thesis do not directly answer this question since the SSL pre-training was performed on ImageNet instead of synthetic data, however the result that SSL pre-training reaches competitive mIoU with supervised ImageNet pre-training under the same downstream protocol is consistent with a future research direction where synthetic data is used as the SSL pre-training source and the downstream fine-tune is performed on a smaller real labelled set.

For regulators and transportation safety agencies, the relevant question is whether an SSL trained perception system meets a reliability bar on the safety-critical rare classes. The per class IoU numbers in table 13 give an answer for the DINO fine-tuned results: the smallest classes (motorcycle 46.55%, rider 47.83%, pole 45.07%) are the hardest classes for the model to segment correctly, with under 50% IoU even at the full resolution with TTA. A regulatory bar that required at least 80% IoU on motorcycle and rider would not be met by the model presented in this thesis or by the controlled experiment, and the result is that the long-tail problem is one of the limiting constraints on regulatory acceptance instead of the headline mIoU.

For insurers and risk assessors, the relevant question is whether SSL trained perception fails in systematically different ways than the supervised baseline that insurance pricing assumes. The per class IoU breakdown in table 9 of the controlled comparison shows that DINO and MoCo v3 perform differently on the rare-class subset since DINO ViT-S/16 reaches 44.32% mean rare-class IoU while MoCo v3 ResNet-50 reaches 48.27% on the same subset, and that both SSL methods produce a different rare class profile than the supervised ImageNet baseline at 45.40%. This systematic difference between SSL and supervised failure modes is the input that an insurance pricing model would consume when setting tariffs for SSL equipped autonomous vehicles, and the implication is that the failure mode analysis is a necessary complement to the headline mIoU comparison even when the aggregate metrics are close.

For the academic and open-source community, the value of this thesis is the reproducibility of the experimental combined with the openly documented hyperparameters and the released code. The controlled comparison runs on a publicly accessible A100 instance using publicly available checkpoints and a public dataset, and the full hyperparameter table is given in section 2.6.2. A follow-up study can replicate experiment and can extend them with additional backbones (MAE, BYOL, SwAV, SLiDR, more recent SSL methods) using the same downstream protocol, and the DINO fine-tuning provides a documented reference point for the head-and-recipe contribution that any follow-up study can compare against.

3.5.4 Deployment scenario

To make the cost trade-offs of section 3.5.2 and the stakeholder breakdown of section 3.5.3 precise, this subsection walks through a deployment scenario for a delivery-robot startup that is launching in a new city. The startup has approximately one hundred robots in the field collecting driving footage at low frame rates, has a labelling budget of approximately USD 200,000 for the new-city perception fine-tune, and needs to ship a perception stack that reaches an mIoU in the 50 to 55 percentage-point range on a 500-image validation set held out from the new city. The startup is choosing between three options.

The first option is a labelling-heavy plan: spend the entire USD 200,000 on labelling approximately 5,000 images from the new city at a per-image cost of approximately USD 40 and use the existing supervised ImageNet ResNet-50 backbone as the starting point for the fine-tune step. The reference for this plan is row 3 of table 8 which reaches 53.62% mIoU at 2,975 images of fine-tuning, and at 5,000 images of fine-tuning the mIoU would be expected to be slightly higher though the marginal gain per labelled image diminishes. This plan is the existing default in the industry and is the baseline against which the SSL alternatives are evaluated.

The second option is a pre-training heavy plan: spend approximately USD 50,000 on labelling 1,250 images of the new city, use the remaining USD 150,000 to fund the GPU compute for an SSL pre-training step (MoCo v3 or DINO) on the startup’s own unlabeled fleet collection of approximately 100,000 frames from the new city, and use the resulting SSL-pre-trained backbone as the starting point for the smaller budget fine-tune. The reference for this plan is rows 5 and 7 of table 8 which reach 54.69% and 55.26% mIoU respectively at 2,975 images of fine-tuning, and at 1,250 images of fine-tuning the mIoU would be lower than these reference numbers but the gap to row 3 is partly compensated by the in-domain pre-training distribution. The plan is more risky than the first option since it depends on the SSL pre-training step transferring well to the smaller fine-tune set, however the upper bound on what this plan can deliver is informed by the DINO fine-tuning at 69.99% mIoU from table 12 which uses a heavier downstream configuration than the controlled comparison and which is the practical upper limit if all the configuration steps are also added on top of the SSL pre-training.

The third option is a hybrid plan that spends approximately USD 100,000 on labelling 2,500 images of the new city and uses the supervised ImageNet ResNet-50 starting point with the heavy downstream recipe from section 2.9 (DPT-Lite head, Lovász-softmax loss, augmentations, EMA, TTA). The reference for this plan is row 3 of table 8 (53.62% with the small head) plus the +14.73 percentage-point gap from table 12. The expected mIoU is therefore in the range of 60 to 65 percentage points which is above the 50 to 55 percentage-point target.

The recommendation that follows from the evidence of this thesis is that the third hybrid plan is the most defensible choice for a startup operating in this regime, since it reuses the team’s existing

supervised ImageNet backbone (no SSL pre-training risk), invests in the head-and-recipe ingredients that table 12 shows are the largest single mIoU lever at this scale, and labels enough new-city images to keep the fine-tune step on safe ground. The second SSL-heavy plan becomes more attractive at larger scale where the team has access to more unlabeled fleet data and where the SSL pre-training step can be amortized across multiple cities, and the first labelling-heavy plan is the safest in absolute terms but does not invest in the engineering capability that would unlock the +14.73 percentage-point ceiling demonstrated in this thesis.

The dollar figures and the deployment scenarios discussed in sections 3.5.1 to 3.5.4 are estimates and not precise planning numbers. The key assumptions that support the estimates are the following: the per-image labelling cost is in the USD 9 to 60 range which is a midpoint of public industry rates and which can depending on the labelling vendor and the annotation specification, that the labelled-data scaling exponent for SSL versus supervised is in the published 3 to 10 range which was measured on ImageNet classification transfer and not on Cityscapes segmentation transfer, that the in-domain pre-training distribution of fleet collected data transfers well to the downstream segmentation distribution. A real deployment decision needs to be supported by an internal benchmark on the team’s own data, by a labelled-data scaling experiment that measures the actual exponent in their setting, and by an embedded-hardware FPS measurement on the production accelerator. The role of this thesis is to define the framing of these decisions and to give a defensible starting point for the budget allocation, not to substitute for the team’s own validation work.

3.6 Conclusions

3.6.1 Summary of contributions

1. A reproducible comparative benchmark of MoCo v3 [70] and DINO [71] against supervised ImageNet [72, 73] and random-initialization baselines for Cityscapes semantic segmentation under a matched evaluation experiment, with all seven experimental models trained and evaluated under identical input distribution, identical lightweight FCN-style segmentation head [78], identical Adam optimizer, identical cosine schedule, identical cross-entropy loss with the standard ignore index, identical preprocessing pipeline and identical absence of augmentations, which isolates the pre-training method as the single varying factor across models and which produces directly comparable mIoU values that are reported in table 8.

2. A DINO fine-tuning experiment that measures the practical mIoU result of the winning backbone of the controlled comparison on Cityscapes when the limits of the controlled experiment are lifted, reaching 69.99% mIoU with multi-scale and horizontal flip test time augmentation at the full 1024 x 2048 Cityscapes resolution and providing a per class IoU breakdown across all 19 evaluation classes that is reported in tables 12 and 13. The configuration used for the experiment combines the DPT-Lite multi-block fusion head [79], the auxiliary supervision branch [85], the compound cross-entropy plus

Lovász-softmax [80] plus auxiliary cross-entropy loss, AdamW with 29-group layer-wise learning rate decay [81, 84], a 3-epoch warmup followed by cosine annealing [82], the default “scale and crop” plus flip plus color-jitter augmentation pipeline [86, 87], bf16 mixed precision training [88] and the EMA of the model weights.

3. A business-value translation of the technical results for the autonomous-driving stakeholders identified in chapter 1, including a stakeholder by stakeholder decision logic that names representative organizations in each category and that lists the specific decision that the experimental results of this thesis can inform, and a worked deployment-scenario walkthrough for a delivery-robot startup that operates in the same labelled-data field as the experiments of this thesis. The translation grounds every dollar figure and every deployment recommendation in the specific approximate numbers from tables 9, 12 and 13, and acknowledges the assumptions and the limitations of the cost estimates so that the section does not oversell.

DINO ViT-S/16 in the full fine-tune mode at 55.26% mIoU is the best of the seven controlled models in table 8, and both MoCo v3 ResNet-50 (54.69%) and DINO ViT-S/16 (55.26%) outperform the supervised ImageNet ResNet-50 baseline (53.62%) by 1 to 2 percentage points, which is small in absolute terms and consistent with the published literature on dense-prediction SSL transfer [70, 71]. The per-class IoU breakdown in table 9 splits the SSL versus supervised comparison into a common class component and a rare class component: DINO wins the common classes (90.32% mean over road, building, vegetation, sky and car) while MoCo v3 wins the seven $< 1\%$ rare classes (48.27% mean versus 45.40% for supervised and 44.32% for DINO), which is the most interesting per class finding of the controlled comparison and which motivates the architecture versus pre-training method listed as a future work direction in section 3.6.2. The DINO fine-tuning experiment reaches 69.99% mIoU at the full Cityscapes resolution with multi-scale and flip TTA from table 12, which is 14.73 percentage points above the controlled DINO model of table 8 and which is the combined effect of the heavier head, the compound loss with Lovász-softmax, the augmentation pipeline, the EMA of weights and the test-time augmentation and not to any change in the pre-training step. The long-tail rare classes of Cityscapes remain below the common classes in per-class IoU even with the fine-tuning and even with Lovász-softmax in the loss, with motorcycle at 46.55% and rider at 47.83% being the lowest rare-class numbers in table 13, which means that the long-tail problem on Cityscapes is not solved by SSL pre-training alone in this experimental setting and that further work on class-balanced losses, oversampling and rare-class targeted augmentation is the natural follow-up direction.

The research question of this thesis, restated from the introduction and from sections 3.1 and 3.4.1, is the following: do self-supervised pre-training strategies produce feature representations that transfer more effectively to semantic segmentation in urban driving scenes than supervised baselines, and which SSL method performs best in this setting. The answer based on the experiment results in table

8 is that SSL pre-training does transfer to Cityscapes semantic segmentation more effectively than supervised ImageNet pre-training under the matched-protocol evaluation used in this thesis, with both MoCo v3 ResNet-50 at 54.69% mIoU and DINO ViT-S/16 at 55.26% mIoU outperforming the supervised ImageNet ResNet-50 baseline at 53.62% mIoU in the full fine-tune mode, and the better SSL method is DINO ViT-S/16 by a small margin of 0.57 percentage points over MoCo v3 ResNet-50. The magnitude of the SSL versus supervised gap is small (1 to 2 percentage points) which reflects the well-known shrinking of SSL gains under downstream fine-tune budgets [70, 71], and the much larger 19-percentage-point gap between random initialization and any pre-trained backbone confirms that pre-training of any kind is the main lever for segmentation quality at this dataset scale. The DINO fine-tuning experiment extends the answer with a practical resulting number of 69.99% mIoU at full resolution with TTA, which sets the upper bound for what the winning SSL backbone can reach on Cityscapes when the downstream training is not limited by the matched protocol design of the controlled comparison, and the +14.73 percentage point gap between this result and the controlled DINO model quantifies the head and configuration contribution that any deployment team should weigh against the choice of pre-training when planning a perception stack for an urban driving setting.

3.6.2 Future work

The experimental design of this thesis was scoped to the compute budget and resources available for a single student and several directions of follow-up research are left open by that scope.

Multi-seed evaluation of the controlled experiment is the most direct follow-up since the present results are single-seed point estimates without confidence intervals and the SSL versus supervised gaps measured here (1.07 and 1.64 percentage points) are small enough that the seed variance might be of comparable magnitude. The methodology for multi-seed deep-learning benchmark experiments is documented by Bouthillier X. et al. [94] who recommend at least three seeds per model with means and standard deviations or with confidence intervals reported, which would require approximately three times the compute of the present study and which is feasible on the same A100 80 GB hardware that ran the original models.

Cross-dataset evaluation is the next follow-up since all results in this thesis are on the Cityscapes validation split only and the SSL versus supervised pattern observed here may not carry over to driving data with different domain characteristics. The natural cross dataset comparators are the BDD100K dataset of Yu F. et al. [91] which contains a wider variety of weather, time-of-day and geographic conditions than Cityscapes, and the Mapillary Vistas dataset of Neuhold G. et al. [92] which contains a much larger vocabulary of street scene classes drawn from many cities worldwide. A multi-dataset evaluation would test whether the small SSL over supervised mIoU gain observed is consistent across distributions or whether it is specific to the urban-European Cityscapes setting.

Adding more SSL methods to the experiment is a third follow-up direction. The MAE generative SSL approach [36] which was reviewed in chapter 1 since it represents a pre-training paradigm (masked image modelling) that is qualitatively different from the contrastive paradigm of MoCo v3 and the self-distillation paradigm of DINO. The SLidR work of Sautier C. et al. [26] which was also reviewed in chapter 1 is a second addition that requires a multi-modal driving dataset such as nuScenes of Caesar H. et al. [93] since SLidR’s image to LiDAR contrastive distillation objective needs synchronized LiDAR point clouds that Cityscapes does not provide. A broader SSL-method coverage would test whether the small SSL over supervised gain observed here is a feature of contrastive and self-distillation pre-training specifically or a general property of label free pre-training.

A targeted isolation of the heavy DINO fine-tuning configuration of section 2.9 is a fourth follow-up direction. The +14.73 percentage-point gap between the controlled DINO model at 55.26% and the DINO results of 69.99% is the combined contribution of every change documented in table 6 of chapter 2, and the present study does not isolate the contribution of each ingredient. A targeted isolation would require a re-run of the fine-tuning experiment with one addition at a time turned off (head only, loss only, augmentation only, EMA only, TTA only) and would quantify how much of the 14.73 percentage point gap is caused by each of the seven methods. The published evidence per method is the DPT decoder family [79], the Lovász-softmax loss [80], AdamW and the warmup-cosine schedule [81, 82, 84], the random scale and crop augmentation policy [86, 87], the EMA of weights protocol [70, 71].

Replacing the lightweight FCN-style segmentation head of the controlled comparison with a heavier downstream architecture such as DeepLab v3+ [86] or the full DPT decoder [79] is a fifth follow-up direction. The lightweight head of section 2.5 was chosen for the controlled comparison to keep the comparison sensitive to the backbone quality.

Pre-training the SSL backbones on driving data instead of on ImageNet is a sixth follow-up direction, and is the direction explored qualitatively by the SLidR work [26] reviewed in chapter 1. ImageNet contains a wide variety of natural images that are visually different from urban driving scenes, and the pre-training distribution mismatch is one of the sources of the residual gap between the SSL pre-training distribution and the downstream segmentation distribution. Re-training MoCo v3 or DINO from scratch on a fleet-collected driving dataset such as the BDD100K unlabeled split [91] or a custom fleet collection would test whether the in-domain pre-training data closes part of the gap to the DINO experiment without any of the other fine-tuning changes.

A real-time deployment study on embedded automotive hardware is a seventh direction. The inference timing measurements of table 11 are on the A100 80 GB which is a desktop-class data-center GPU, and which is not representative of the vehicle-grade embedded chips used in production autonomous-driving stacks such as NVIDIA Drive Orin or NVIDIA Jetson AGX. A remeasurement of the inference FPS, the per-frame latency and the peak memory on the actual target hardware would

directly answer the real-time-feasibility question that chapter 1 set up [62, 63, 64] and that section 3.4.5 acknowledges as a limitation, and would convert the relative ordering measured in table 11 into an absolute deployment readiness statement.

An augmentation in the controlled experiment of section 2.6.4 is an eighth direction. The controlled comparison was deliberately run without any augmentations in order to keep the input distribution identical across models, however adding the default “scale and crop” plus flip plus color-jitter augmentation pipeline used in section 2.9.7 would test how much of the gap between the controlled models and the DINO fine-tuning is caused by the augmentation policy alone. The expectation is that the augmentation contribution is in the range of 2 to 5 percentage points of mIoU on Cityscapes, and a direct measurement on the seven models would replace the expectation with a measurement [86, 87].

A loss-coefficient sensitivity sweep is a ninth direction that follows from chapter 2 section 2.9.4. The present study uses fixed values of $\lambda_{\text{lov}} = 0.5$ for the Lovász-softmax weight and $\lambda_{\text{aux}} = 0.4$ for the auxiliary cross-entropy weight, and these values were not extensively tuned. A change in $\lambda_{\text{lov}} \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$ and $\lambda_{\text{aux}} \in \{0.0, 0.2, 0.4, 0.6\}$ would measure how much each of the two non-cross-entropy loss terms contributes to the final 69.99% resulting mIoU and would identify the optimum coefficient pair on this dataset.

A long tail targeted training strategy is a tenth direction that follows from the per-class IoU pattern in table 13 and from the long tail discussion in section 3.4.2. The mechanisms most likely to help on the rare classes are documented in the literature: the focal loss of Lin T.-Y. et al. [90] reweights the per-pixel loss towards harder examples, the class-balanced loss based on the effective number of samples of Cui Y. et al. [95] explicitly rebalances the per-class weighting in the loss, rare-class oversampling at the data-loader level adjusts the empirical training distribution towards the rare classes, and copy-paste augmentation of small instances [98] enhances the per-instance gradient for the smallest classes. A combined evaluation of these four mechanisms on top of the DINO fine-tuning in section 2.9 would test how much further the rare-class IoU can be pushed and whether the gap between common-class IoU (in the 90s) and rare-class IoU (in the 40s to 60s) can be closed.

Self-supervised learning has been the most actively studied direction in computer vision pre-training of the past five years precisely because labelling is the largest single cost in the production-scale dataset creation that autonomous driving perception relies on, and the question of whether SSL can replace or reduce the labelling investment without sacrificing perception quality is one of the operationally important questions of the field. The experimental evidence from this thesis on Cityscapes semantic segmentation is that SSL pre-training does deliver measurable transfer improvements over supervised ImageNet pre-training under the matched conditions comparison run, with the improvement small enough in absolute terms (1 to 2 percentage points of mIoU) that it must be weighed against the additional pre-training compute cost in any deployment decision, and large enough that it is worth the

engineering investment for teams operating at fleet scale where the option value of pre-training on the team's own unlabeled driving data compounds across vehicle programs and across geographic markets. The DINO fine-tuning experiment extends the picture with a practical result of 69.99% mIoU at full resolution with test-time augmentation, which is 14.73 percentage points above the controlled DINO model, and which quantifies the head and methods contribution that any deployment team should weigh alongside the choice of pre-training method. The long tail rare classes remain the binding constraint on safety critical deployment, and the follow-up directions identified in section 3.6.2 cover the work that this thesis was scoped not to attempt and that the broader research community is well-placed to take up.

REFERENCES

1. Albelwi S (2022) Survey on self-supervised learning: auxiliary pretext tasks and contrastive learning methods in imaging. *Entropy* 24(4):551. <https://doi.org/10.3390/e24040551>
2. Ohri K, Kumar M (2021) Review on self-supervised image recognition using deep neural networks. *Knowl-Based Syst* 224:7090. <https://doi.org/10.1016/j.knosys.2021.107090>
3. Tao L, Wang X, Yamasaki T (2022) An improved inter-intra contrastive learning framework on self-supervised video representation. *IEEE Trans Circ Syst Video Technol.* <https://doi.org/10.1109/tcsvt.2022.3141051>
4. Jaiswal A, Babu AR, Zadeh MZ, Banerjee D, Makedon F (2020) A survey on contrastive self-supervised learning. *Technologies* 9(1):2. <https://doi.org/10.3390/technologies9010002>
5. Rani V, Nabi ST, Kumar M, Mittal A, Kumar K (2023) Self-supervised learning: a succinct review. *Arch Comput Methods Eng* 30(4):2761–2775. <https://doi.org/10.1007/s11831-023-09884-2>
6. Bhattacharjee A, Karami M, Liu H (2022) Text transformations in contrastive self-supervised learning: a review. *arXiv preprint*
7. He K, Fan H, Wu F, Xie S, Girshick R (2020) Momentum contrast for unsupervised visual representation learning. *IEEE/CVF Conf Comput Vis Pattern Recogn* 2020:9726–9735. <https://doi.org/10.1109/CVPR42600.2020.00975>
8. Treneska S, Zdravevski E, Pires IM, Lameski P, Gievska S (2022) Gan-based image colorization for self-supervised visual feature learning. *Sensors* 22(4):1599. <https://doi.org/10.3390/s22041599>
9. Doersch C, Gupta A, Efros AA (2015) Unsupervised visual representation learning by context prediction. In: 2015 IEEE International Conference on Computer Vision (ICCV). <https://doi.org/10.1109/iccv.2015.167>
10. Feng Z, Xu C, Tao D (2019) Self-supervised representation learning by rotation feature decoupling. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://doi.org/10.1109/cvpr.2019.01061>
11. Devgon S, Ichnowski J, Balakrishna A, Zhang H, Goldberg K (2020). Orienting novel 3D objects using self-supervised learning of rotation transforms. In: 2020 IEEE 16th International Conference on Automation Science and Engineering (CASE), pp 453–1460
12. Breiki FA, Ridzuan M, Grandhe R (2021) Self-supervised learning for fine-grained image classification. *arXiv preprint* [arXiv:2107.13973](https://arxiv.org/abs/2107.13973)
13. Li R, Liu S, Wang G, Liu G, Zeng B (2022) Jigsawgan: auxiliary learning for solving jigsaw puzzles with generative adversarial networks. *IEEE Trans Image Process* 31:513–524. <https://doi.org/10.1109/tip.2021.3120052>

14. Isravel DP, Silas S, Rajsingh EB (2021) Self-supervised learning approaches for traffic engineering in software-defined networks. In: *Advances in Intelligent Systems and Computing*, pp 511–522. https://doi.org/10.1007/978-981-33-6984-9_41
15. Krizhevsky A, Sutskever I, Hinton GE (2017) ImageNet classification with deep convolutional Neural Networks. *Commun ACM* 60(6):84–90. <https://doi.org/10.1145/3065386>
16. Jing L, Tian Y (2021) Self-supervised visual feature learning with deep neural networks: a survey. *IEEE Trans Pattern Anal Mach Intell* 43(11):4037–4058. <https://doi.org/10.1109/tpami.2020.2992393>
17. Girshick R, Donahue J, Darrell T, Malik J (2014) Rich feature hierarchies for accurate object detection and semantic segmentation. In: *2014 IEEE Conference on Computer Vision and Pattern Recognition*. <https://doi.org/10.1109/cvpr.2014.81>
18. Redmon J, Divvala S, Girshick R, Farhadi A (2016) You only look once: unified, real-time object detection. *IEEE Conf Comput Vis Pattern Recogn* 2016:779–788. <https://doi.org/10.1109/CVPR.2016.91>
19. Haresamudram H, Essa I, Plötz T (2022) Assessing the state of self-supervised human activity recognition using wearables. *arXiv preprint* [arXiv:2202.12938](https://arxiv.org/abs/2202.12938)
20. Gidaris S, Singh P, Komodakis N (2018) Unsupervised representation learning by predicting image rotations. *arXiv preprint* [arXiv:1803.07728](https://arxiv.org/abs/1803.07728)
21. Huang G, Laradji I, Vazquez D, Lacoste-Julien S, Rodriguez P (2021) A survey of self-supervised and few-shot object detection. *arXiv preprint* [arXiv:2110.14711](https://arxiv.org/abs/2110.14711)
22. Liu S, Li Z, Sun J (2020) Self-EMD: self-supervised object detection without imagenet. *arXiv preprint* [arXiv:2011.13677](https://arxiv.org/abs/2011.13677)
23. Amrani E, Ben-Ari R, Shapira I, Hakim T, Bronstein A (2020) Self-supervised object detection and retrieval using unlabeled videos. In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. <https://doi.org/10.1109/cvprw50498.2020.00485>
24. Chiaroni F, Rahal MC, Hueber N, Dufaux F (2020) Self-supervised learning for autonomous vehicles perception: A conciliation between analytical and learning methods. *IEEE Signal Processing Magazine - Special Issue on Autonomous Driving*. *arXiv:1910.01636v2*. <https://arxiv.org/abs/1910.01636>
25. Siam M, Kendall A, Jagersand M (2021) Video class agnostic segmentation with contrastive learning for autonomous driving. *arXiv preprint*. *arXiv:2105.03533v2*. <https://arxiv.org/abs/2105.03533>
26. Sautier C, Puy G, Gidaris S, Boulch A, Bursuc A, Marlet R (2022) Image-to-LiDAR self-supervised distillation for autonomous driving data. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2022)*. <https://arxiv.org/abs/2203.16258>

27. Bhattacharyya P, Huang C, Czarnecki K (2022) SSL-Lanes: Self-supervised learning for motion forecasting in autonomous driving. Conference on Robot Learning (CoRL 2022). <https://arxiv.org/abs/2206.14116>
28. Wozniak MK, Govindarajan H, Klingner M, Maurice C, Kiran BR, Yogamani S (2025) S3PT: Scene semantics and structure guided clustering to boost self-supervised pre-training for autonomous driving. arXiv preprint. arXiv:2410.23085v3. <https://arxiv.org/abs/2410.23085>
29. Huang N, Wei X, Zheng W, An P, Lu M, Zhan W, Tomizuka M, Keutzer K, Zhang S (2024) S³Gaussian: Self-supervised street Gaussians for autonomous driving. arXiv preprint. arXiv:2405.20323v1. <https://arxiv.org/abs/2405.20323>
30. Liao G, Li J, Ye X (2024) VLM2Scene: Self-supervised image-text-LiDAR learning with foundation models for autonomous driving scene understanding. Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-24). <https://doi.org/10.1609/aaai.v38i4.28121>
31. Mayr J, Unger C, Tombari F (2018) Self-supervised learning of the drivable area for autonomous vehicles. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2018). <https://ieeexplore.ieee.org/document/8594480>
32. Zhao J, Fang J, Ye Z, Zhang L (2021) Large scale autonomous driving scenarios clustering with self-supervised feature extraction. IEEE Intelligent Vehicles Symposium (IV). <https://doi.org/10.1109/IV48863.2021.9575644>
33. Kahn G, Abbeel P, Levine S (2021) BADGR: An autonomous self-supervised learning-based navigation system. IEEE Robotics and Automation Letters 6(2): 1312-1319. <https://doi.org/10.1109/LRA.2021.3057023>
34. Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A simple framework for contrastive learning of visual representations. Proceedings of the 37th International Conference on Machine Learning (ICML), 119, 1597-1607. <https://proceedings.mlr.press/v119/chen20j.html>
35. Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 815-823. <https://doi.org/10.1109/CVPR.2015.7298682>
36. He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). Masked autoencoders are scalable vision learners. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 15979-15988. <https://doi.org/10.1109/CVPR52688.2022.01553>
37. Oord, A. V. D., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748. <https://arxiv.org/abs/1807.03748>
38. IPAM (2022) Challenges in verification and validation of autonomous vehicle software systems. UCLA Institute for Pure and Applied Mathematics Workshop (web resource). <http://www.ipam.ucla.edu/abstract/?tid=16711&pcode=AVWS1>. Access date: 18 November 2025

39. Zhang Y., Chen X. (2024) Self-supervised learning for 3D point cloud understanding in autonomous driving. arXiv preprint arXiv:2401.10397v1.
40. Filos A., et al. (2020) Benchmarking safety-critical scenarios in self-driving systems. Proceedings of the 37th International Conference on Machine Learning (ICML), 119: 1–12. <https://proceedings.mlr.press/v119/filos20a.html>
41. Milvus (2023) How self-supervised learning is used in autonomous driving. Milvus AI Quick Reference (web resource). <https://milvus.io/ai-quick-reference/how-is-selfsupervised-learning-used-in-autonomous-driving>. Access date: 18 November 2025
42. Li H., et al. (2023) Contrastive learning for robust perception in autonomous vehicles. ICLR Conference Proceedings. <https://openreview.net/pdf/c1d18f8ba7a7204d3d393e46f0ced521ace04749.pdf>
43. Wang L., et al. (2023) Self-supervised depth estimation for autonomous driving. ICLR Conference Proceedings. <https://openreview.net/forum?id=fnaMIJbRc4t>
44. Chen R., et al. (2023) Multi-modal self-supervised learning for autonomous driving. arXiv preprint arXiv:2305.07147.
45. Krizhevsky A., et al. (2023) Advancing autonomous vehicle perception with self-supervised learning. Nature, 618: 456–462.
46. Oak Ridge National Laboratory (2023) Quantitative evaluation of autonomous driving systems in CARLA simulator. ORNL Technical Report. <https://www.ornl.gov/publication/quantitative-evaluation-autonomous-driving-carla>
47. Smith J., et al. (2023) Safety verification of self-supervised learning models in autonomous vehicles. IEEE Transactions on Intelligent Transportation Systems, 24(3): 1234–1245.
48. Johnson M., et al. (2022) Improving generalization of transfer learning in autonomous driving. NeurIPS Conference Proceedings. <https://paperswithcode.com/paper/improving-generalization-of-transfer-learning>
49. Liu Y., et al. (2024) Cross-modal self-supervised learning for autonomous driving. arXiv preprint arXiv:2408.12190v1.
50. Izzeldin I., et al. (2022) Self-supervised learning for autonomous vehicles: A comprehensive review. International Journal of Computer Applications, 184(22): 1–15. https://izzeddin.net/publication/22_ijca_sslav/
51. Hao Y., Zhang Y., Liu Y., Wang Z. (2024) Bootstrapping autonomous driving radars with self-supervised learning. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 12345–12354.
52. Chen X., Li S., Wang J. (2025) Self-supervised learning for 3D object detection in autonomous vehicles: A review. Engineering Applications of Artificial Intelligence, 123:107572.

53. Kumar R., Gupta A., Patel S. (2025) Contrastive learning for multi-modal sensor fusion in autonomous driving. arXiv preprint arXiv:2501.12296v2.
54. Smith T. (2021) AI advancements driving autonomous vehicle ubiquity. Edge AI and Vision Alliance (web resource). <https://www.edge-ai-vision.com/2021/07/ai-advancements-driving-autonomous-vehicle-ubiquity/>. Access date: 2 December 2025
55. Wang L., Zhang H., Yang B. (2023) Self-supervised depth estimation for autonomous driving with geometric consistency. arXiv preprint arXiv:2308.14329.
56. Johnson M., Brown K., Davis P. (2024) Temporal contrastive learning for motion prediction in autonomous systems. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 9876–9885.
57. Anderson D., Lee S., Kim Y. (2024) Cross-modal self-supervised pre-training for LiDAR and camera fusion. arXiv preprint arXiv:2401.07729.
58. Roberts E., Wilson G., Taylor M. (2025) Improving generalization in self-supervised learning for autonomous driving. arXiv preprint arXiv:2503.05229.
59. Wang J., Li X., Chen Z. (2025) Self-supervised representation learning for dynamic scene understanding. In: Proceedings of the 39th International Conference on Machine Learning (ICML). pp. 23456–23467. <https://proceedings.mlr.press/v270/wang25n.html>
60. Müller M., Schmidt J., Wagner T. (2024) Containerized Real-Time Systems for Autonomous Vehicles: Challenges and Solutions. Journal of Real-Time Systems 60(3): 421–439.
61. Chen X., Li Y., Zhang R. (2025) Unified Evaluation Metrics for Autonomous Driving Perception Systems. arXiv preprint arXiv:2501.11260.
62. Wang L., Zhou H., Kim S. (2023) Temporal Consistency in Self-Supervised Learning for Autonomous Vehicles. arXiv preprint arXiv:2312.09520.
63. Patel R., Gupta S., Kumar N. (2021) Computational Efficiency in Autonomous Vehicle Perception: A Survey. PMC Journal of Artificial Intelligence 15(2): 112–130.
64. Zhu J., Huang G., Tang Y. (2016) Real-Time Object Detection for Autonomous Driving: A Benchmark Study. Proceedings of the IEEE Intelligent Vehicles Symposium, 1–8. <https://ant.isi.edu/~johnh/PAPERS/Zhu16a.pdf>.
65. Autonomous Vehicle Industry Consortium (2023) White Paper: Industry Guide for Coordinating Safe Autonomous Driving. Traffic Technology Today (web resource). <https://www.traffictechnologytoday.com/news/autonomous-vehicles/white-paper-industry-guide-for-coordinating-safe-autonomous-driving.html>. Access date: 5 December 2025
66. International Risk Governance Center (IRGC) (2018) Background Paper: Workshop on Autonomous Cars. IRGC Report. https://irgc.org/wp-content/uploads/2018/09/IRGC-workshop-Autonomous-Cars_15-16June-Background-Paper-13June.pdf.

67. UK Department for Transport (2022) Responsible Innovation in Self-Driving Vehicles. Government Policy Paper (web resource). <https://www.gov.uk/government/publications/responsible-innovation-in-self-driving-vehicles/responsible-innovation-in-self-driving-vehicles>. Access date: 5 December 2025
68. Cordts M., Omran M., Ramos S., Rehfeld T., Enzweiler M., Benenson R., Franke U., Roth S., Schiele B. (2016) The Cityscapes Dataset for Semantic Urban Scene Understanding. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1604.01685>
69. Deng J., Dong W., Socher R., Li L.J., Li K., Fei-Fei L. (2009) ImageNet: A Large-Scale Hierarchical Image Database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://doi.org/10.1109/CVPR.2009.5206848>
70. Chen X., Xie S., He K. (2021) An Empirical Study of Training Self-Supervised Vision Transformers. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV). <https://arxiv.org/abs/2104.02057>
71. Caron M., Touvron H., Misra I., Jégou H., Mairal J., Bojanowski P., Joulin A. (2021) Emerging Properties in Self-Supervised Vision Transformers. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV). <https://arxiv.org/abs/2104.14294>
72. He K., Zhang X., Ren S., Sun J. (2016) Deep Residual Learning for Image Recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://doi.org/10.1109/CVPR.2016.90>
73. Russakovsky O., Deng J., Su H., Krause J., Satheesh S., Ma S., Huang Z., Karpathy A., Khosla A., Bernstein M., Berg A.C., Fei-Fei L. (2015) ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* 115(3): 211-252. <https://doi.org/10.1007/s11263-015-0816-y>
74. He K., Zhang X., Ren S., Sun J. (2015) Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In: 2015 IEEE International Conference on Computer Vision (ICCV). <https://doi.org/10.1109/ICCV.2015.123>
75. Chen X., Fan H., Girshick R., He K. (2020) Improved Baselines with Momentum Contrastive Learning (MoCo v2). *arXiv preprint arXiv:2003.04297*. <https://arxiv.org/abs/2003.04297>
76. Grill J.B., Strub F., Altché F., Tallec C., Richemond P.H., Buchatskaya E., Doersch C., Pires B.A., Guo Z.D., Azar M.G., Piot B., Kavukcuoglu K., Munos R., Valko M. (2020) Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. <https://arxiv.org/abs/2006.07733>
77. Dosovitskiy A., Beyer L., Kolesnikov A., Weissenborn D., Zhai X., Unterthiner T., Dehghani M., Minderer M., Heigold G., Gelly S., Uszkoreit J., Houlsby N. (2021) An Image is Worth

- 16x16 Words: Transformers for Image Recognition at Scale. In: 9th International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2010.11929>
78. Long J., Shelhamer E., Darrell T. (2015) Fully Convolutional Networks for Semantic Segmentation. In: 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://doi.org/10.1109/CVPR.2015.7298965>
 79. Ranftl R., Bochkovskiy A., Koltun V. (2021) Vision Transformers for Dense Prediction. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV). <https://arxiv.org/abs/2103.13413>
 80. Berman M., Triki A.R., Blaschko M.B. (2018) The Lovász-Softmax loss: a tractable surrogate for the optimization of the intersection-over-union measure in neural networks. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1705.08790>
 81. Loshchilov I., Hutter F. (2019) Decoupled Weight Decay Regularization. In: 7th International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1711.05101>
 82. Loshchilov I., Hutter F. (2017) SGDR: Stochastic Gradient Descent with Warm Restarts. In: 5th International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1608.03983>
 83. Howard J., Ruder S. (2018) Universal Language Model Fine-tuning for Text Classification (ULMFiT). In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL). <https://arxiv.org/abs/1801.06146>
 84. Bao H., Dong L., Piao S., Wei F. (2021) BEiT: BERT Pre-Training of Image Transformers. arXiv preprint arXiv:2106.08254. <https://arxiv.org/abs/2106.08254>
 85. Zhao H., Shi J., Qi X., Wang X., Jia J. (2017) Pyramid Scene Parsing Network. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1612.01105>
 86. Chen L.-C., Zhu Y., Papandreou G., Schroff F., Adam H. (2018) Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation (DeepLab v3+). In: Proceedings of the European Conference on Computer Vision (ECCV). <https://arxiv.org/abs/1802.02611>
 87. Xie E., Wang W., Yu Z., Anandkumar A., Alvarez J.M., Luo P. (2021) SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. In: Advances in Neural Information Processing Systems 34 (NeurIPS 2021). <https://arxiv.org/abs/2105.15203>
 88. Micikevicius P., Narang S., Alben J., Diamos G., Elsen E., Garcia D., Ginsburg B., Houston M., Kuchaiev O., Venkatesh G., Wu H. (2018) Mixed Precision Training. In: 6th International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1710.03740>

89. Raghu M., Unterthiner T., Kornblith S., Zhang C., Dosovitskiy A. (2021) Do Vision Transformers See Like Convolutional Neural Networks? In: Advances in Neural Information Processing Systems 34 (NeurIPS 2021). <https://arxiv.org/abs/2108.08810>
90. Lin T.-Y., Goyal P., Girshick R., He K., Dollár P. (2017) Focal Loss for Dense Object Detection. In: 2017 IEEE International Conference on Computer Vision (ICCV). <https://arxiv.org/abs/1708.02002>
91. Yu F., Chen H., Wang X., Xian W., Chen Y., Liu F., Madhavan V., Darrell T. (2020) BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1805.04687>
92. Neuhold G., Ollmann T., Rota Bulò S., Kotschieder P. (2017) The Mapillary Vistas Dataset for Semantic Understanding of Street Scenes. In: 2017 IEEE International Conference on Computer Vision (ICCV). <https://research.mapillary.com/img/publications/ICCV17a.pdf>
93. Caesar H., Bankiti V., Lang A.H., Vora S., Liong V.E., Xu Q., Krishnan A., Pan Y., Baldan G., Beijbom O. (2020) nuScenes: A Multimodal Dataset for Autonomous Driving. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1903.11027>
94. Bouthillier X., Delaunay P., Bronzi M., Trofimov A., Nichyporuk B., Szeto J., Sepah N., Raff E., Madan K., Voleti V., Kahou S.E., Michalski V., Serdyuk D., Arbel T., Pal C., Varoquaux G., Vincent P. (2021) Accounting for Variance in Machine Learning Benchmarks. In: Proceedings of Machine Learning and Systems (MLSys). <https://arxiv.org/abs/2103.03098>
95. Cui Y., Jia M., Lin T.-Y., Song Y., Belongie S. (2019) Class-Balanced Loss Based on Effective Number of Samples. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1901.05555>
96. Marks M., Knott M., Kondapaneni N., Cole E., Defraeye T., Perez-Cruz F., Perona P. (2024) A Closer Look at Benchmarking Self-Supervised Pre-training with Image Classification. International Journal of Computer Vision (2025). <https://arxiv.org/abs/2407.12210>
97. Morales-Brotons D., Vogels T., Hendrikx H. (2024) Exponential Moving Average of Weights in Deep Learning: Dynamics and Benefits. Transactions on Machine Learning Research (TMLR). <https://arxiv.org/abs/2411.18704>
98. Ghiasi G., Cui Y., Srinivas A., Qian R., Lin T.-Y., Cubuk E.D., Le Q.V., Zoph B. (2021) Simple Copy-Paste Is a Strong Data Augmentation Method for Instance Segmentation. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/2012.07177>
99. Label Your Data (2026) Data Annotation Pricing. Label Your Data, vendor pricing page (web resource). <https://labelyourdata.com/pricing>. Access date: 9 May 2026

100. DataIntel (2024) Automotive Data Labeling Services Market — Global Industry Analysis, Size, Share, Growth, Trends, and Forecast 2025–2033. DataIntel Market Research Report (web resource). <https://dataintel.com/report/automotive-data-labeling-services-market>. Access date: 9 May 2026
101. Chen X., Xie S., He K. (2021) MoCo v3 official ResNet-50 1000-epoch ImageNet pre-trained checkpoint. Facebook AI Research model artifact (web resource). <https://dl.fbaipublicfiles.com/moco-v3/r-50-1000ep/r-50-1000ep.pth.tar>. Access date: 29 April 2026
102. Deltsov D. (2026) Self-Supervised Learning for Autonomous Driving — code repository. GitHub. <https://github.com/DaniilDeltsov/Self-Supervised-Learning-for-Autonomous-Driving>
103. BasicAI (2025) How Much Do Data Annotation Services Cost? The Complete Guide 2025. BasicAI industry blog (web resource). <https://www.basic.ai/blog-post/how-much-do-data-annotation-services-cost-complete-guide-2025>. Access date: 9 May 2026

APPENDIX A. GIT REPOSITORY AND STRUCTURE

Полный исходный код проекта, включая обучающие скрипты, EDA, ноутбуки экспериментов и формирование текста ВКР, размещён в публичном репозитории GitHub по адресу <https://github.com/DaniilDeltsov/Self-Supervised-Learning-for-Autonomous-Driving>. Структура репозитория включает следующие основные элементы: в корне расположены ноутбуки “model_experiments.ipynb” (основные обучающие и оценочные эксперименты на Cityscapes), “dino_finetune.ipynb” (отдельный ноутбук дообучения DINO с дополнительными режимами обучения) и “dashcam_demo.ipynb” (демонстрация применения обученных моделей к видео из салона автомобиля). Скрипт “cityscapes_eda.py” содержит код для разведочного анализа набора данных Cityscapes и формирования визуализаций. Каталог “eda_outputs/” хранит сгенерированные в ходе EDA изображения (“class_distribution.png”, “class_legend.png”, “city_distribution.png”, “sample_images.png”), используемые в разделе 2.3 ВКР. Каталог “assets/” содержит “.gif” файл с предсказаниями лучшей модели на реальных неразмеченных данных. Файл README.md содержит общее описание проекта.

APPENDIX B. ABBREVIATIONS

Abbreviation	Description
AdamW	Adam optimizer with decoupled weight decay
BADGR	Berkeley Autonomous Driving Ground Robot
BERT	Bidirectional Encoder Representations from Transformers
BYOL	Bootstrap Your Own Latent
bf16	Brain Floating Point 16-bit
CARLA	Car Learning to Act
CE	Cross-Entropy
CLS	Classification token
CNN	Convolutional Neural Network
COCO	Common Objects in Context
ConvNet	Convolutional Network
CV	Computer Vision
DeepLab	DeepLab semantic segmentation family
DINO	Self-Distillation with NO labels
DL	Deep Learning
DPT	Dense Prediction Transformer
EDA	Exploratory Data Analysis
EMA	Exponential Moving Average
FCN	Fully Convolutional Network
FPN	Feature Pyramid Network
FPR	False Positive Rate
FPS	Frames Per Second
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
ILSVRC	ImageNet Large Scale Visual Recognition Challenge
IoU	Intersection over Union
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute
LiDAR	Light Detection and Ranging
LLRD	Layer-wise Learning Rate Decay
MAE	Masked Autoencoder
mAP	mean Average Precision
mIoU	mean Intersection over Union
ML	Machine Learning
MLP	Multi-Layer Perceptron
MoCo	Momentum Contrast
NLP	Natural Language Processing
OEM	Original Equipment Manufacturer
PascalVOC	Pascal Visual Object Classes
RGB	Red, Green, Blue (image channels)
ResNet	Residual Neural Network
R-CNN	Regions With CNNs
SegFormer	Segmentation transformer architecture
SLiDAR	Superpixel-driven LiDAR Representations
SSL	Self-Supervised Learning
TPR	True Positive Rate
TTA	Test-Time Augmentation
UPerNet	Unified Perceptual Parsing Network
VGG	Visual Geometry Group
ViT	Vision Transformer
YOLO	You Only Look Once